

FULL STACK DEVELOPMENT

- Full Stack Development refers to the practice of developing both the front-end (client-side) and back-end (server-side) portions of web applications.
- A full stack developer is proficient in working with both the front-end and back-end technologies, allowing them to build complete web applications independently or as part of a team.

Technologies used in full stack development

Front-End Technologies:

- HTML (Hypertext Markup Language): Used for structuring web pages.
- CSS (Cascading Style Sheets): Used for styling the appearance of web pages.
- JavaScript: A programming language used for adding interactivity and dynamic behavior to web pages.
- Front-end frameworks/libraries such as React.js, AngularJS, or Vue.js: These provide tools and utilities for building user interfaces and managing application state.

Back-End Technologies:

- Server-side languages like JavaScript (Node.js), Python (Django, Flask), Ruby (Ruby on Rails), Java (Spring Boot), or PHP (Laravel), C#(.Net Framework), java(Servlets)
- Databases such as MySQL, PostgreSQL, MongoDB, or Firebase for storing and managing data.
- Web servers like Apache or Nginx or IIS or Tomcat or Caddy(with built in support for https) for handling HTTP requests.

Development Tools and Environment:

- Version control systems like Git for managing code changes.
- Integrated Development Environments (IDEs) such as Visual Studio Code, Sublime Text, or Atom.
- Command-line tools for tasks like package management (npm for Node.js, pip for Python, nugget for .Net), running servers, and deployment.

Important STACKS

- MEAN Stack
- MERN Stack
- Django Stack
- LAMP
- WAMP

Why Django?

- Rapid Development:- DRY (Don't repeat Yourself)
- Saves Time and Money
- Rounded Solution
- Great Exposure
- Complete Ownership
- Greater opportunity with more learning
- Security (Prevention of threats such as SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), and clickjacking by providing `{% csrf_token %}` django tag to be included inside the form.
This token generates a hidden input field containing a unique CSRF token. This token is then validated by Django when the form is submitted, ensuring that the request originated from the same site and protecting against CSRF attacks.
- Batteries Included Philosophy (Model Forms)
- Built in database (Sqlite DB)

What is Django

- Django is a free and open source web application framework which offers fast and effective dynamic website development.
- It is written using python.
- It follows MVT (model, view and template architecture)

Framework

A framework is a pre-built collection of libraries, modules, and tools that provides a structured approach to developing software applications.

- Django is a web development framework.

- AngularJS is a Web Frontend development framework
- React is a Web frontend development library.

Django Evolution

1. Write a Web application from scratch.
2. Write another Web application from scratch.
3. Realize the application from step 1 shares much in common with the application from step 2.
4. Refactor the code so that application 1 shares code with application 2.
5. Repeat steps 2–4 several times.
6. Realize you've invented a framework

Django MVT

The MVT is a software design pattern which includes three important components Model, View and Template.

- The **Model** helps to handle database. It is a data access layer which handles the data.
- The **Template** is a presentation layer which handles User Interface part completely.
- The **View** is used to execute the business logic and interact with a model to carry data and renders a template.

Characteristics of Django

- Loosely Coupled – Django helps you to make each element of its stack independent of the others.
- Less code - Ensures effective development
- Not repeated- Everything should be developed in precisely one place instead of repeating it again
- Fast development- Django's offers fast and reliable application development.
- Consistent design - Django maintains a clean design and makes it easy to follow the best web development practices.

Python Virtual Environment

- A Python Virtual Environment is an isolated space where you can work on your Python projects, separately from your system-installed Python.
- You can set up your own libraries and dependencies without affecting the system Python.
- There are no limits to the number of virtual environments
- It allows you to have multiple Python environments with different versions of Python and different sets of installed packages on the same system.
- It is generally good to have one new virtual environment for every Python-based project you work on
- You can change the system python version, django version and other dependencies without affecting the project python version, django versions and dependencies.

Experiment-01

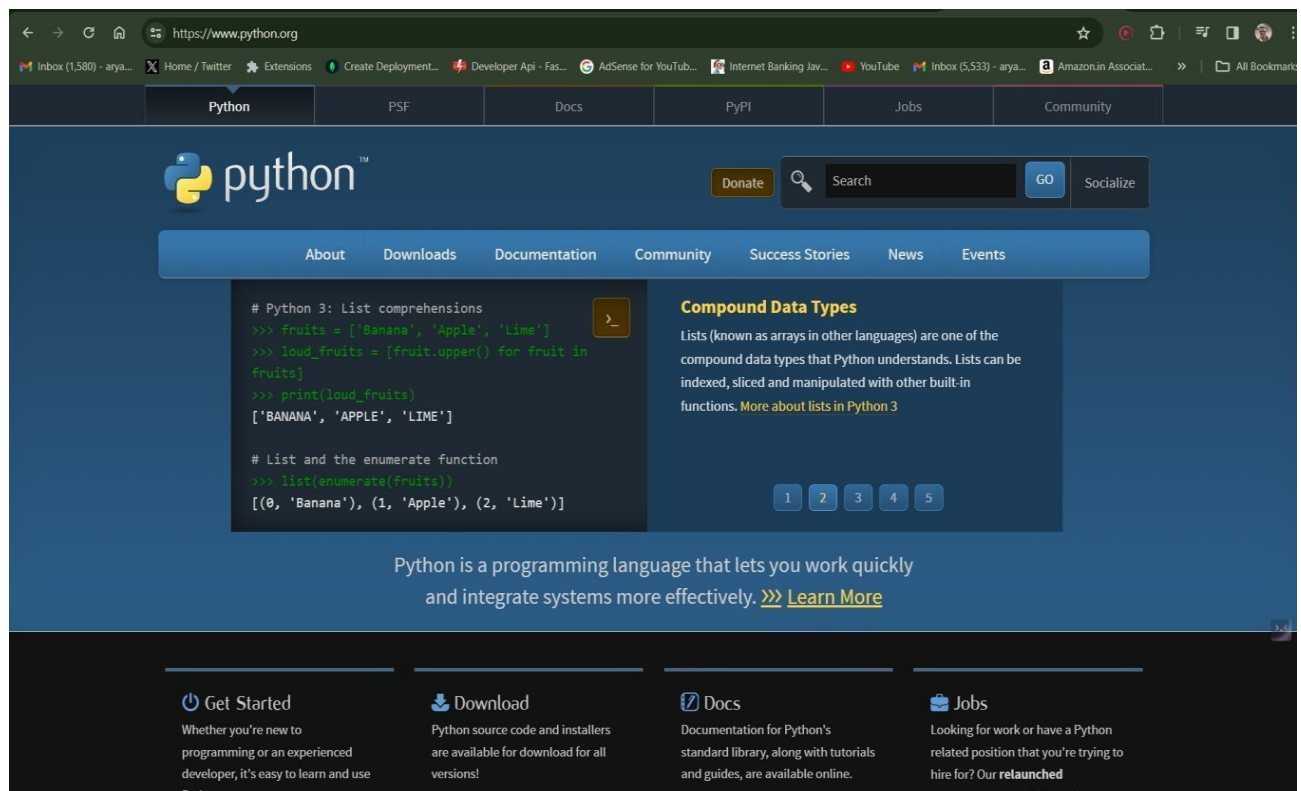
Installation of Python, Django and Visual Studio code editors can be demonstrated.

Python

- Python is a popular high-level, general-use programming language.
- Python is a programming language that enables rapid development as well as more effective system integration.
- Python has two main different versions: Python2 and Python 3. Both are really different.

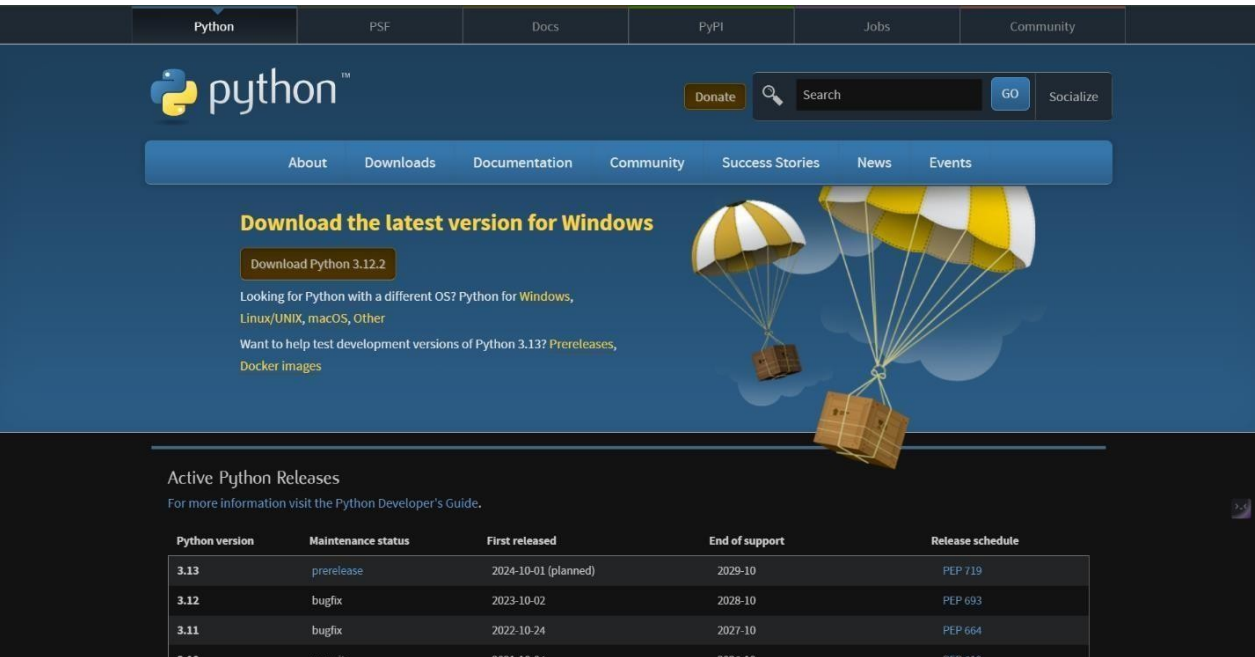
Here are the steps you can follow to download Python: Steps to Download & Install Python

Visit the link <https://www.python.org> to download the latest release of Python.

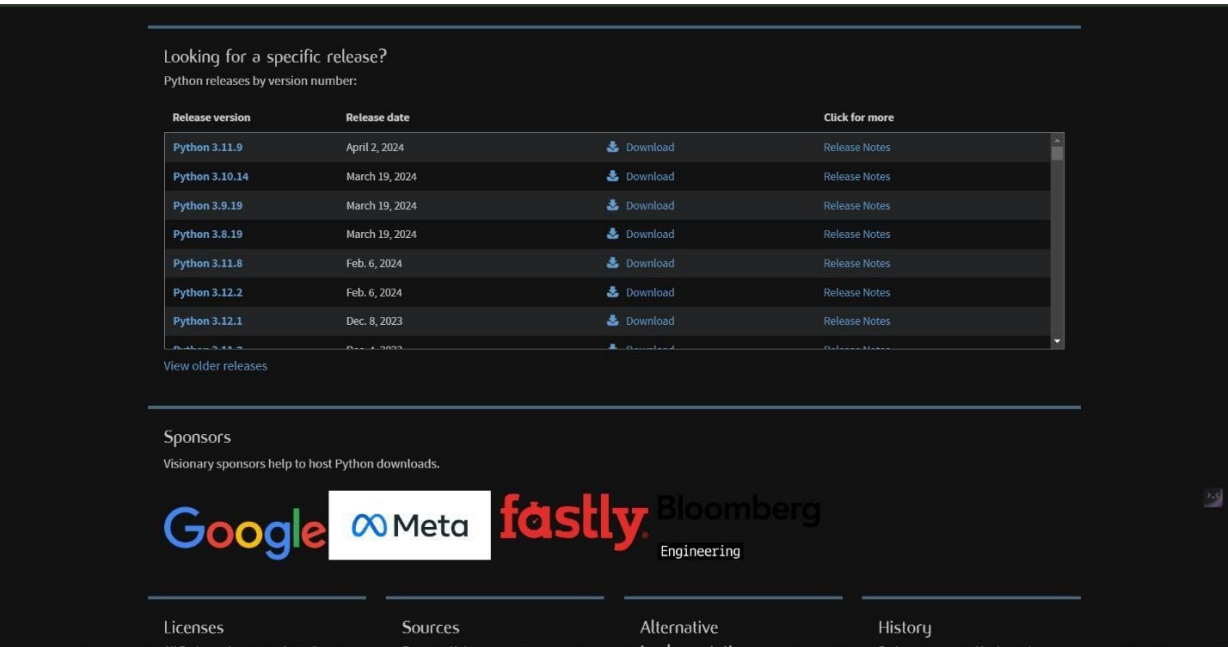


Step -1: Select the Python's version to download.

Click on the download button to download the exe file of Python.

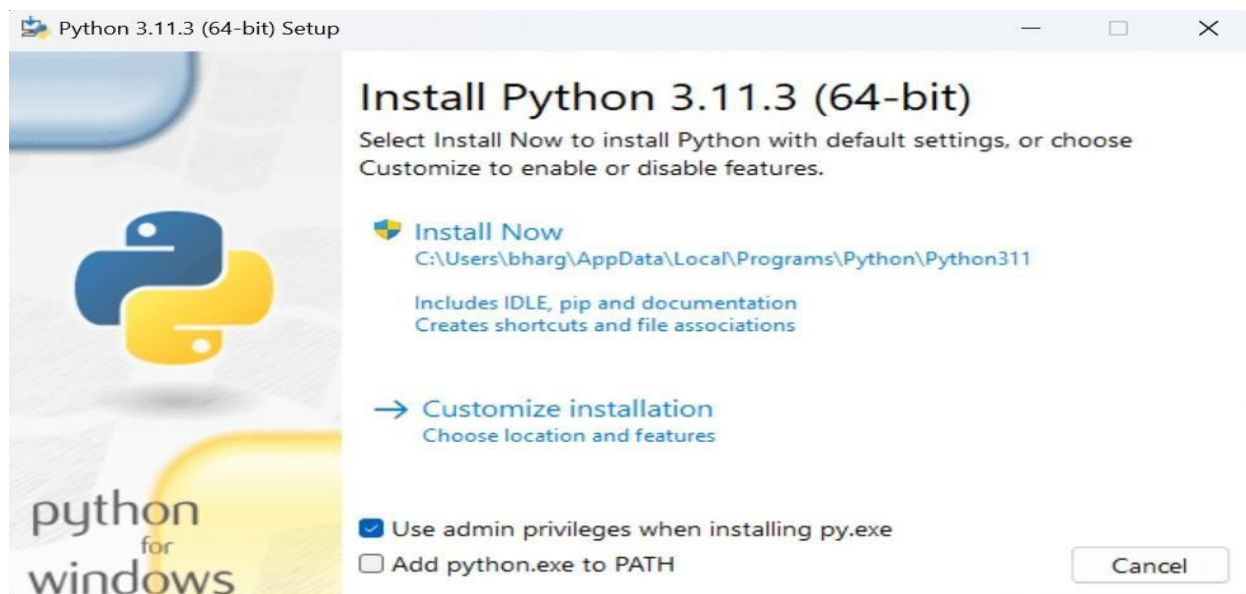


If in case you want to download the specific version of Python. Then, you can scroll down further below to see different versions from 2 and 3 respectively. Click on download button right next to the version number you want to download.



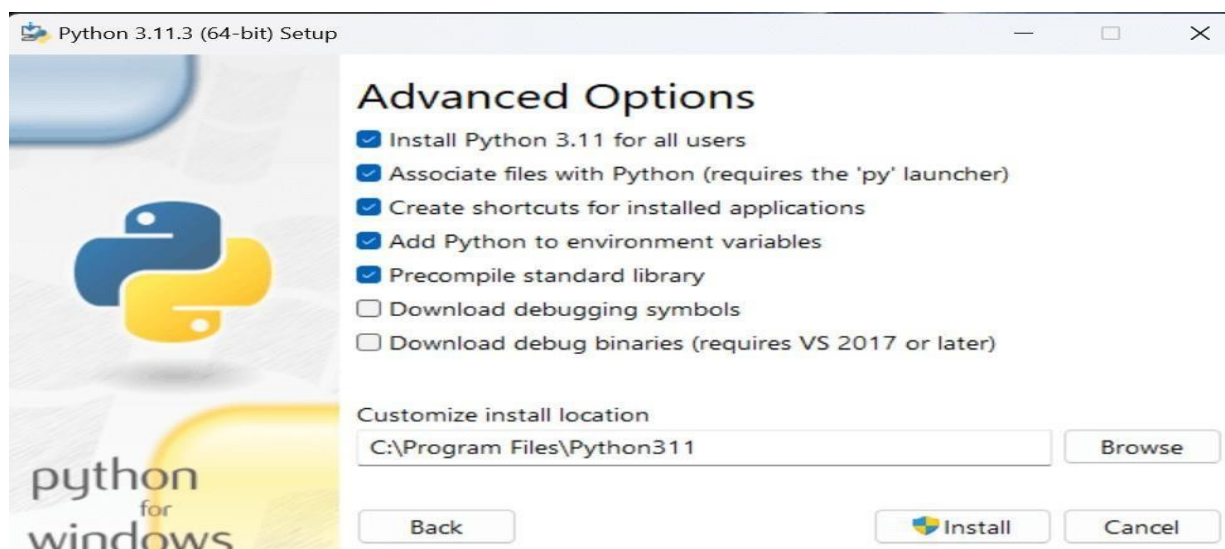
Step -2: Click on the Install Now

Double-click the executable file, which is downloaded.

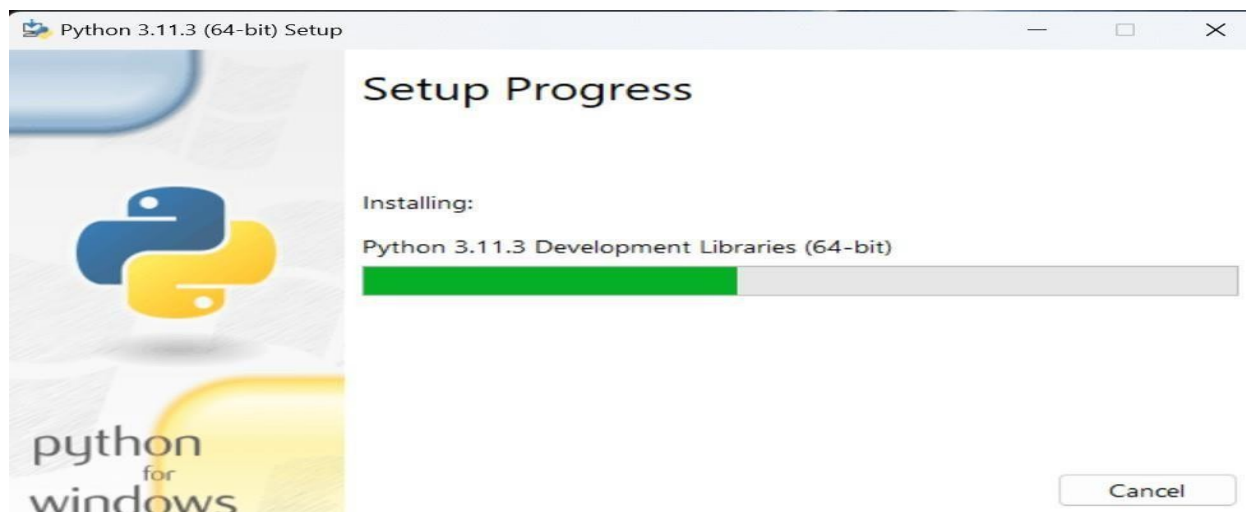


The following window will open. Click on the Add Path check box, it will set the Python path automatically.

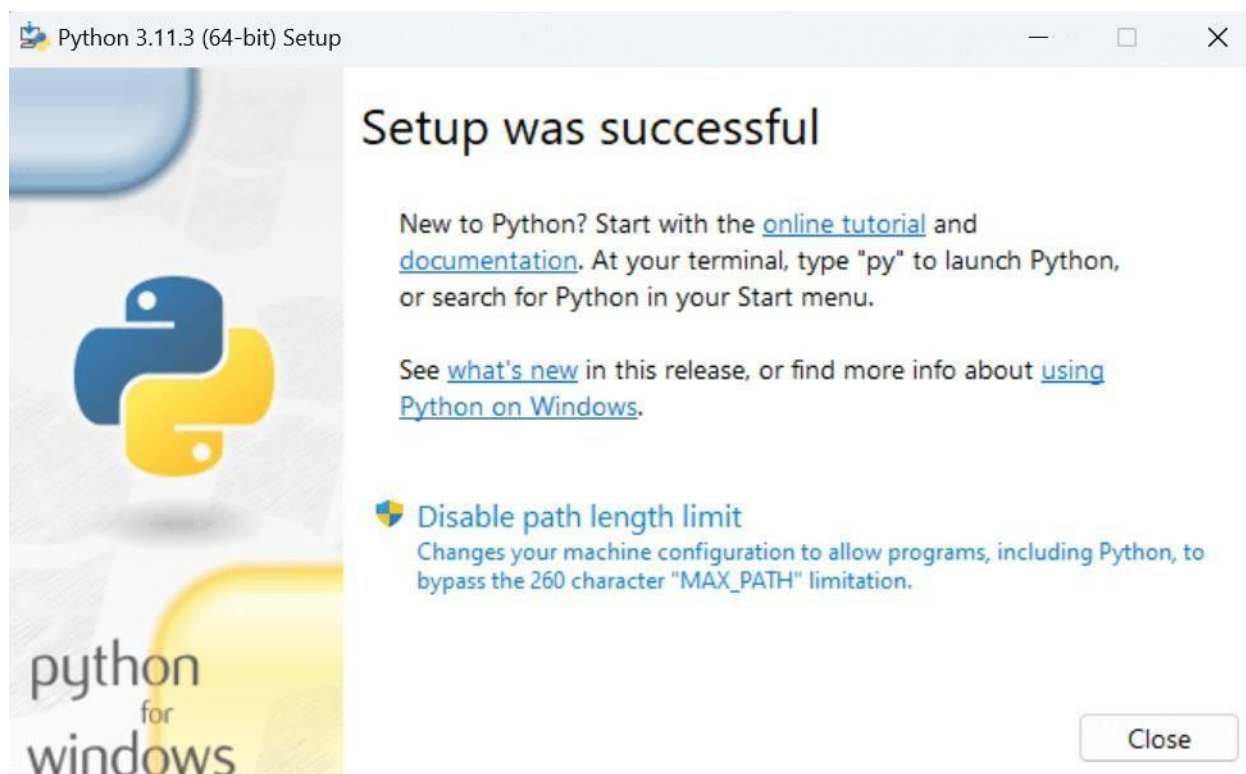
Now, Select Customize installation and proceed. We can also click on the customize installation to choose desired location and features. Other important thing is installing launcher for the all user must be checked.



Step -3 Installation in Process



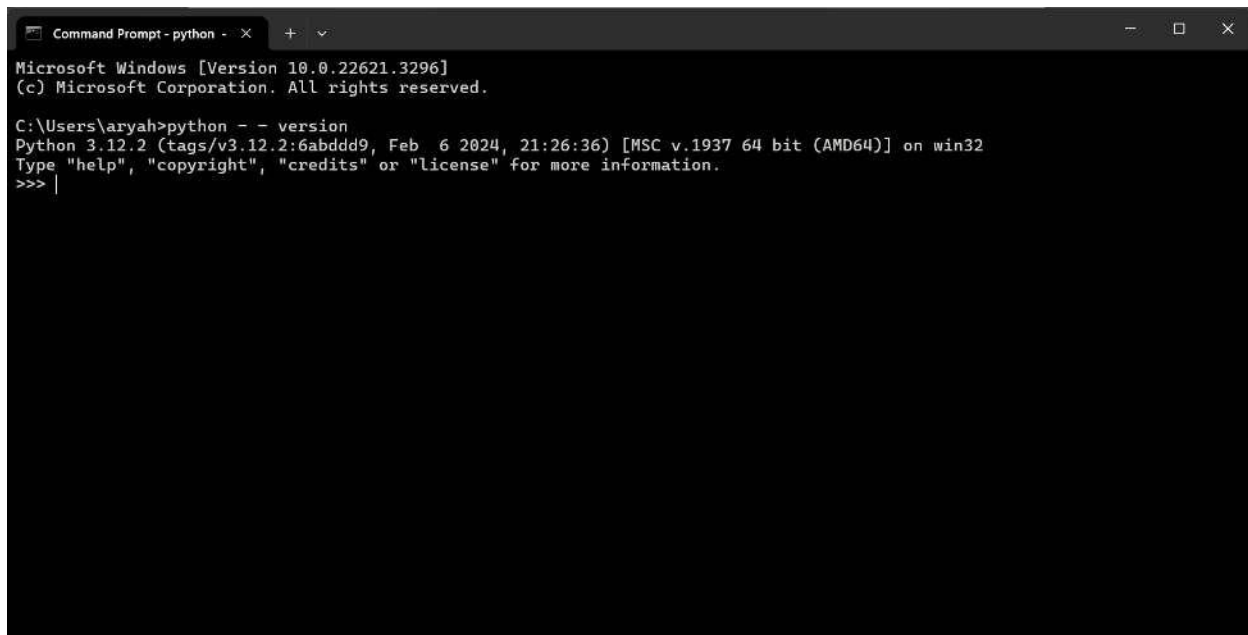
The setup is in progress. All the python libraries, packages, and other python default files will be installed in our system. Once the installation is successful, the following page will appear saying "Setup was successful".



Step -4: Verifying the Python Installation

To verify whether the python is installed or not in our system, we have to do the following.

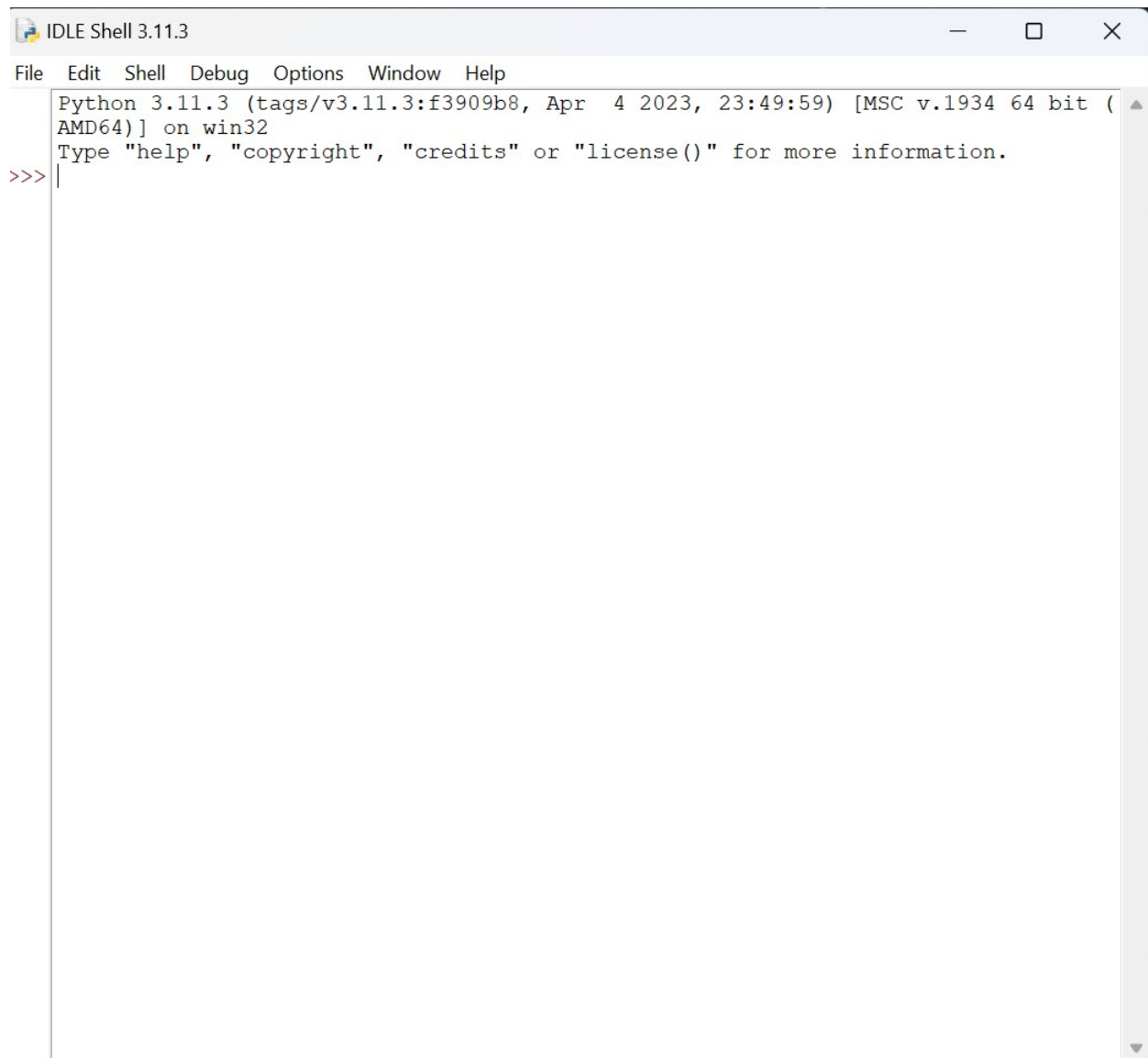
- Go to "Start" button, and search "cmd".
- Then type, "python --version".
- If python is successfully installed, then we can see the version of the python installed.
- If not installed, then it will print the error as "'python' is not recognized as an internal or external command, operable program or batch file."



```
Command Prompt - python - X + v
Microsoft Windows [Version 10.0.22621.3296]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aryah>python --version
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb  6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

We are ready to work with the Python.

Step-5: Opening idle

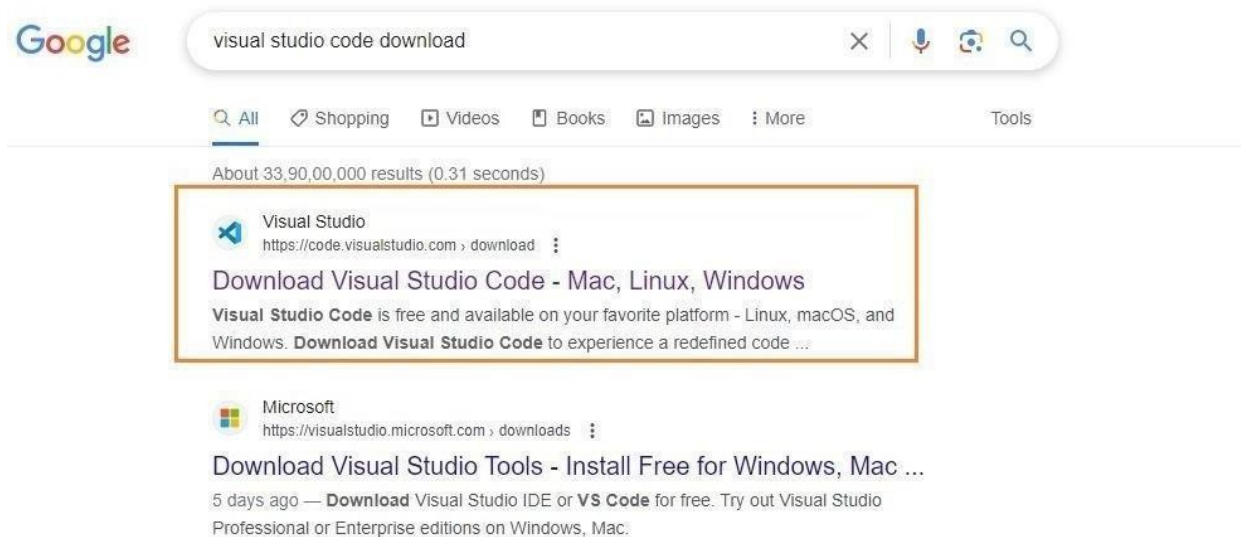
Now, to work on our first python program, we will go the interactive interpreter prompt(idle).To open this, go to "Start" and type idle. Then, click on open to start working on idle.

Here are the steps you can follow: Steps to Download & Install VS Code

Step –1: Open Google and type Visual Studio Code download in the search bar.



Step –2: Click on the below highlighted link for any OS.

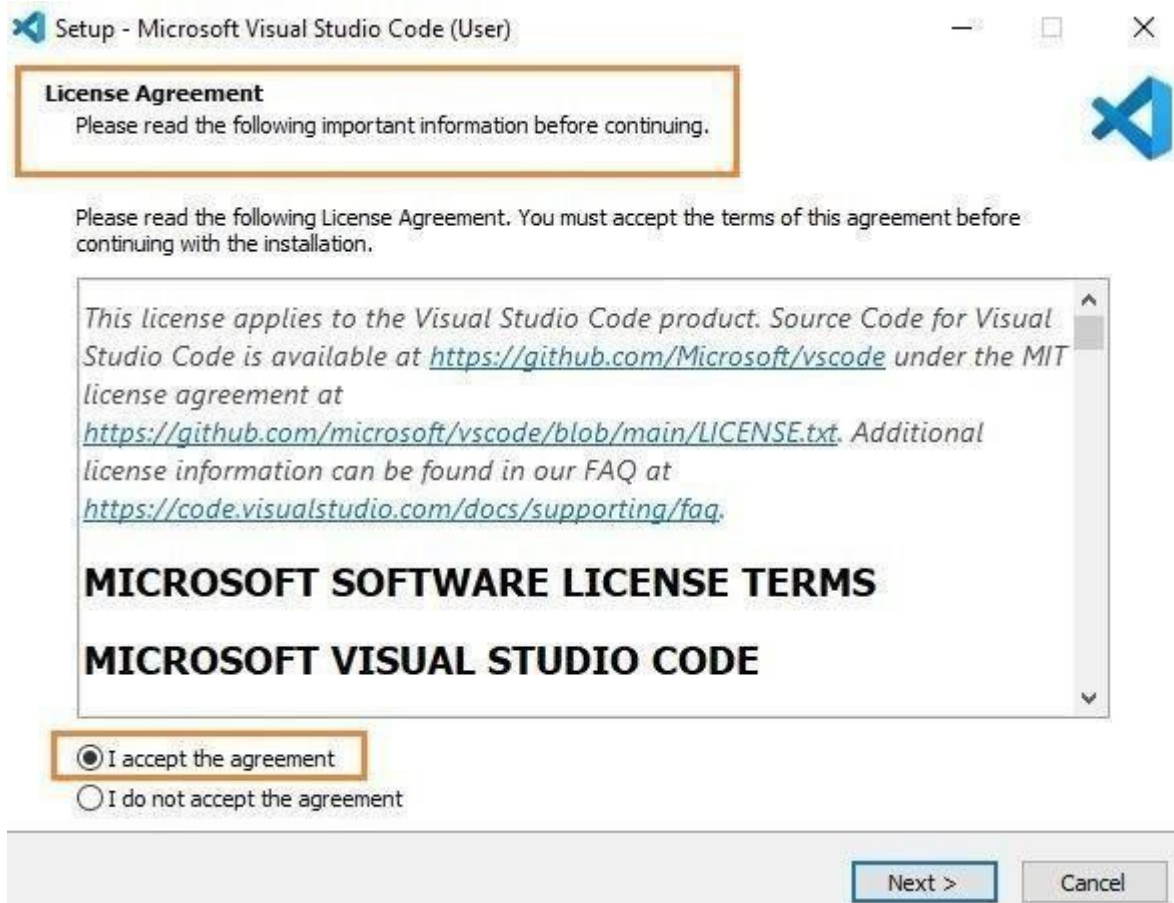


Step-3: Now, select the respective OS. In this case we are selecting Windows.



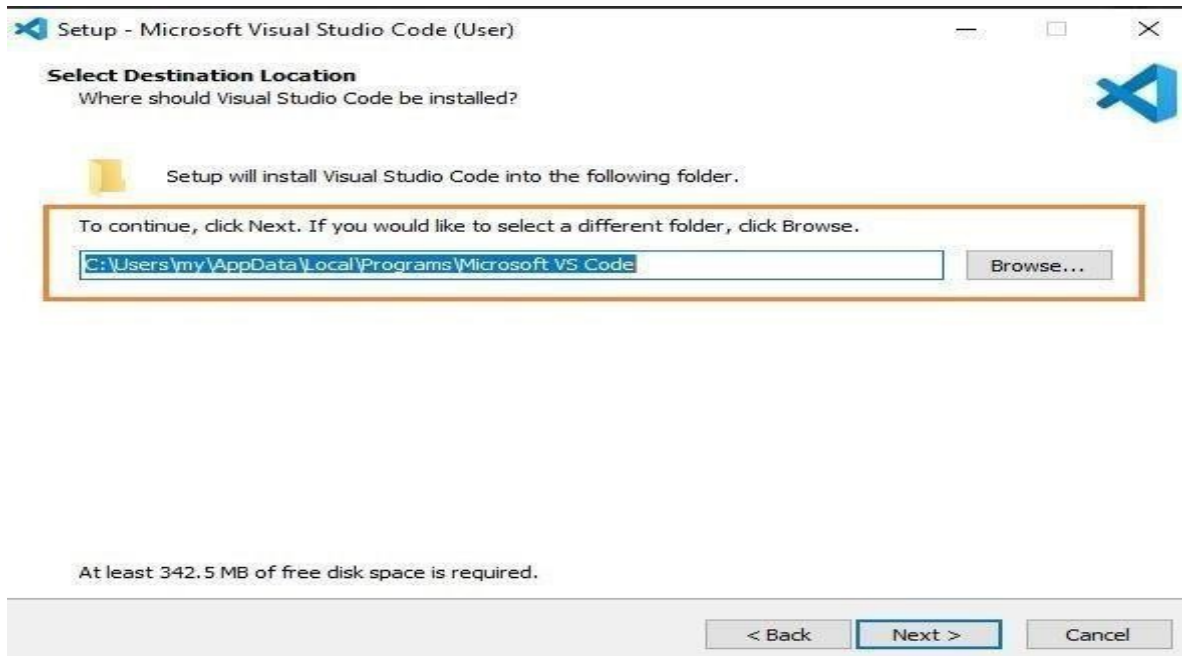
Step-4: The file will be downloaded on to your system. Open the file and then click on Install. After downloading the VS Code file, the official site will display a Thanks message for downloading the file.



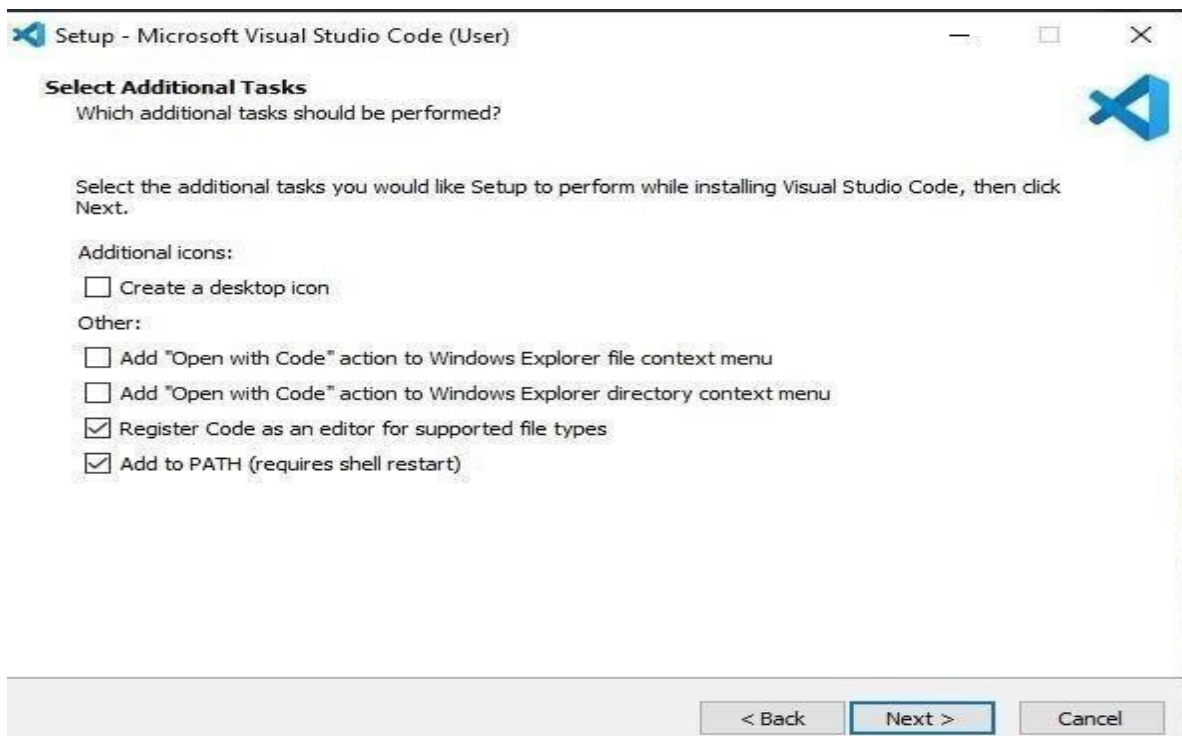
Step –5: Now accept the license agreement.

Step-6: Then it prompts for the file location, where you want to save the VSCode file.

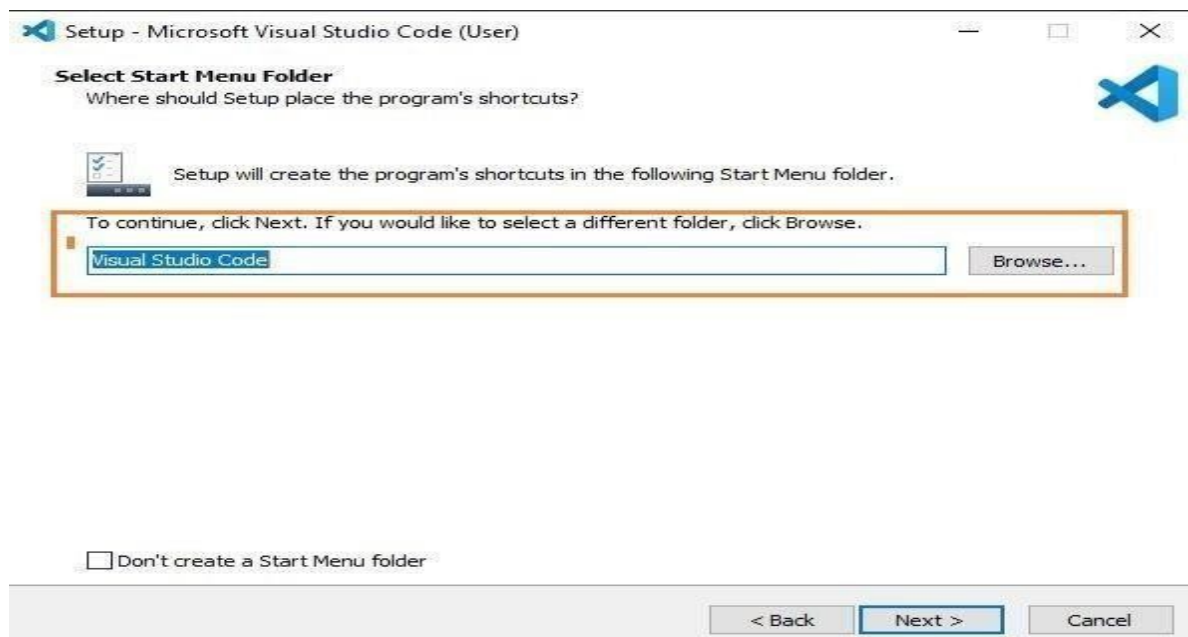
Browse the location and then click on Next.



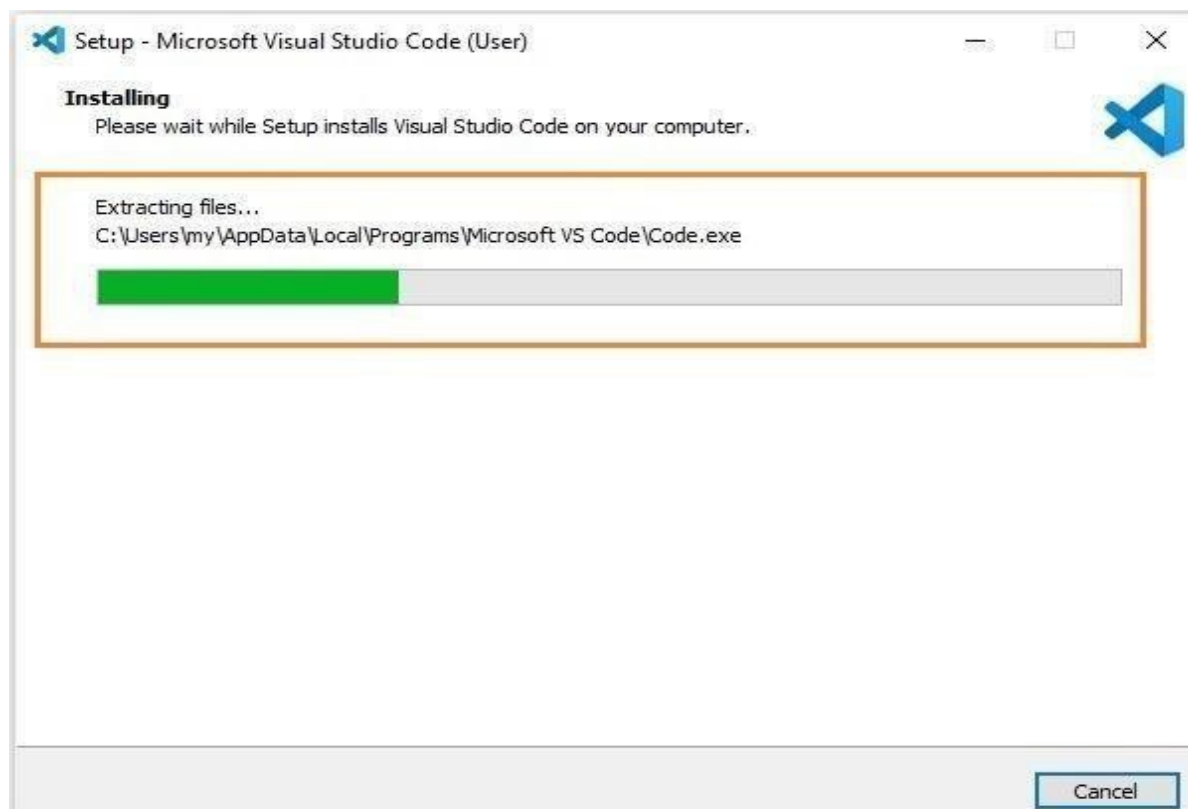
Step-7: Next ,you see the prompt for the additional task which we want the VS Code to perform. At this step, choose the default settings and then click on next.



Step–8: The next prompt is how you want the VS Code on your startup.Change according to your convenience and click on Next.



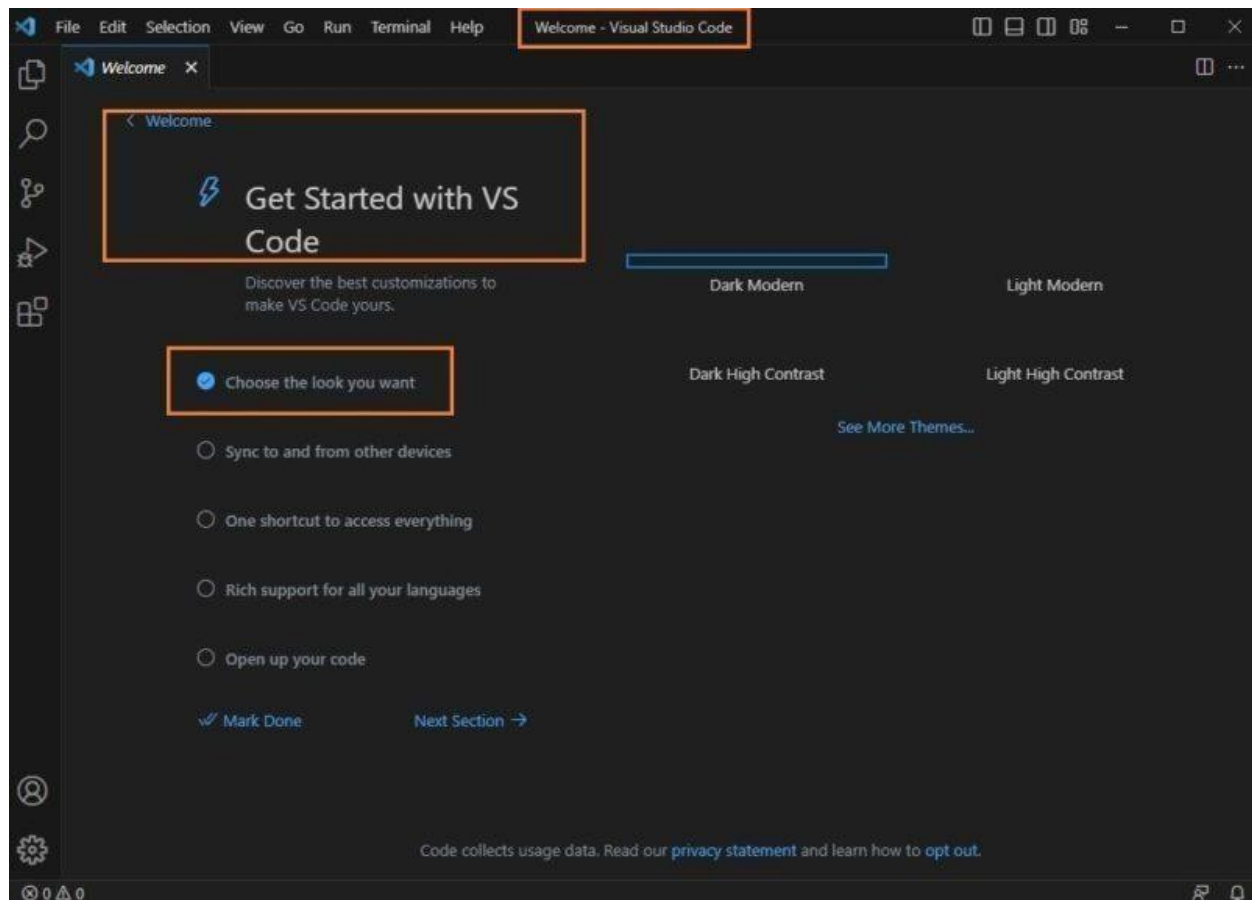
Step –9: The installation of VS Code will now begin.



Step-10: At this step, we have completed installing VS Code, click on Finish.



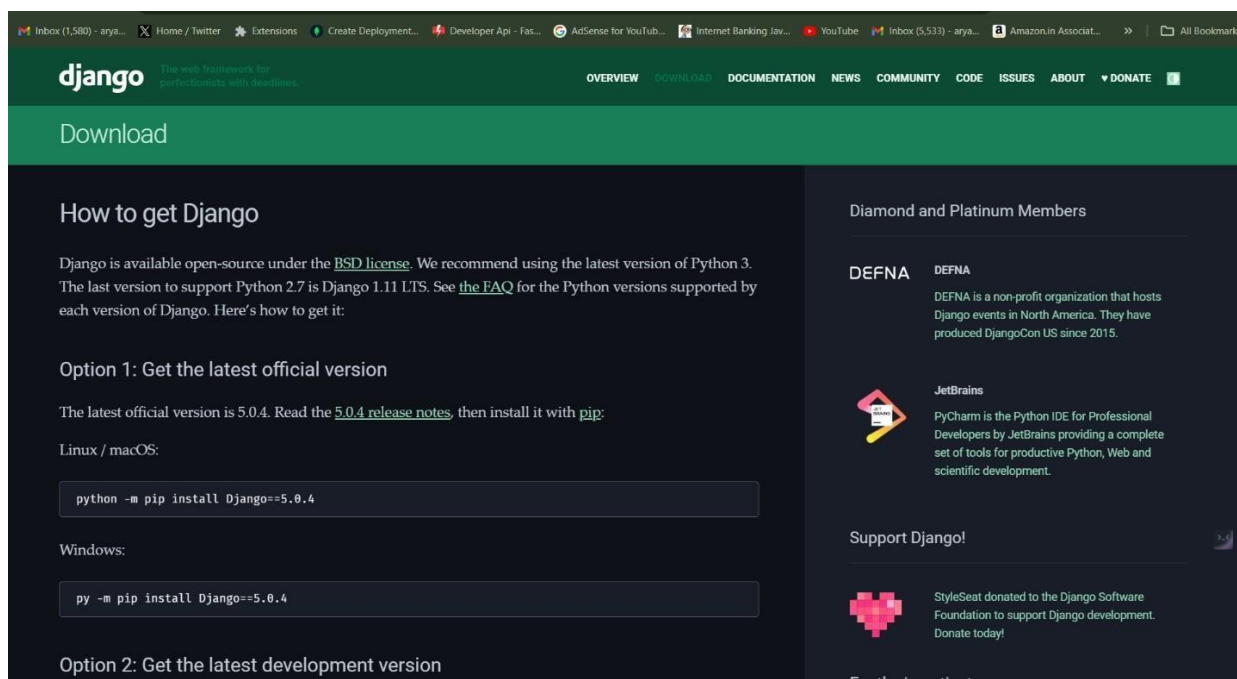
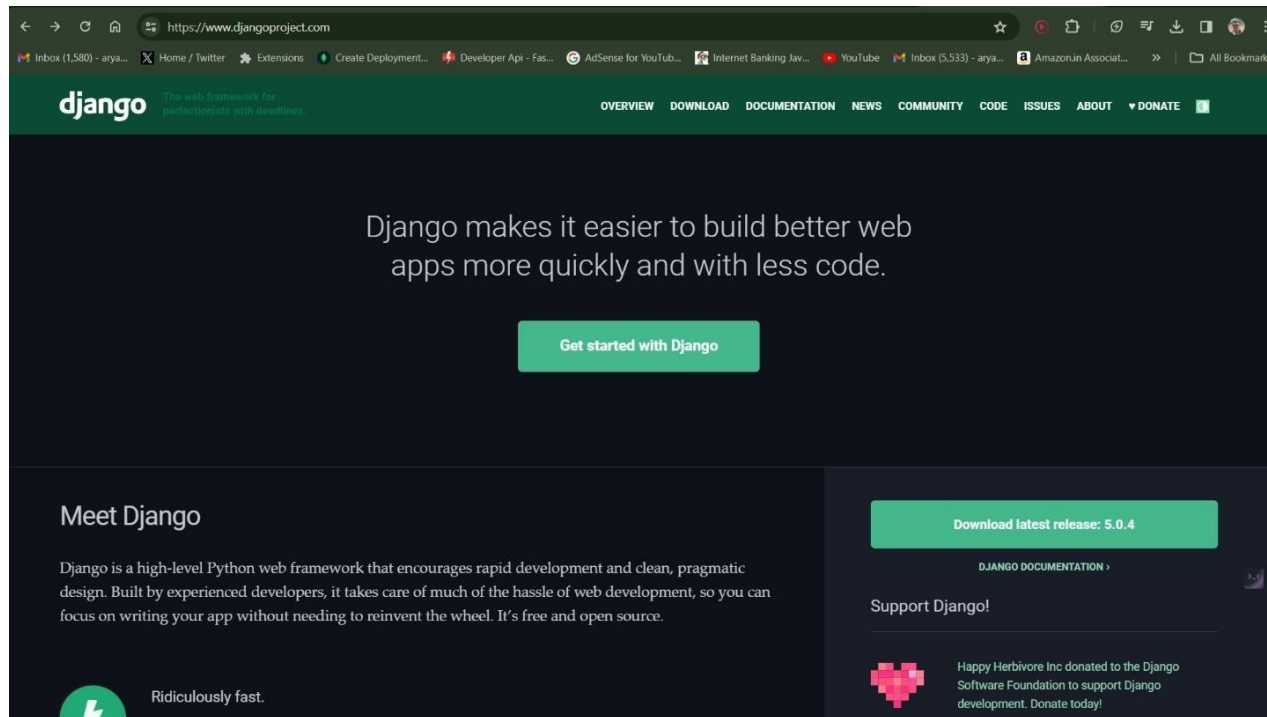
Step –11: Now that VS Code installation is successful, the page appears as below:



We can change the look as per our choice and continue working on it.

Here are the steps you can follow: Steps to Install Django

To install Django, first visit to **Django official site** (<https://www.djangoproject.com>) and download Django by clicking on the download section.

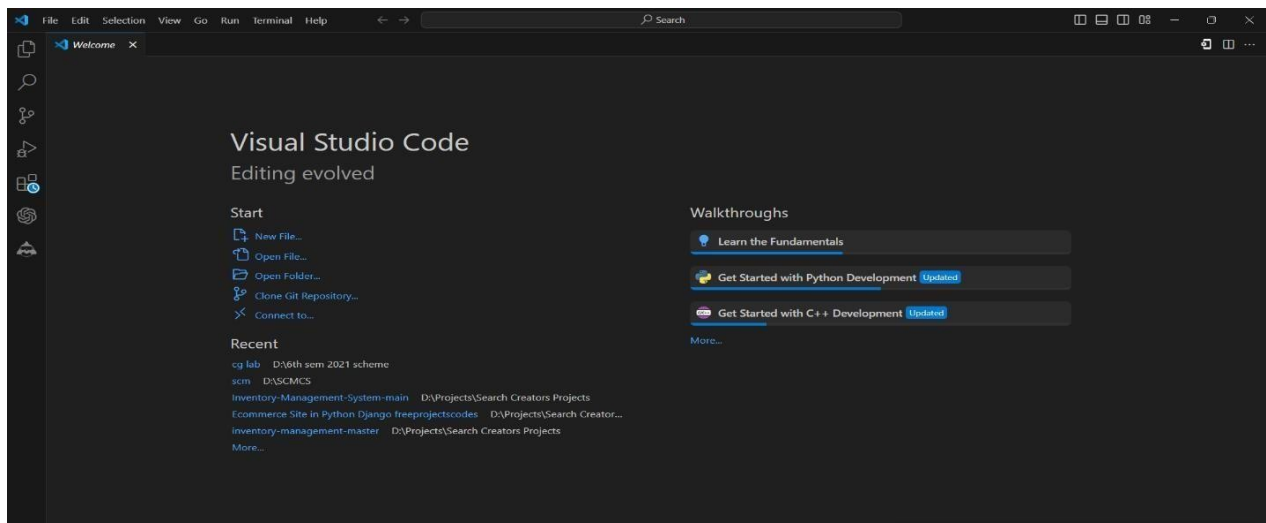


Experiment-02

Creation of virtual environment, Django project and App should be demonstrated.

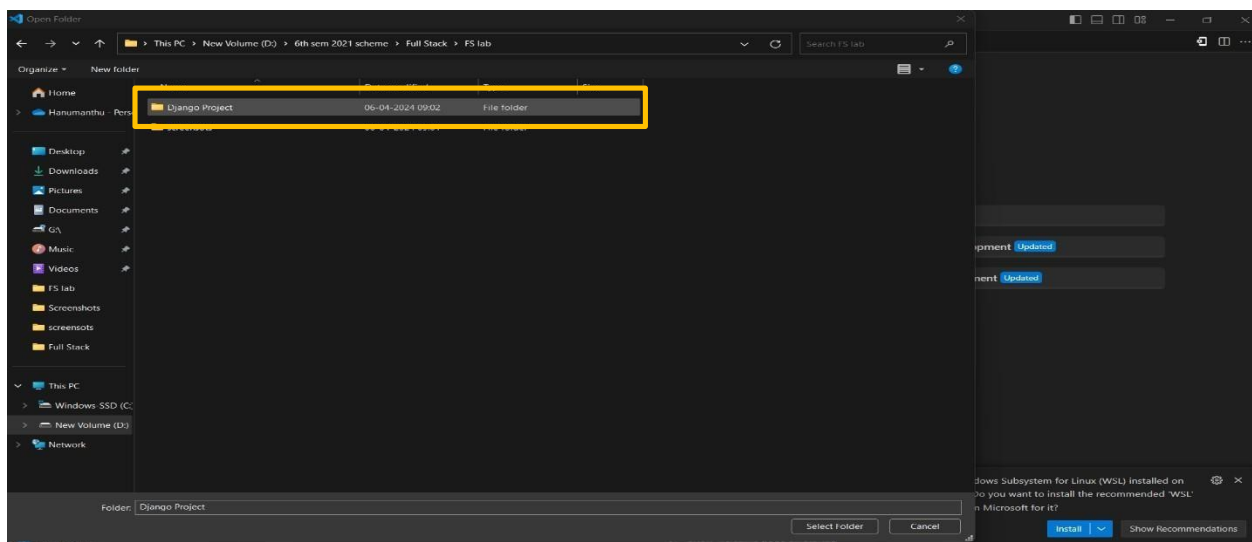
Here are the steps you can follow to create virtual environment: Steps to Create Virtual Environment, Django Project and App

Step-01: Open Visual Studio Code



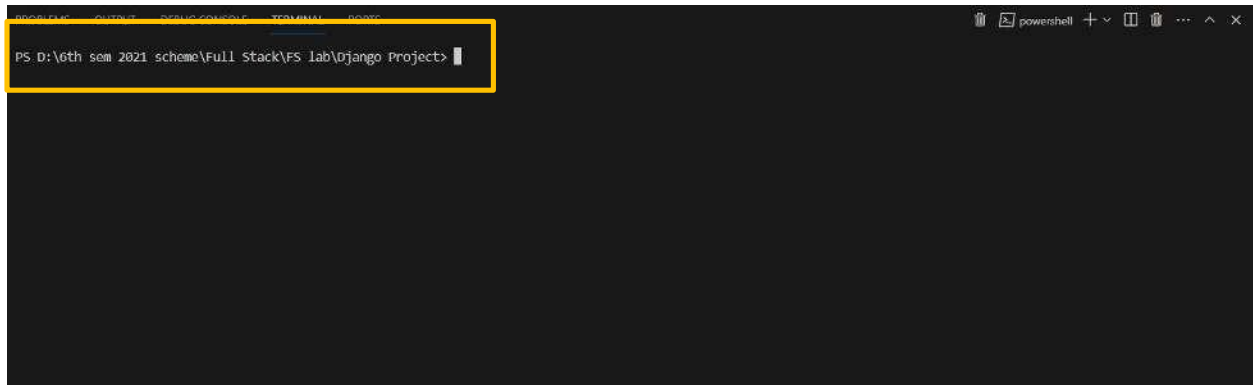
Step-02: Create a new folder for your project

In VS Code, go to **File > Open...** and create a new folder or select an existing folder where you want to create your Django project.



Step-03: Open the integrated terminal

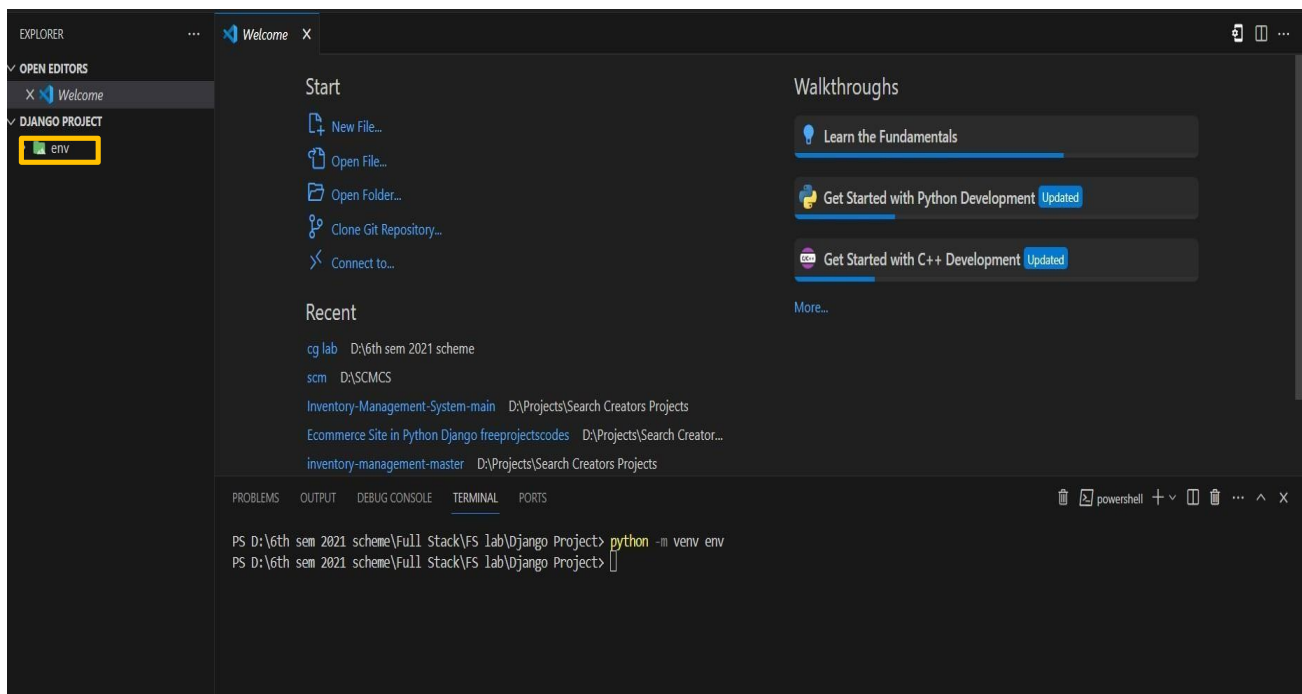
In VSCode, go to **View>Terminal** or use the keyboard shortcut **Ctrl+`** (Windows/Linux) or **Cmd+`** (macOS) to open the integrated terminal.



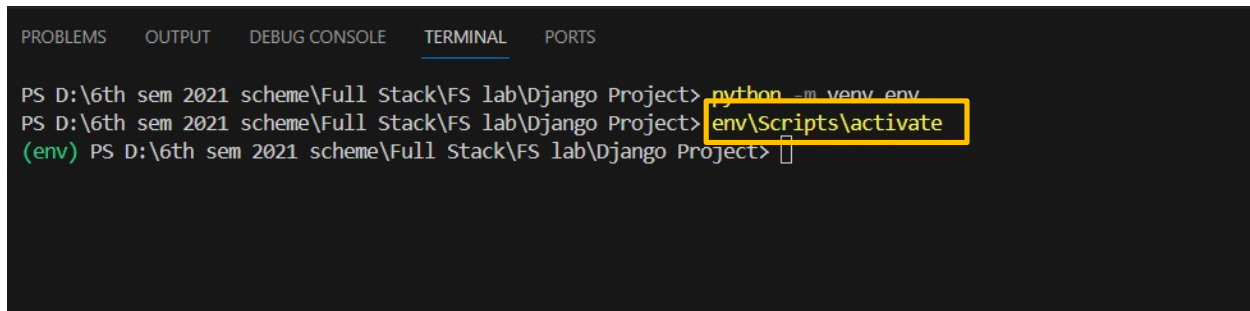
Step-03: Create a virtual environment

In the terminal, run the following command to create a new virtual environment:

python -m venv env



Step-04: Activate the virtual environment

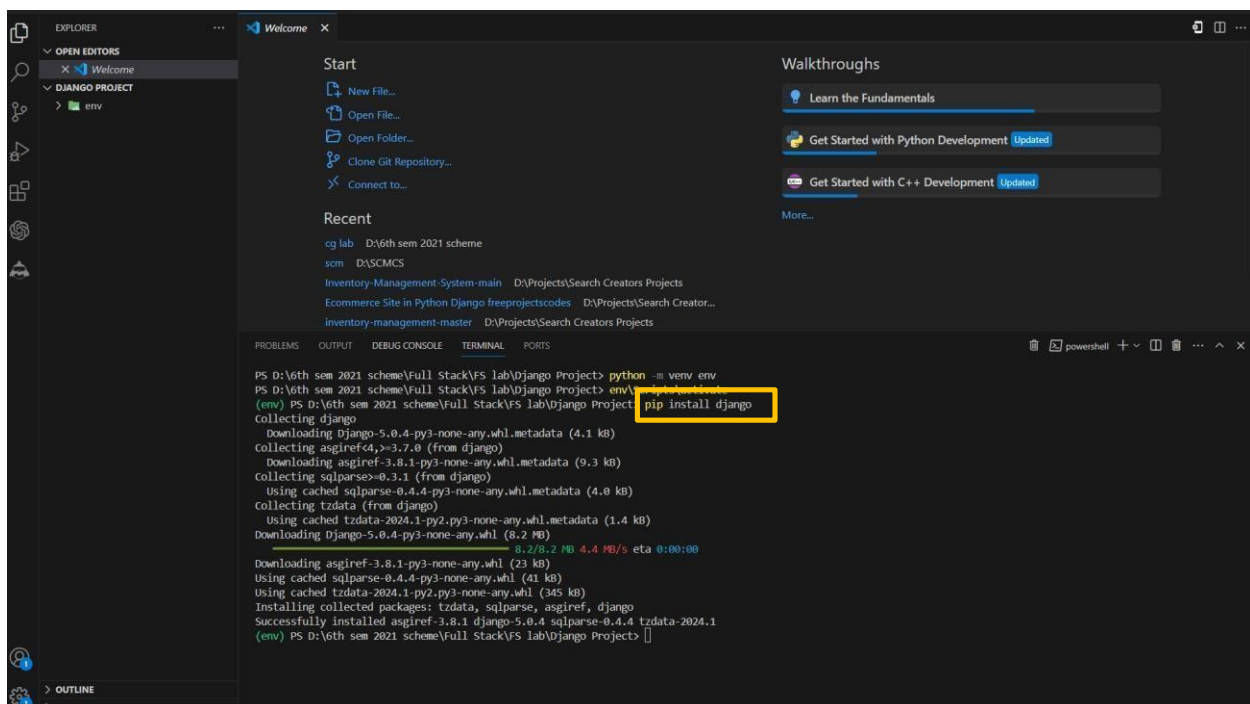


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> python -m venv env
PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> env\Scripts\activate
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> 
```

Step-05: Install Django

With the virtual environment active, install Django by running the following command in the terminal: **pip install django**



```
EXPLORER  Welcome x
  OPEN EDITORS
  Welcome
  DIANGO PROJECT
  env

Start
  New File...
  Open File...
  Open Folder...
  Clone Git Repository...
  Connect to...

Recent
  cg lab  D:\6th sem 2021 scheme
  scm    D:\SCMCS
  Inventory-Management-System-main  D:\Projects\Search Creators Projects
  Ecommerce Site in Python Django freeprojectscodes  D:\Projects\Search Creator...
  inventory-management-master  D:\Projects\Search Creators Projects

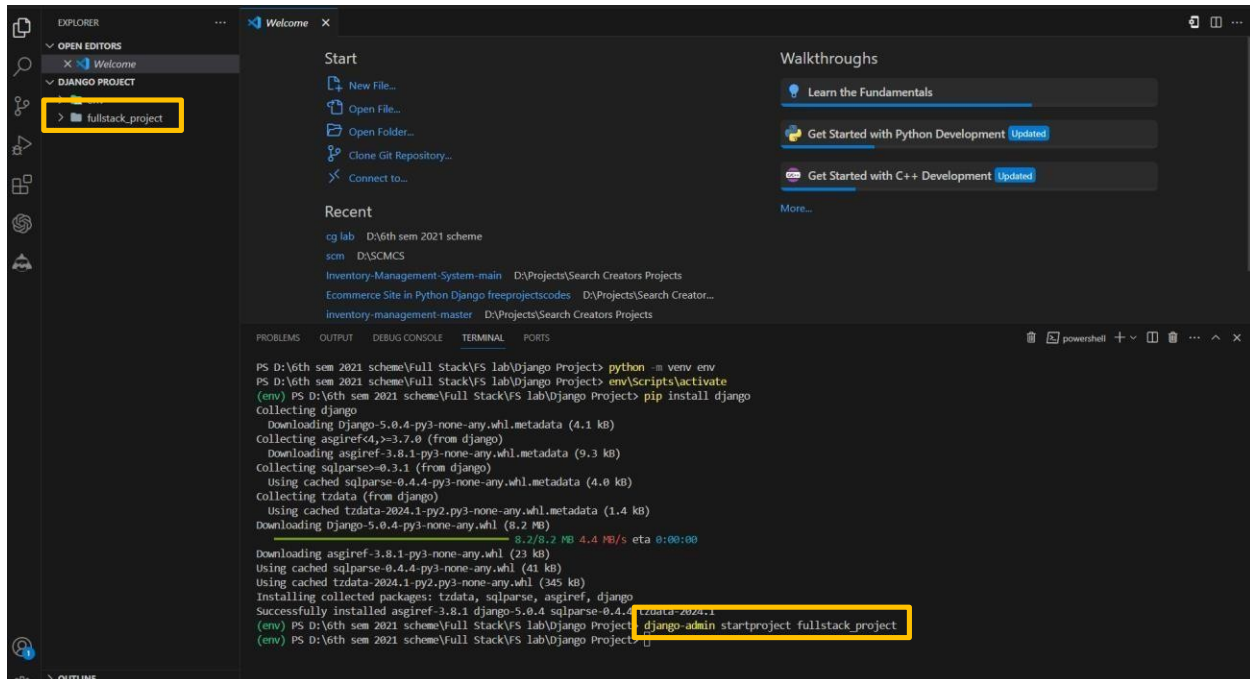
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> python -m venv env
PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> env\Scripts\activate
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> pip install django
Collecting django
  Downloading Django-5.0.4-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,>=3.7.0 (from django)
  Downloading asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse==0.3.1 (from django)
  Using cached sqlparse-0.4.4-py3-none-any.whl.metadata (4.0 kB)
Collecting tzdata (from django)
  Using cached tzdata-2024.1-py2.py3-none-any.whl.metadata (1.4 kB)
Downloaded Django-5.0.4-py3-none-any.whl (8.2 MB)
8.2/8.2 MB 4.4 MB/s eta 0:00:00
Downloaded asgiref-3.8.1-py3-none-any.whl (23 kB)
Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
Using cached tzdata-2024.1-py2.py3-none-any.whl (345 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.8.1 django-5.0.4 sqlparse-0.4.4 tzdata-2024.1
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> 
```

Step-06: Create a new Django project

Run the following command to create a new Django project:

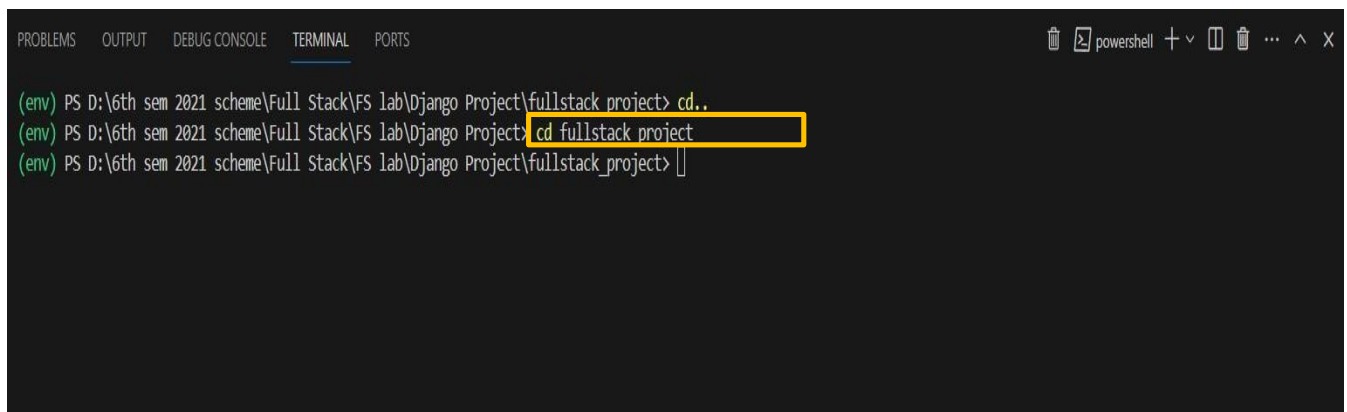
django-admin startproject myproject [change Project name as You Desired]



Step-07: Create a new Django app

In the VS Code terminal, navigate to your project directory:

cd myproject [change Project name as You Desired]



Create a new Django app by running the following command

Python manage.py startapp myapp

[Replace myapp with the name you want to give to your app.]

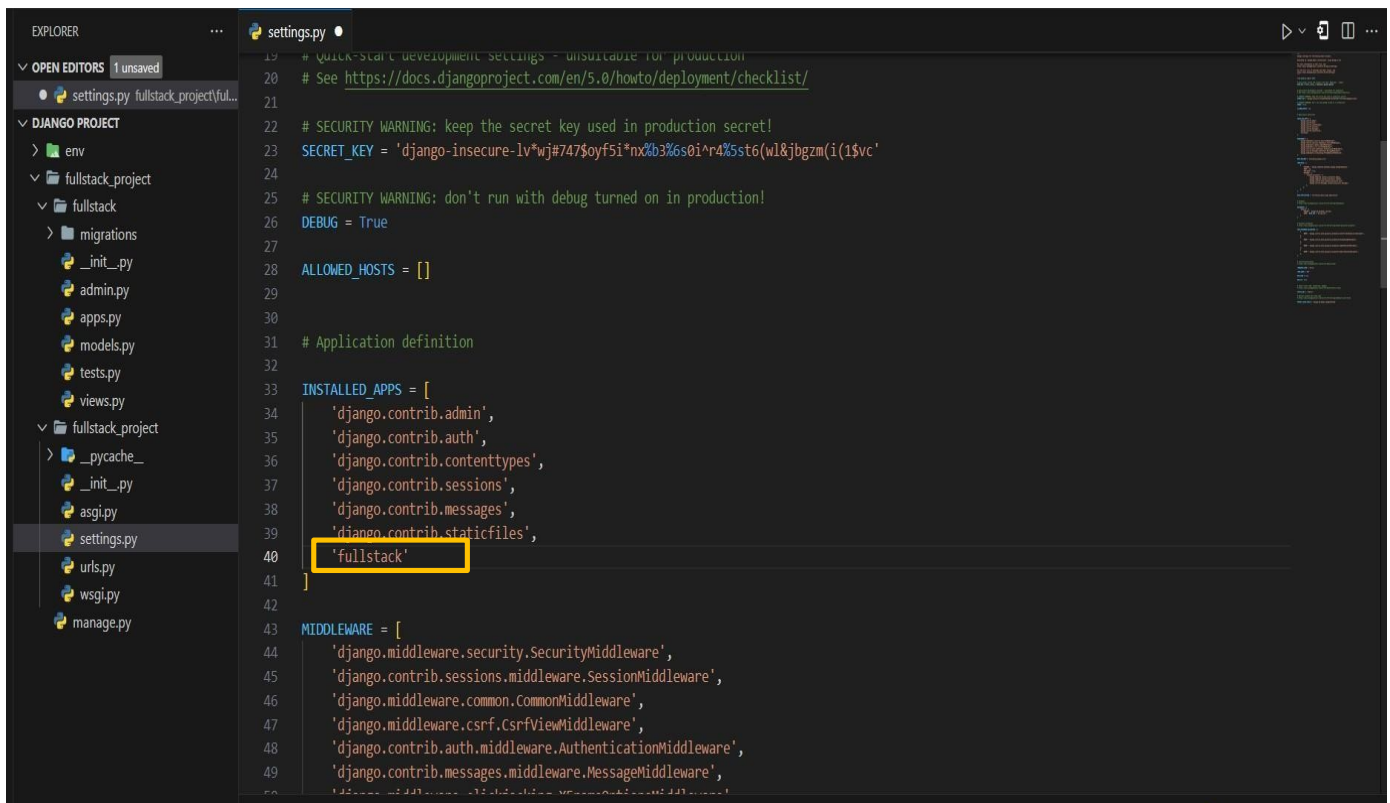


```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> cd ..
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project> cd fullstack_project
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py startapp fullstack
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> |
```

Step-08: Add the app to the INSTALLED_APPS list

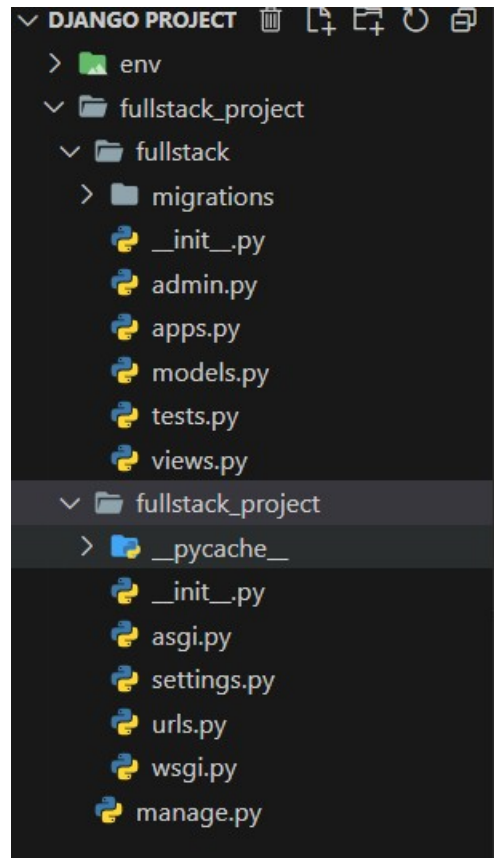
In the VSCode file explorer, locate the settings.py file(usually located in the myproject directory) and open it.

Locate the **INSTALLED_APPS** list and add the name of your new app to the list



```
19 # Quick-start development settings - unsuitable for production
20 # See https://docs.djangoproject.com/en/5.0/howto/deployment/checklist/
21
22 # SECURITY WARNING: keep the secret key used in production secret!
23 SECRET_KEY = 'django-insecure-lv*uj#747$oyf5i*n%b3%6s0i*r4%5st6(wl&jbgzm(i(1$vc'
24
25 # SECURITY WARNING: don't run with debug turned on in production!
26 DEBUG = True
27
28 ALLOWED_HOSTS = []
29
30 # Application definition
31
32 INSTALLED_APPS = [
33     'django.contrib.admin',
34     'django.contrib.auth',
35     'django.contrib.contenttypes',
36     'django.contrib.sessions',
37     'django.contrib.messages',
38     'django.contrib.staticfiles',
39     'fullstack'
40 ]
41
42 MIDDLEWARE = [
43     'django.middleware.security.SecurityMiddleware',
44     'django.contrib.sessions.middleware.SessionMiddleware',
45     'django.middleware.common.CommonMiddleware',
46     'django.middleware.csrf.CsrfViewMiddleware',
47     'django.contrib.auth.middleware.AuthenticationMiddleware',
48     'django.contrib.messages.middleware.MessageMiddleware',
49     'django.middleware.clickjacking.XFrameOptionsMiddleware',
50 ]
```

Here is the Django project structure that you wanted to create



After creating your Django app, the next step is to **create database migrations** for your app's models (if you have any defined). Here's how you can do that using the **python manage.py makemigrations** command in Visual Studio Code (VS Code)

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations
No changes detected
```

Step-08: Apply the migrations

Once you've reviewed the migration files and are ready to apply the changes to your database, run the following command in the terminal:

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Step-09: Run Your Project

Here's how you can run the Django development server using the **python manage.py runserver**

Command in Visual Studio Code(VSCoDe)

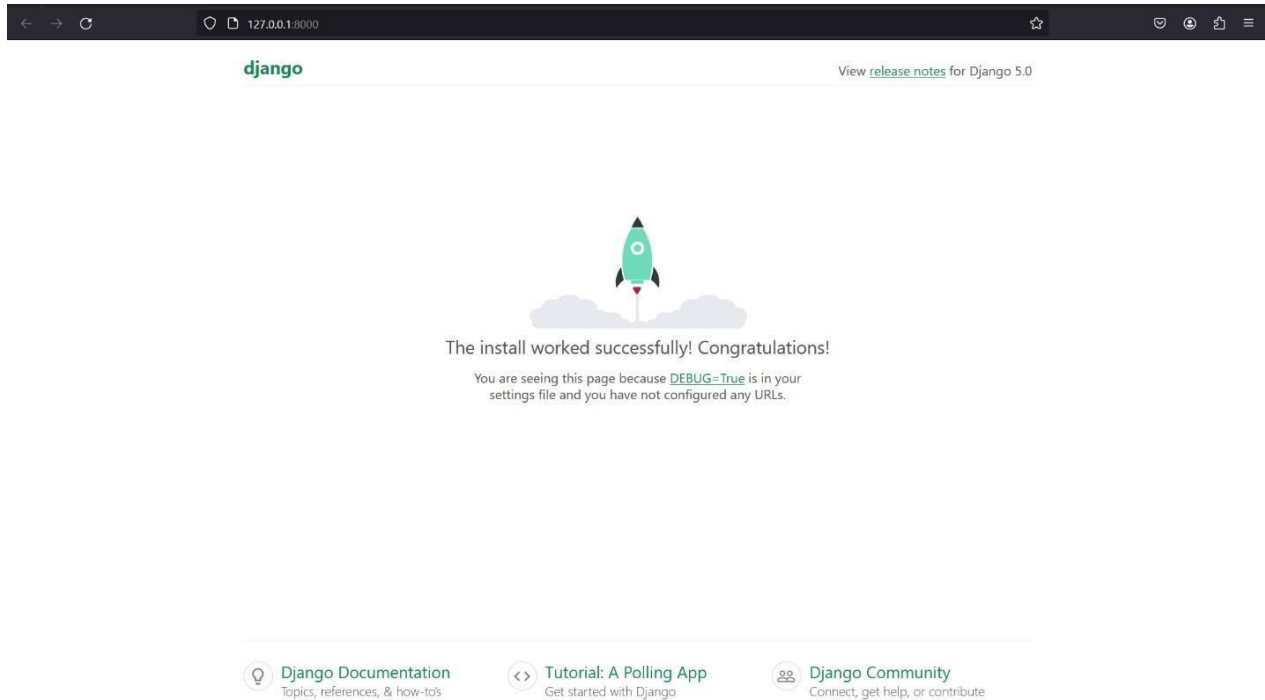
```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 11:23:06
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Check the server output

The terminal will display the URL where the development server is running, typically <http://127.0.0.1:8000/>. It will also show any startup logs or warnings, if any. Open the development server in your browser Copy the URL from the terminal output (e.g., <http://127.0.0.1:8000/>) and paste it into your web browser's address bar.

Here Displays the Development Server After Running Server



Experiment-03

Develop a Django app that displays current date and time in server.

Step-01: Create project

```
django-admin startproject datetime_project
```

Step-02: Change directory:

```
cd datetime_project
```

Create App:

```
python manage.py startapp datetime_app
```

Step-03: Create view.py in your app (datetime_app)

```
#views.py
```

```
from django.shortcuts import render
```

```
from django.utils import timezone
```

```
def current_datetime(request):
```

```
    now = timezone.now()
```

```
    context = {'datetime': now}
```

```
    return render(request, 'datetime_app/current_datetime.html', context)
```

Step-04: Create a template

- In your app's directory (e.g., **datetime_app**), create a new folder named 'templates'.
- Inside the templates folder, create another folder with the same name as your app (e.g., **datetime_app**).
- Inside the **datetime_app** folder, create a new file named **current_datetime.html**.
- Open **current_datetime.html** and add the following code to display the current date and time:

```
<!DOCTYPE html>
<html>
<head>
    <title>
        Current date and time
    </title>
</head>
<body>
    <h1>Current Date and Time:</h1>
    {{datetime}}
</body>
</html>
```

Step-05: Map the view function to a URL

- Create and open the urls.py file in your Django app's directory (e.g., datetime_app/urls.py).
- Import the view function at the top of the file
- Add a new URL pattern to the urlpatterns list

```
#urls.py

from django.urls import path

from . import views

urlpatterns = [

    path(' ',views.current_datetime, name='current_datetime'),

]
```

Step-06: Add the app to INSTALLED_APPS

- Open the settings.py file in your project's directory (e.g., datetime_project/settings.py).
- Locate the **INSTALLED_APPS** list and add the name of your new app to the list:

Step-07: Include the app's URLs in the project's URL patterns

- Open the **urls.py** file in your project's directory (e.g., **datetime_project/urls.py**).
- **Include the following code:**

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [

    path('admin/', admin.site.urls),

    path('',include('datetime_app.urls')),

]
```

- Import the include function from **django.urls** and the path function from **django.urls**:

from django.urls import include, path

- Add a new URL pattern to the urlpatterns list

path ('', include('datetime_app.urls')),

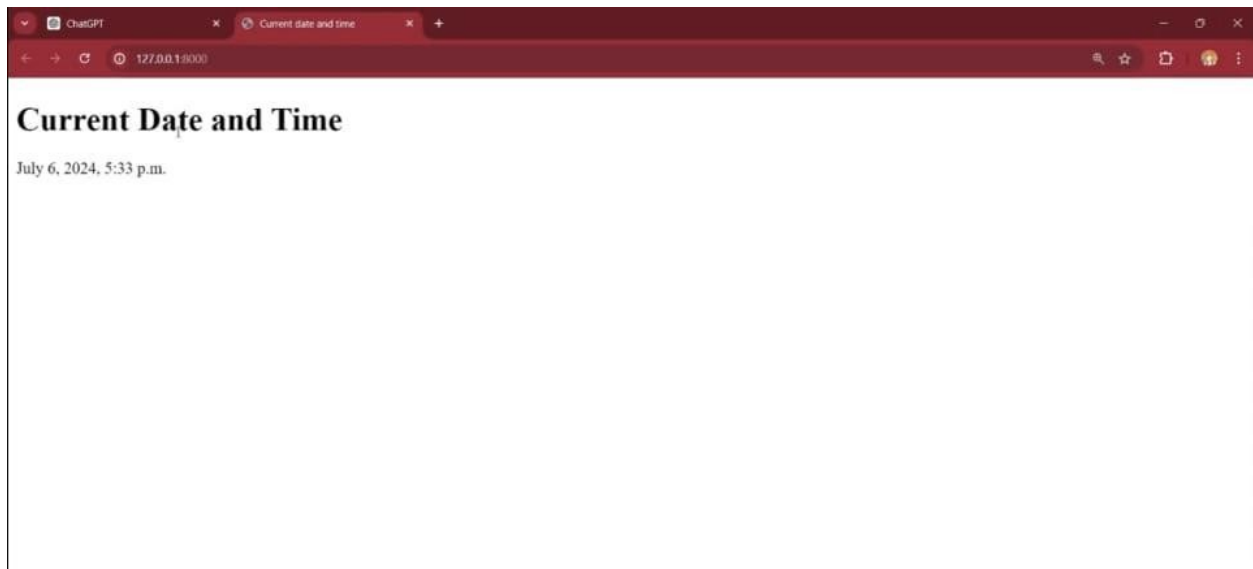
- This includes the URL patterns from your app's urls.py file.

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.

OUTPUT:

Experiment-04

Develop a Django app that displays date and time four hours ahead and four hours before as an offset of current date and time in server.

Step-01: Create project

```
django-admin startproject fourdatetime_project
```

Step-02: Change directory:

```
cd fourdatetime_project
```

Create App:

```
python manage.py startapp fourdatetime_app
```

Step-03: Create view.py in your app (fourdatetime_app/views.py)

```
#views.py

from django.shortcuts import render

import datetime

def datetime_offsets(request):

    now = datetime.datetime.now()

    context = {

        'current_datetime': now,

        'four_hours_ahead': now + datetime.timedelta(hours=4),

        'four_hours_before': now - datetime.timedelta(hours=4),

    }

    return render(request, 'fourdatetime_app/datetime_offsets.html', context)
```

- This view function gets the current date and time using `datetime.datetime.now()`, calculates the date and time four hours ahead and four hours before using `datetime.timedelta`, and then passes all three values to the template as context variables.

Step-04: Create a new template

- In your app's directory (e.g., `fourdatetime_app`), create a new folder named `templates` (if it doesn't already exist).
- Inside the `templates` folder, create another folder with the same name as your app (e.g., `fourdatetime_app`).
- Inside the `fourdatetime_app` folder, create a new file named `datetime_offsets.html`.
- Open `datetime_offsets.html` and add the following code to display the current date and time, along with the offsets:

```
<!DOCTYPE html>
<html>
<head>
  <title>
    Current date and time
  </title>
</head>
<body>
  <h1>Current Date and Time on the Server</h1>
  <p>{{ current_datetime }}</p>
  <h2>Four Hours Ahead</h2>
  <p>{{ four_hours_ahead }}</p>
  <h2>Four Hours Before</h2>
  <p>{{ four_hours_before }}</p>
</body>
</html>
```

Step-05: Map the view function to a URL

- Create and open the `urls.py` file in your Django app's directory (e.g., `fourdatetime_app/urls.py`).
- Import the view function at the top of the file
- Add a new URL pattern to the `urlpatterns` list.

```
from django.urls import path

from . import views

urlpatterns = [

    path('datetime-offsets/', views.datetime_offsets, name='datetime_offsets'),

]
```

Step-06: Add the app to INSTALLED_APPS

- Open the settings.py file in your project's directory (e.g., fourdatetime_project/settings.py).
- Locate the INSTALLED_APPS list and add the name of your new app to the list.

Step-07: Include the app's URLs in the project's URL patterns

- Open the urls.py file in your project's directory (e.g., fourdatetime_project/urls.py).
- Import the include function from **django.urls** and the path function from **django.urls**:

```
from django.urls import include, path
```

- Add a new URL pattern to the urlpatterns list

```
path('', include('fourdatetime_app.urls')),
```

- This includes the URL patterns from your app's urls.py file

- Include the following code:

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('fourdatetime_app.urls')),

]
```

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **<http://127.0.0.1:8000/datetime-offsets/>**

OUTPUT:

Current Date and Time on the Server

April 6, 2024, 10:15 p.m.

Four Hours Ahead

April 7, 2024, 2:15 a.m.

Four Hours Before

April 6, 2024, 6:15 p.m.

Experiment-05

Develop a simple Django app that displays an unordered list of fruits and ordered list of selected students for an event.

Step-01: Create project

```
django-admin startproject listfruit_project
```

Step-02: Change directory:

```
cd listfruit_project
```

Create App:

```
python manage.py startapp listfruit_app
```

Step-03: Create view.py in your app (listfruit_app//views.py)

```
from django.shortcuts import render
def fruit_student(request):
    fruitList=['Mango','Kiwi','Banana','Apple','Grapes']
    studentList=['Rama','Chetan','Kumar','Harish','Geetha']
    context = {
        'fruitList':fruitList,
        'studentList':sorted(studentList),
    }
    return render(request, 'listfruit_app/fruits_student.html', context)
```

- Here, we define a view function ‘fruit_student’ that creates two lists: fruitList and studentList. These lists are then passed to the template as a context dictionary.

Step-04: Create a template

- In your app's directory (e.g., **listfruit_app**), create a new folder named 'templates'.
- Inside the templates folder, create another folder with the same name as your app (e.g., **listfruit_app**).
- Inside the **listfruit_app** folder, create a new file named **fruits_student.html**.
- Open **fruits_student.html** and add the following code:

fruits_student.html

```
<!DOCTYPE html>
<html>
<head>
    <style>
        #a1 {background-color: lightblue;color:brown}
        #a2 {background-color:blue;color:yellow}
    </style>
    <title>
        Unordered Fruits and Ordered Students
    </title>
</head>
<body>
    <h1 id="a1">Unordered List of Fruits</h1>
    <ul>
        {% for fruit in fruitList %}
            <li>{{ fruit }} </li>
        {% endfor %}
    </ul>
    <h1 id="a2">Ordered List of Students Selected for an Event</h1>
    <ol>
        {% for student in studentList %}
            <li>{{ student }} </li>
        {% endfor %}
    </ol>
</body>
</html>
```

Step-05: Map the view function to a URL

- Create and open the urls.py file in your Django app's directory (e.g., listfruit_app/urls.py).
- Import the view function at the top of the file
- Add a new URL pattern to the urlpatterns list

```
from django.urls import path
from . import views
urlpatterns = [
    path('fruits/', views.fruit_student, name='fruit_student '),
]
```

Step-06: Add the app to INSTALLED_APPS

- Open the settings.py file in your project's directory (e.g., listfruit_project/settings.py).
- Locate the **INSTALLED_APPS** list and add the name of your new app to the list:

Step-07: Include the app's URLs in the project's URL patterns

- Open the urls.py file in your project's directory (e.g., listfruit_project/urls.py).
- Import the include function from **django.urls** and the path function from **django.urls**:

```
from django.urls import include, path
```

- Add a new URL pattern to the urlpatterns list

```
path ('', include('listfruit_app.urls')),
```

- This includes the URL patterns from your app's urls.py file.
- Include the following code:

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [

    path('admin/', admin.site.urls),

    path('',include('listfruit_app.urls')),

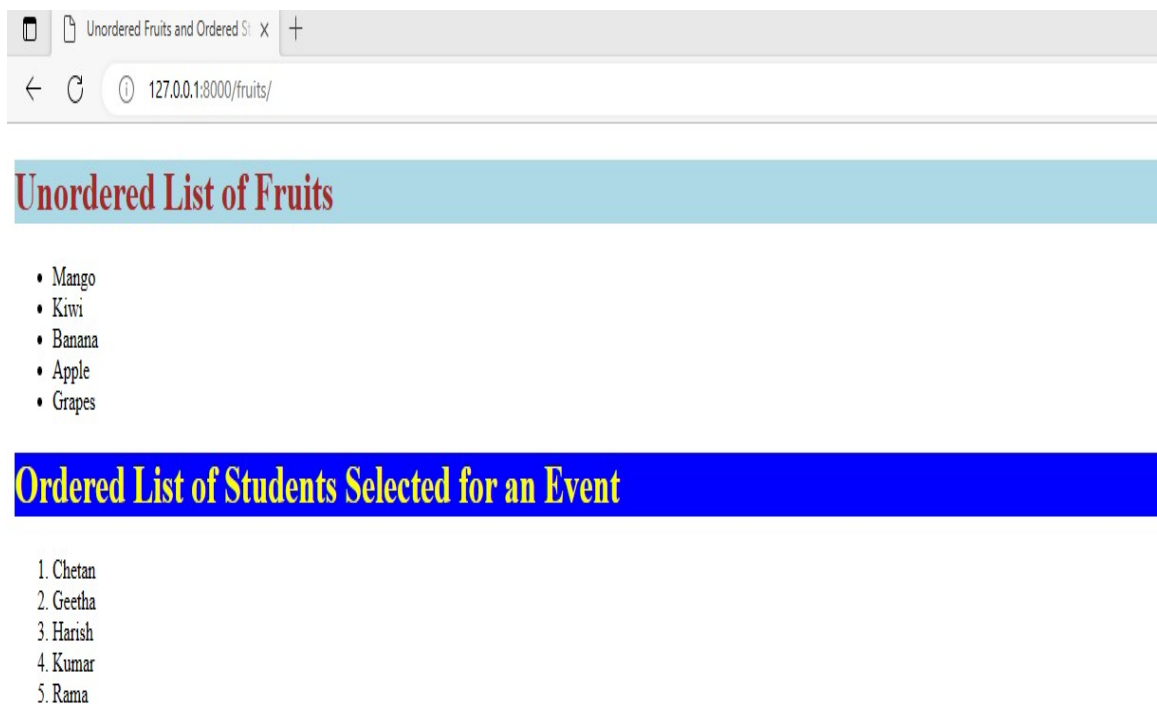
]
```

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit <http://127.0.0.1:8000/fruits/>.

OUTPUT:

Experiment-06

Develop a layout.html with a suitable header (containing navigation menu) and footer with copyright and developer information. Inherit this layout.html and create 3 additional pages: contact us, About Us and Home page of any website.

Step-01: Create project.

Step-02: Change directory and Create App. (Eg. App name: portfolioapp)

Step-03: Add the app to INSTALLED_APPS.

- Open the settings.py file in your project's directory.
- Locate the INSTALLED_APPS list and add the name of your new app to the list:

Step-04: Create a new template

- In your app's directory, create a new folder named 'templates'.
- Inside the templates folder, create another folder (portfolioapp) with the same name as your app.
- Inside the app folder, create a new files named **layout.html**, **home.html**, **about.html**, **contactus.html**
- Open **layout.html** and add the following code:

layout.html

```
<!DOCTYPE html>
<html>
<head>
    <style>
        nav{background-color: lightblue;padding: 15px;}
    </style>
    <title>
        {% block title %} {% endblock %}
    </title>
</head>
<body>
    <nav>
        <a href="{% url 'main' %}">HOME</a>
        <a href="{% url 'contact' %}">CONTACT US</a>
        <a href="{% url 'about' %}">ABOUT US</a>
    </nav>
```

```
<section>
    {% block content %} {% endblock %}
</section>
<footer>
    <hr>
    &copy; Designed and Developed by CSD Department FETC, SUK Kalburgi.
</footer>
</body>
</html>
```

home.html

```
{% extends 'portfolioapp/layout.html' %}
{% block title %} HOME Page {% endblock %}
{% block content %}
<h1>This is my home page</h1>
{% endblock %}
```

about.html

```
{% extends 'portfolioapp/layout.html' %}
{% block title %} ABOUT PAGE {% endblock %}
{% block content %}
<h1>About Us</h1>
<p> CSD Department FETC SUK KLB</p>
{% endblock %}
```

contactus.html

```
{% extends 'portfolioapp/layout.html' %}
{% block title %} Contact us {% endblock %}
{% block content %}
<h1>Contact us</h1>
<p>Name: Sharanabasappa Noola</p>
<p>Designation:Asst. Prof </p>
<p>Mobile: 8523697412</p>
<p>Email: abc@gmail.com</p>
{% endblock %}
```

Step-05: Create a view function

- Open the `views.py` file in your Django app's directory.

```
from django.shortcuts import render
def main(request):
    return render(request, 'portfolioapp/home.html')
def about(request):
    return render(request, 'portfolioapp/about.html')
def contact(request):
    return render(request, 'portfolioapp/contactus.html')
```

Step-06: Map the view function to a URL

- Open the `urls.py` file in your Django app's directory (e.g., `portfolioapp/urls.py`).
- Import the view function at the top of the file
- Add a new URL pattern to the `urlpatterns` list

```
from django.urls import path
from . import views
urlpatterns = [
    path('main/', views.main, name='main'),
    path('about/', views.about, name='about'),
    path('contact/', views.contact, name='contact'),
]
```

Step-07: Include the app's URLs in the project's URL patterns

- Open the `urls.py` file in your project's directory.
- Import the `include` function from **django.urls** and the `path` function from **django.urls**:
- Add a new URL pattern to the `urlpatterns` list

```
from django.urls import include, path [if does not exists]

path("", include(portfolioapp.urls)),
```

- This includes the URL patterns from your app's `urls.py` file.
- Include the following code:

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [
```

```
path('admin/', admin.site.urls),  
path('',include('portfolioapp.urls')),  
]
```

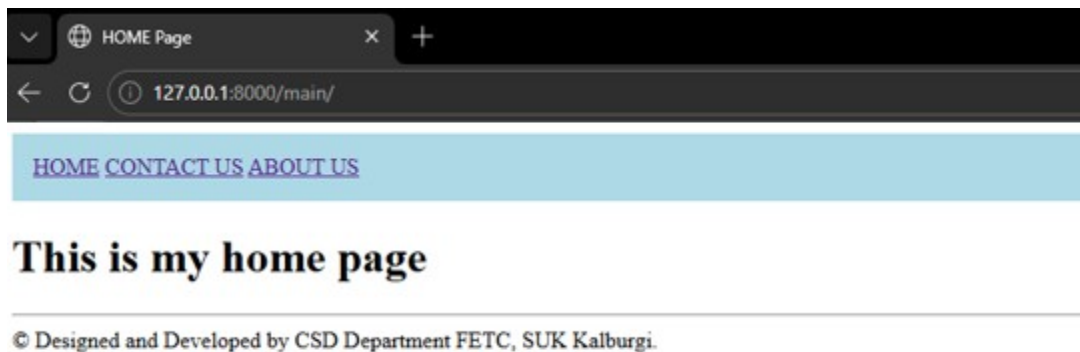
Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

```
python manage.py runserver
```

- Open your web browser and visit <http://127.0.0.1:8000/main/>.

OUTPUT:





Experiment-07

Develop a Django app that performs student registration to a course. It should also display list of students registered for any selected course. Create students and course as models with enrolment as ManyToMany field.

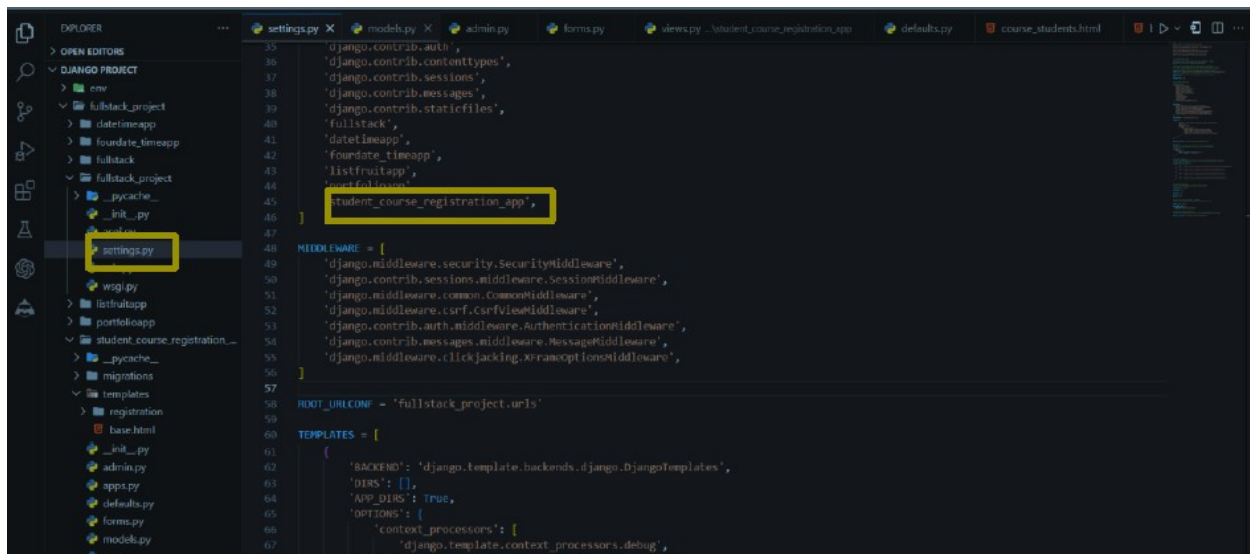
Step-01: Create project.

Step-02: Change directory and Create App. (Eg. App name: student_course_registration_app)

Step-03: Add the app to INSTALLED_APPS

Open the settings.py file in your project's directory.

Locate the **INSTALLED_APPS** list and add the name of your new app to the list:



Step-04: Create models

- Open the **models.py** file inside the student_course_registration_app and define your models:

from django.db import models

class Course(models.Model):

name = models.CharField(max_length=255)

```
description = models.TextField(blank=True, null=True)
```

```
def __str__(self):
```

```
    return self.name
```

```
class Student(models.Model):
```

```
    first_name = models.CharField(max_length=255, default='')
```

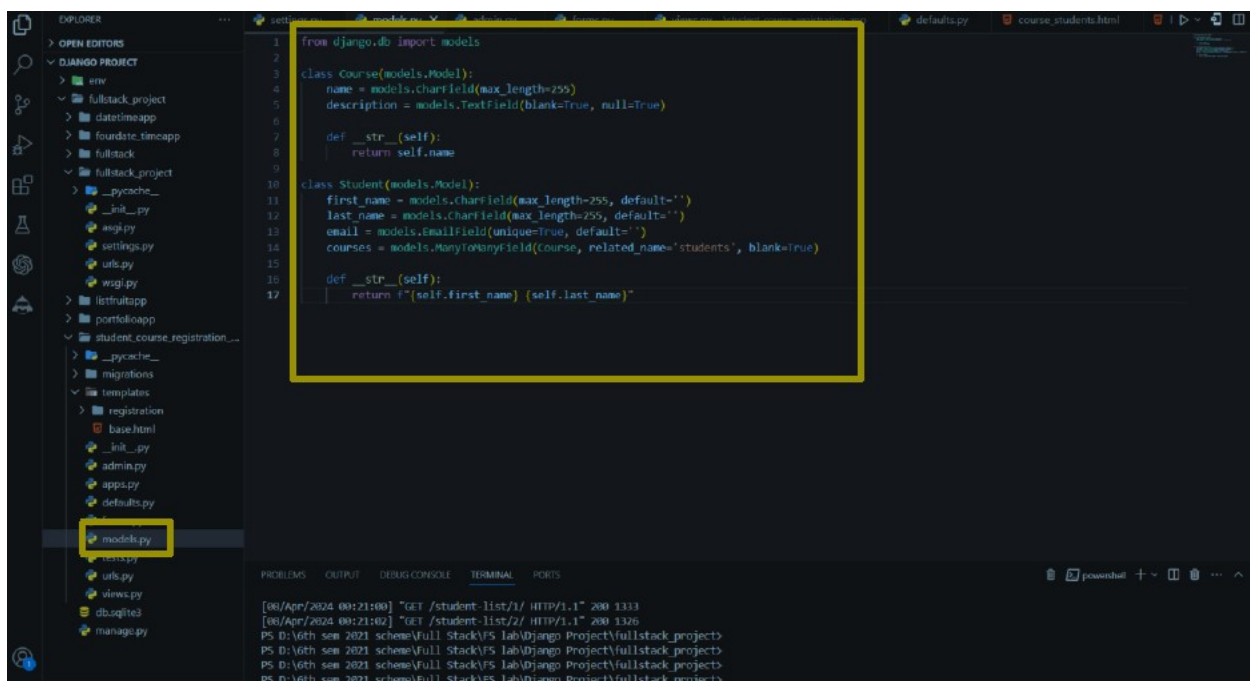
```
    last_name = models.CharField(max_length=255, default='')
```

```
    email = models.EmailField(unique=True, default='')
```

```
    courses = models.ManyToManyField(Course, related_name='students', blank=True)
```

```
def __str__(self):
```

```
    return f'{self.first_name} {self.last_name}'
```



Step-05: Register models in the admin site

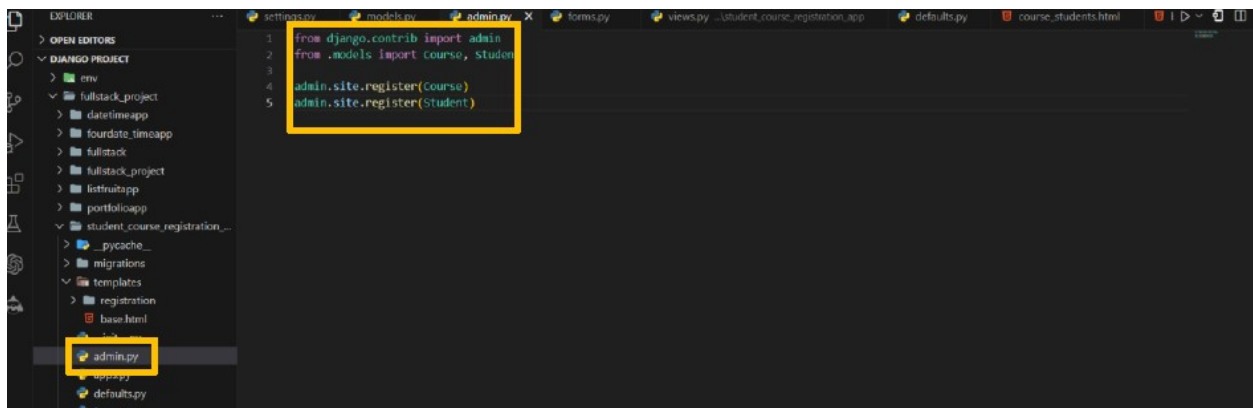
- Open the admin.py file inside the student_course_registration_app and register your models:

```
from django.contrib import admin
```

```
from .models import Course, Student
```

```
admin.site.register(Course)
```

```
admin.site.register(Student)
```

**Step-06: Create forms**

- Create a new file called **forms.py** inside the student_course_registration_app and define your forms:

```
from django import forms
```

```
from .models import Student, Course
```

```
class StudentForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Student
```

```
        fields = ['first_name', 'last_name', 'email', 'courses']
```

```
        widgets = {
```

```
            'first_name': forms.TextInput(attrs={'class': 'form-control'}),
```

```
'last_name': forms.TextInput(attrs={'class': 'form-control'}),  
  
'email': forms.EmailInput(attrs={'class': 'form-control'}),  
  
'courses': forms.SelectMultiple(attrs={'class': 'form-select', 'data-toggle': 'dropdown'}),  
  
}
```

```
class CourseForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Course
```

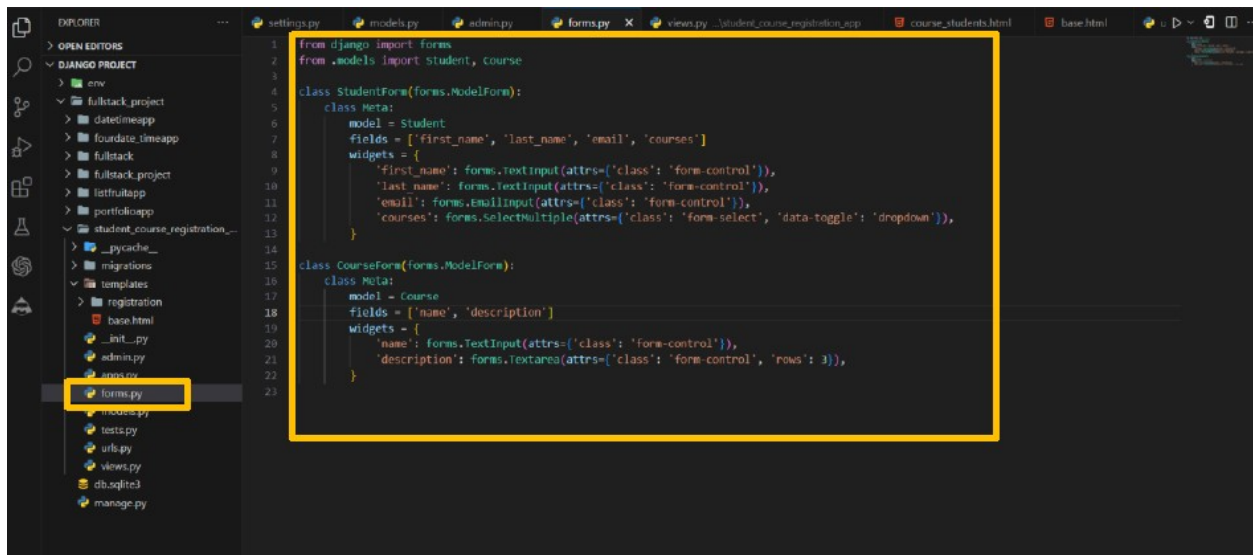
```
        fields = ['name', 'description']
```

```
        widgets = {
```

```
            'name': forms.TextInput(attrs={'class': 'form-control'}),
```

```
            'description': forms.Textarea(attrs={'class': 'form-control', 'rows': 3}),
```

```
        }
```



Step-07: Create views

- Open the **views.py** file inside the `student_course_registration_app` and define your views:

```
from django.shortcuts import render, redirect

from .models import Course, Student

from .forms import StudentForm, CourseForm

def index(request):

    courses = Course.objects.all()

    return render(request, 'registration/index.html', {'courses': courses})

def register_student(request):

    if request.method == 'POST':

        form = StudentForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('index')

    else:

        form = StudentForm()

    return render(request, 'registration/register_student.html', {'form': form})

def register_course(request):

    if request.method == 'POST':

        form = CourseForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('index')

    else:
```

```

form = CourseForm()

return render(request, 'registration/register_course.html', {'form': form})

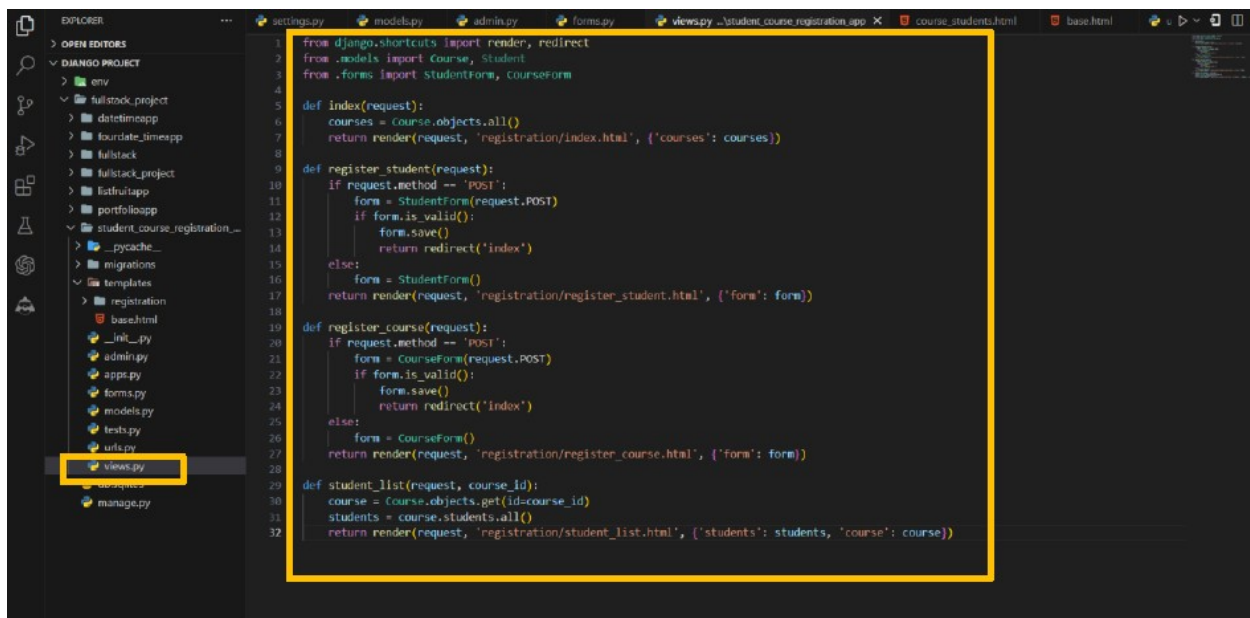
def student_list(request, course_id):

    course = Course.objects.get(id=course_id)

    students = course.students.all()

    return render(request, 'registration/student_list.html', {'students': students, 'course':
course})

```



Step-08: Create templates

- Create a new directory called templates inside your student_course_registration_app, and create the following template files:

index.html

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div class="container">
```

```
<h1 class="mt-5">Courses</h1>
```

```
<ul class="list-group mt-3">
```

```
{% for course in courses %}
```

```
<li class="list-group-item"><a href="{% url 'student_list' course.id %}">{{ course.name }}</a></li>
```

```
{% endfor %}
```

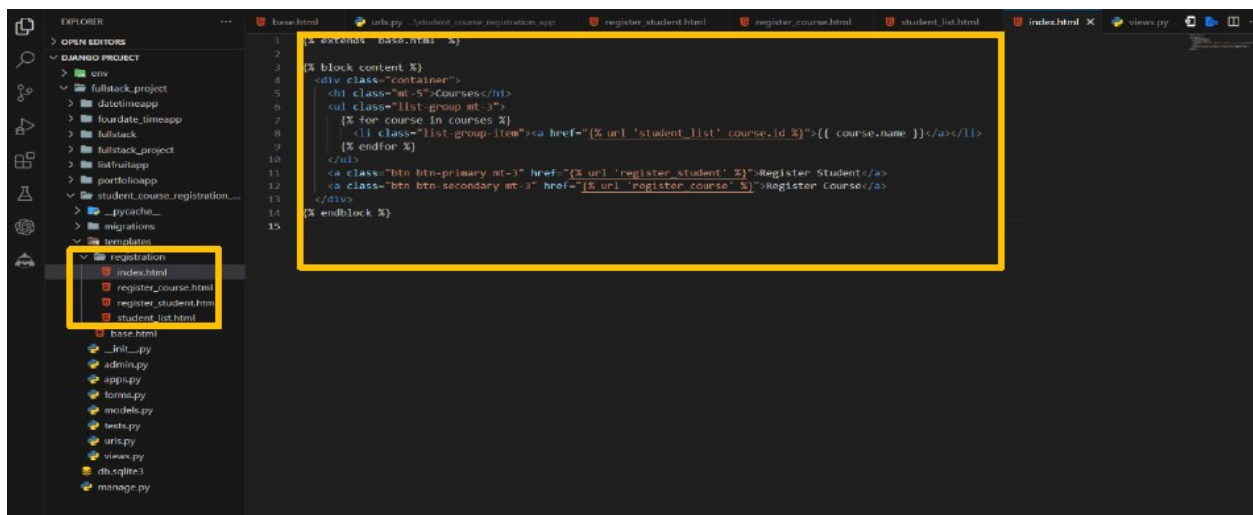
```
</ul>
```

```
<a class="btn btn-primary mt-3" href="{% url 'register_student' %}">Register Student</a>
```

```
<a class="btn btn-secondary mt-3" href="{% url 'register_course' %}">Register Course</a>
```

```
</div>
```

```
{% endblock %}
```



register_student.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Student Registration</title>

<!-- Bootstrap CSS -->

<link                                href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/css/bootstrap.min.css" rel="stylesheet">

</head>

<body>

<div class="container">

    <h1>Register Student</h1>

    <form method="post">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit" class="btn btn-primary">Register</button>

    </form>

</div>

<!-- Bootstrap JavaScript (Optional) -->

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>

<!-- Your custom JavaScript to initialize the dropdown (Optional) -->

<script>

    // Example JavaScript to initialize the dropdown

    var dropdownElementList = [].slice.call(document.querySelectorAll('.dropdown-toggle'))

    var dropdownList = dropdownElementList.map(function (dropdownToggleEl) {

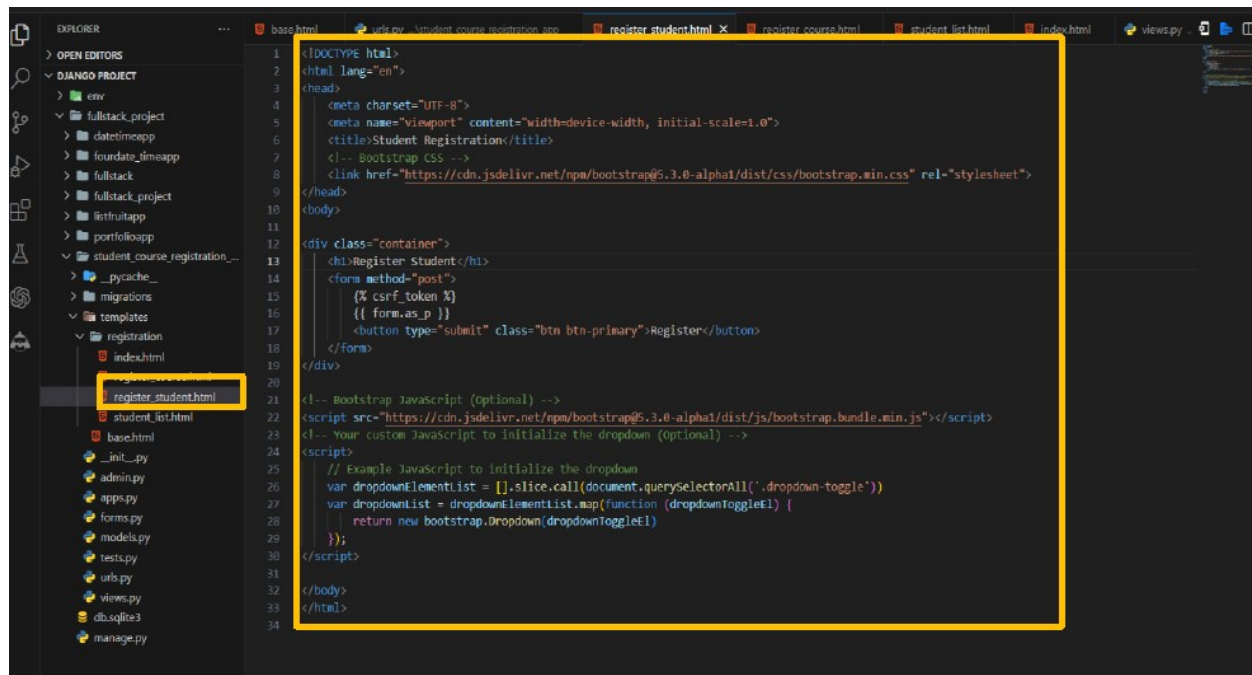
        return new bootstrap.Dropdown(dropdownToggleEl)

    });

</script>

</body>
```

</html>

**register_course.html**

{% extends 'base.html' %}

{% load static %}

{% block content %}

<h1>Register Course</h1>

<div class="container">

```

        {% if field.help_text %}

            <small class="form-text text-muted">{{
                field.help_text }}</small>

        {% endif %}

        {% for error in field.errors %}

            <div class="alert alert-danger">{{ error
                }}</div>

        {% endfor %}

    </div>

    {% endfor %}

    <button type="submit" class="btn btn-primary text-
center">Register</button>

</form>

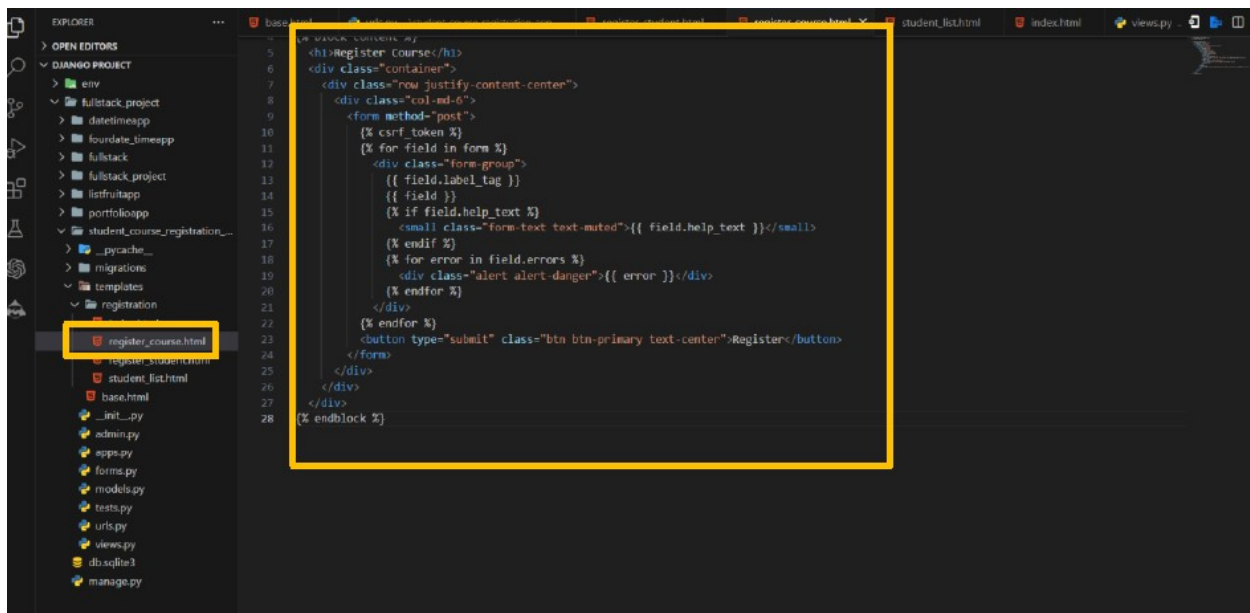
</div>

</div>

</div>

{% endblock %}

```



student_list.html

```
{% extends 'base.html' %}

{% block content %}

    <h1>Students Registered for {{ course.name }}</h1>

    <ul>

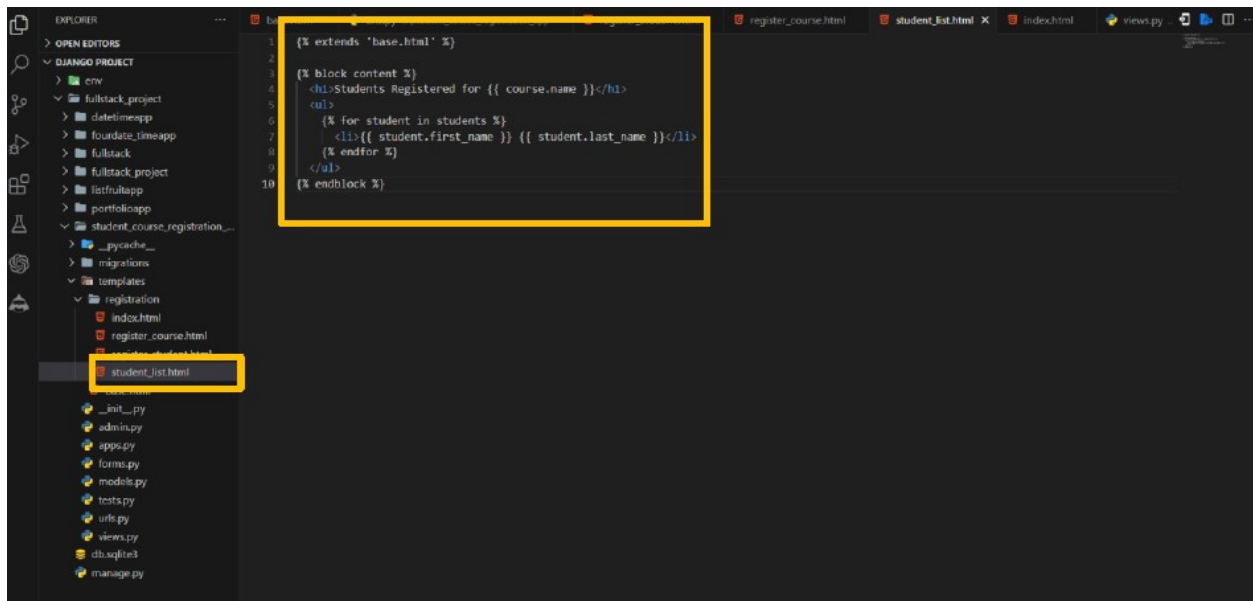
        {% for student in students %}

            <li>{{ student.first_name }} {{ student.last_name }}</li>

        {% endfor %}

    </ul>

{% endblock %}
```

**Step-09: Create a base template**

- Create a **base.html** file inside the templates directory with the following content:

base.html

```
<!DOCTYPE html>

<html>
```

```

<head>

<title>Student Registration</title>

</head>

<body>

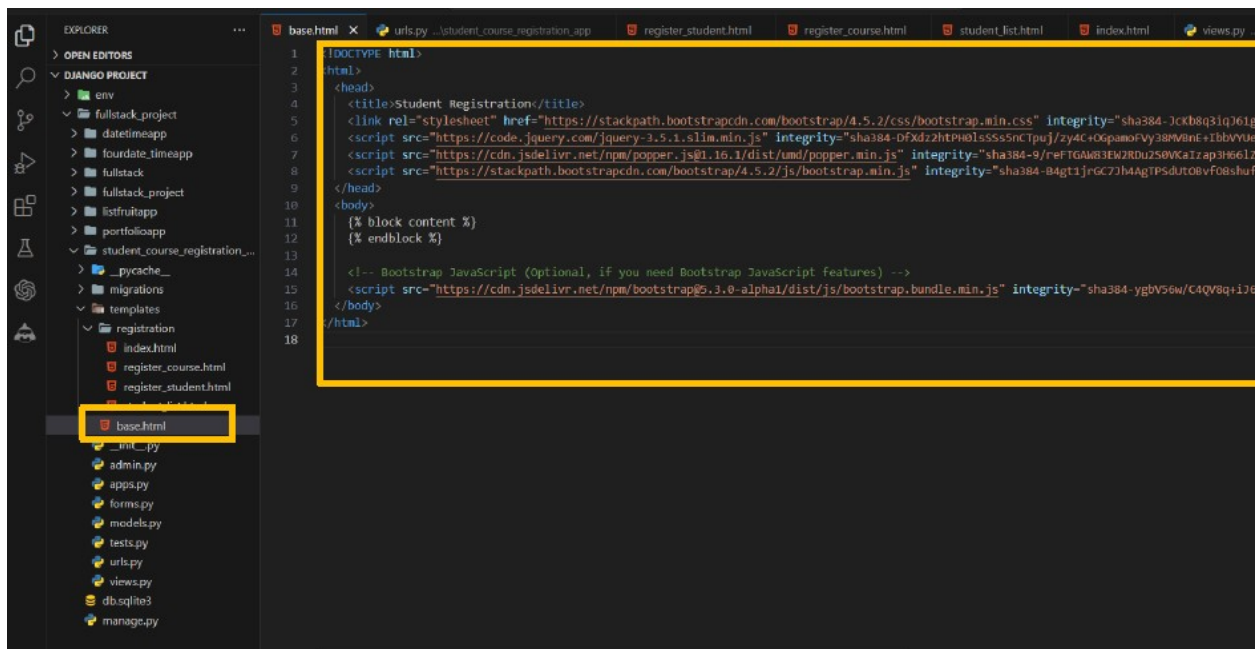
    {% block content %}

    {% endblock %}

</body>

</html>

```



Step-10: Configure URLs

- Open the `urls.py` file inside your `student_course_registration_app` and define your URLs:

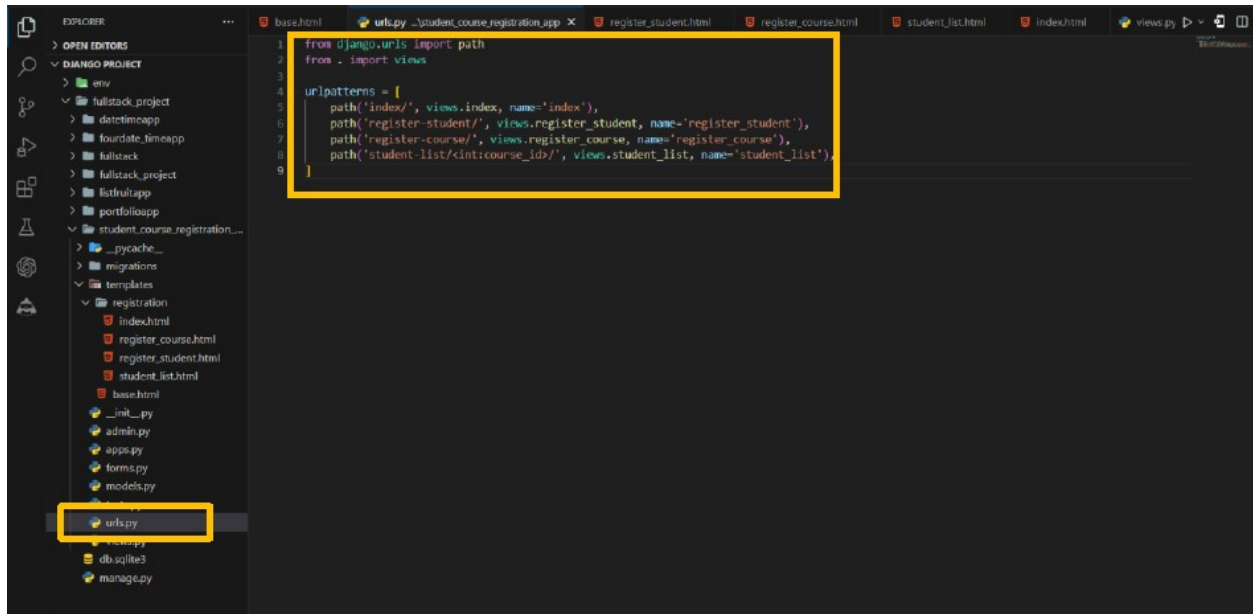
```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

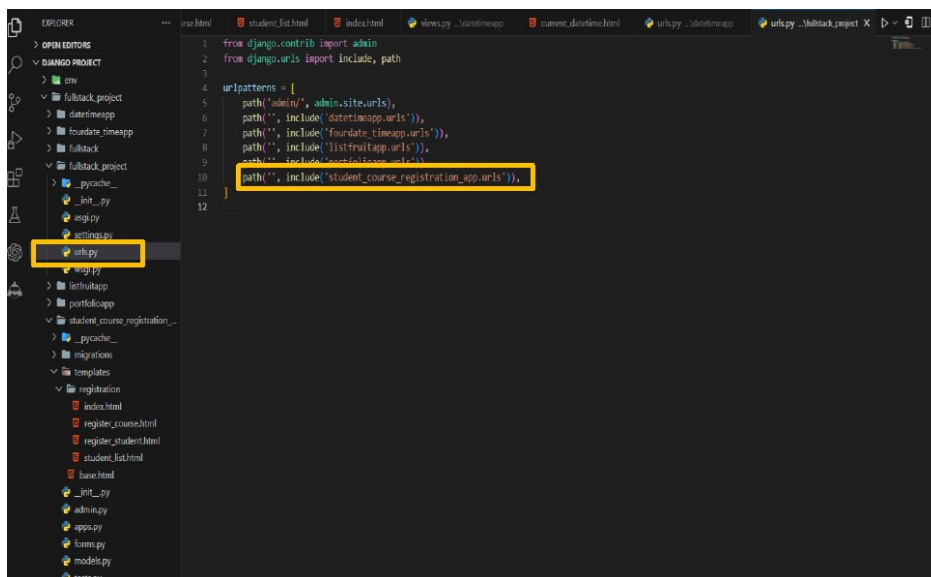
```
    path('index/', views.index, name='index'),
```

```
path('register-student/', views.register_student, name='register_student'),  
path('register-course/', views.register_course, name='register_course'),  
path('student-list/<int:course_id>/', views.student_list, name='student_list'),  
]
```



Step-11: Update project URLs

- Open the urls.py file inside your project and include the URLs from the student_course_registration_app:



Step-12: Make Migration for check Models saved or not into the database

```
PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations
Migrations for 'student_course_registration_app':
  student_course_registration_app\migrations\0002_remove_student_name_student_courses_and_more.py
  - Remove field name from student
  - Add field courses to student
  - Add field first_name to student
  - Add field last_name to student
  - Alter field description on course
  - Alter field name on course
  - Alter field email on student
  - Delete model Enrollment
```

Step-13: Migrate To Save the Models into database

```
PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, student_course_registration_app
Running migrations:
  Applying student_course_registration_app.0002_remove_student_name_student_courses_and_more... OK
```

Step-14: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.
- Type or copy this **http://127.0.0.1:8000/index/**

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 16:02:28
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[]
```

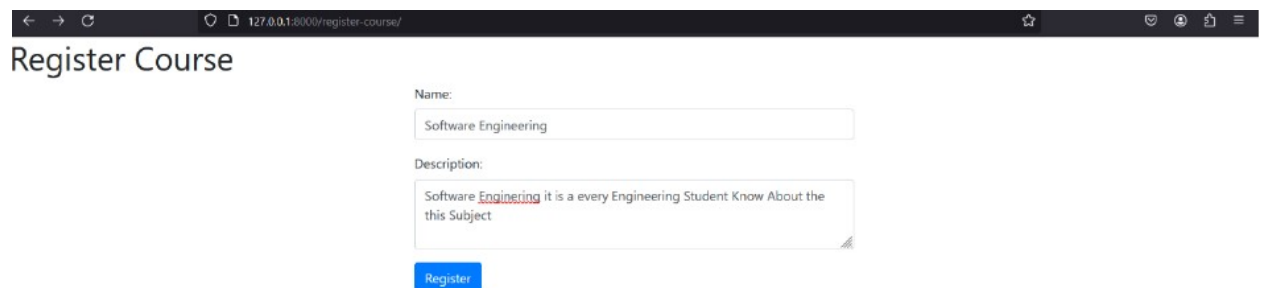
Final Output By Clicking Register Student and Register Course**Fig: Index Screen****Fig: Student Register Screen****Fig: Course Register Screen**



Fig: Students Register Particular Courses Screen [You can Check as You Register Courses]

Experiment-08

For student and course models created in Lab experiment for Module2, register admin interfaces, perform migrations and illustrate data entry through admin forms.

Step-01: Register models in the admin site

```
from django.contrib import admin

from .models import Course, Student

@admin.register(Course)

class CourseAdmin(admin.ModelAdmin):

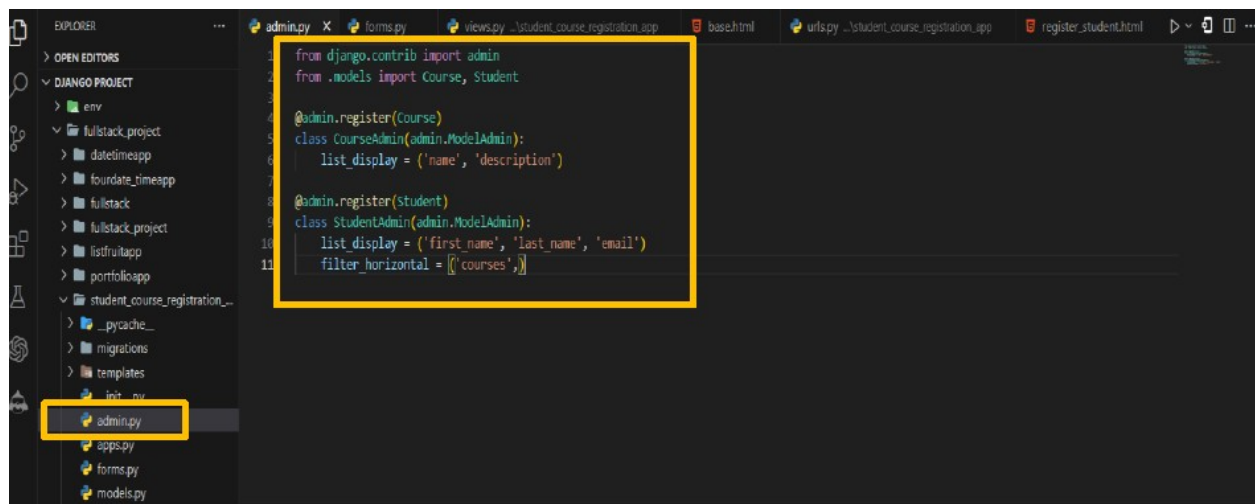
    list_display = ('name', 'description')

@admin.register(Student)

class StudentAdmin(admin.ModelAdmin):

    list_display = ('first_name', 'last_name', 'email')

    filter_horizontal = ('courses',)
```



- In this code, we've registered the Course and Student models with the Django admin site.
- We've also specified the fields to be displayed in the list view for each model using the list_display attribute.
- For the StudentAdmin class, we've added the filter_horizontal attribute to display the courses field (which is a many-to-many relationship) as a horizontal filter in the admin interface.

Step-02: Create and apply migrations

python manage.py makemigrations

```
no such file or directory
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project> cd fullstack_project
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations
No changes detected
```

python manage.py migrate

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, student_course_registration_app
Running migrations:
  No migrations to apply.
```

Step-03: Create a superuser

- If you haven't already created a superuser for your project, you'll need to do so to access the admin interface.
- In your terminal or command prompt, run the following command

python manage.py createsuperuser

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py createsuperuser
Username (leave blank to use 'aryah'): admin
Email address:
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Type Username: admin

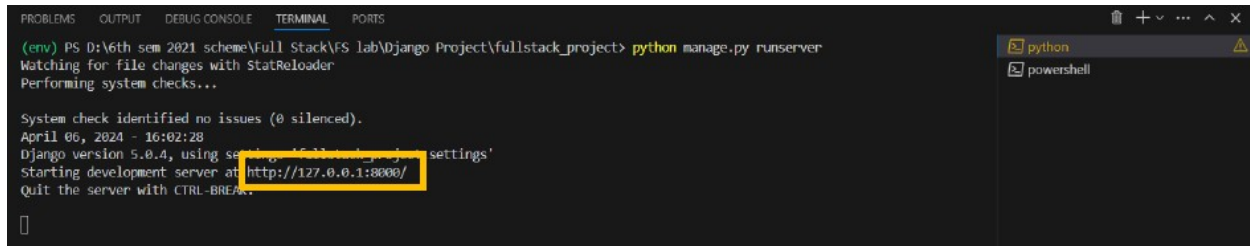
Password:1234 [For Simply but your wish to create your own username and password]

Step-04: Access the admin interface

- Start the Django development server by running the following command:
- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.



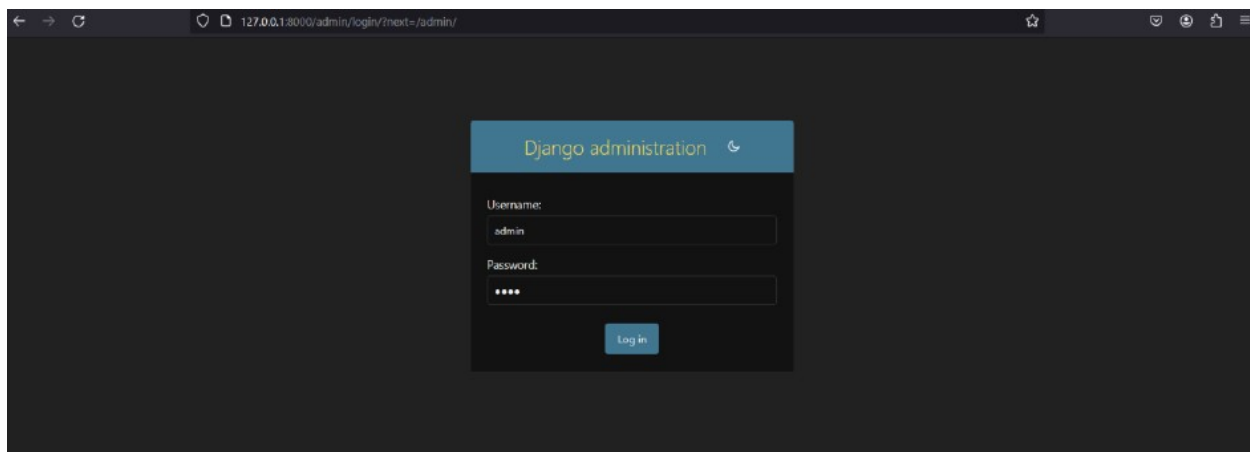
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 16:02:28
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

- Type or copy this **<http://127.0.0.1:8000/admin/>**

Step-05: Log in to the admin interface

- Back in your web browser, enter the superuser credentials you just created and log in to the admin interface.



Step-06: Add data through admin forms

Once you're logged in to the admin interface, you'll see the "Courses" and "Students" sections in the left sidebar. Click on "Courses" to add a new course.

- Click on the "Add course" button in the top-right corner.
- Fill in the "Name" and "Description" fields for the new course.
- Click the "Save" button to create the new course.

Next, click on "Students" in the left sidebar to add a new student.

- Click on the "Add student" button in the top-right corner.
- Fill in the "First name", "Last name", and "Email" fields for the new student.
- In the "Courses" section, select the courses you want to enroll the student in by checking the appropriate boxes.
- Click the "Save" button to create the new student.

You can repeat these steps to add more courses and students through the admin interface. Additionally, you can view, edit, and delete existing courses and students from the admin interface.

Final Output

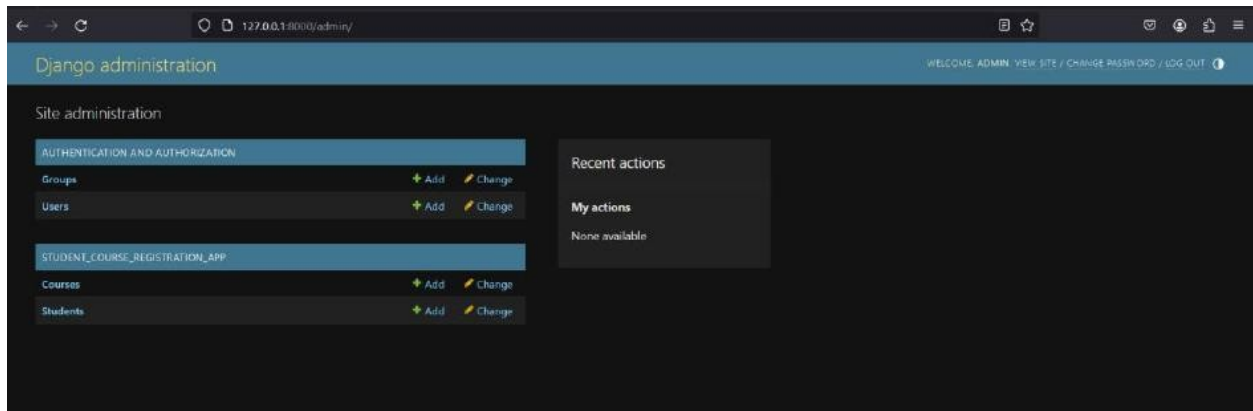


Fig: Admin Main Screen

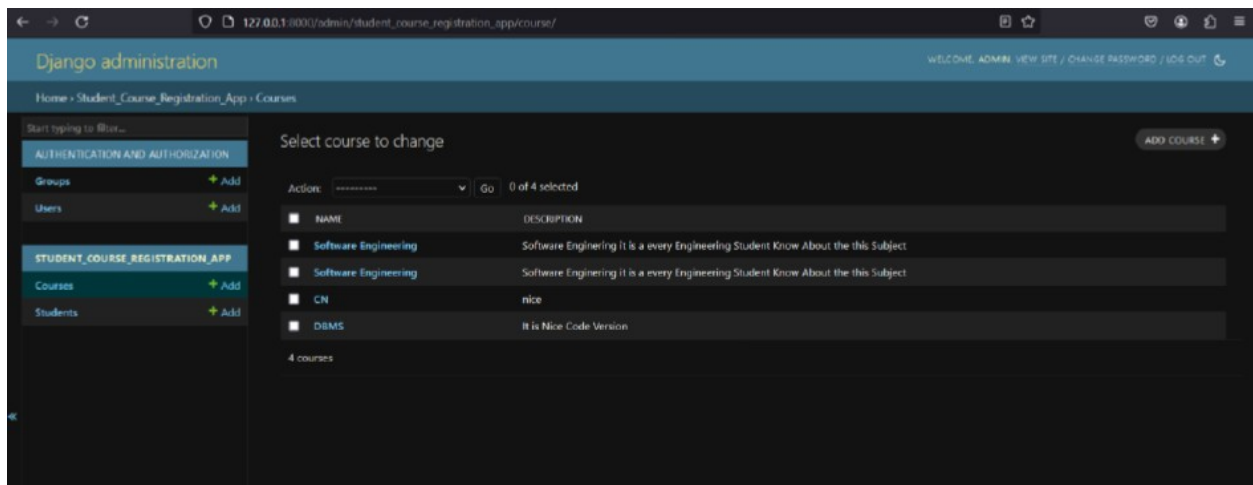
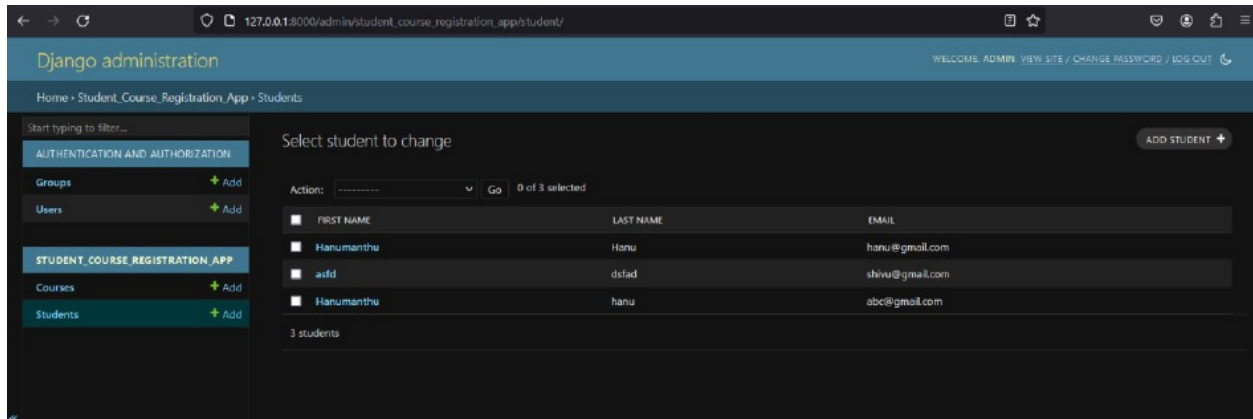


Fig: Admin Course Screen

**Fig: Admin Student Screen**

Experiment-09

Develop a Model form for student that contains his topic chosen for project, languages used and duration with a model called project.

Step 1: Use an existing app i.e. student_course_registration_app

```
[08/Apr/2024 00:21:00] "GET /student-list/1/ HTTP/1.1" 200 1333
[08/Apr/2024 00:21:02] "GET /student-list/2/ HTTP/1.1" 200 1326
PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations student_course_registration_app
```

Step 2: Open the models.py file in the projects app, add the below line in the Student model.

```
project = models.ManyToManyField('Project', related_name='students', blank=True)
```

Define the Project model

```
class Project(models.Model):
```

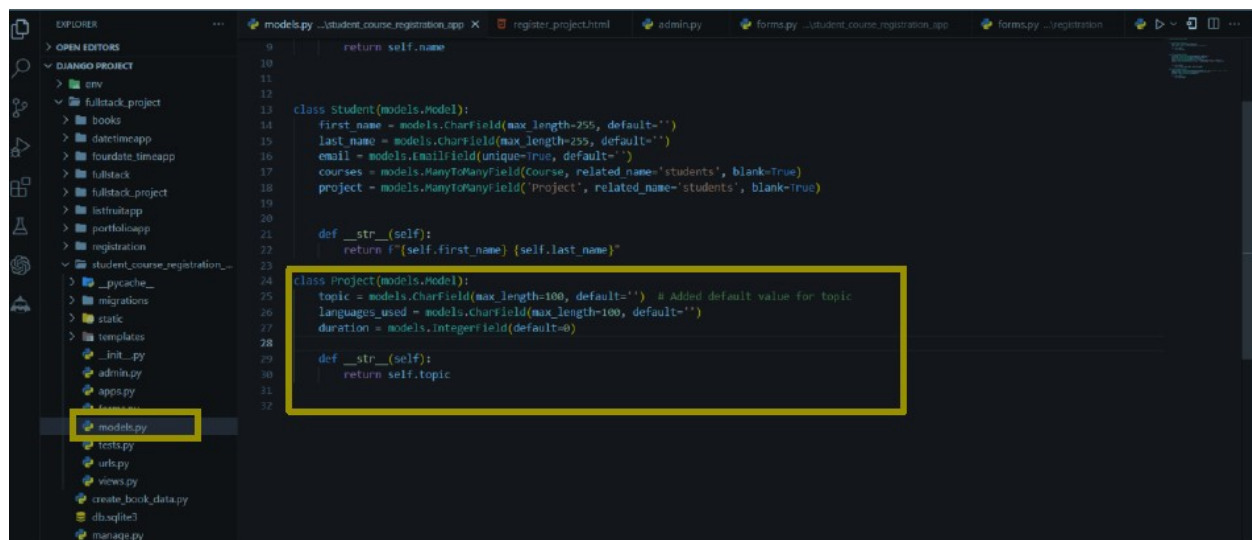
```
    topic = models.CharField(max_length=100, default='') # Added default value for topic
```

```
    languages_used = models.CharField(max_length=100, default='')
```

```
    duration = models.IntegerField(default=0)
```

```
    def __str__(self):
```

```
        return self.topic
```



Step 3: Create and apply migrations to create the necessary database tables.**python manage.py makemigrations**

```
No such file or directory
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> cd fullstack_project
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project\fullstack_project> python manage.py makemigrations
No changes detected
```

python manage.py migrate

```
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, student_course_registration_app
Running migrations:
  No migrations to apply.
```

Step 4: In the existing forms.py file in the projects app, define a ProjectForm based on the Project model.

```
class ProjectForm(forms.ModelForm):

    class Meta:

        model = Project

        fields = ('topic', 'languages_used', 'duration')

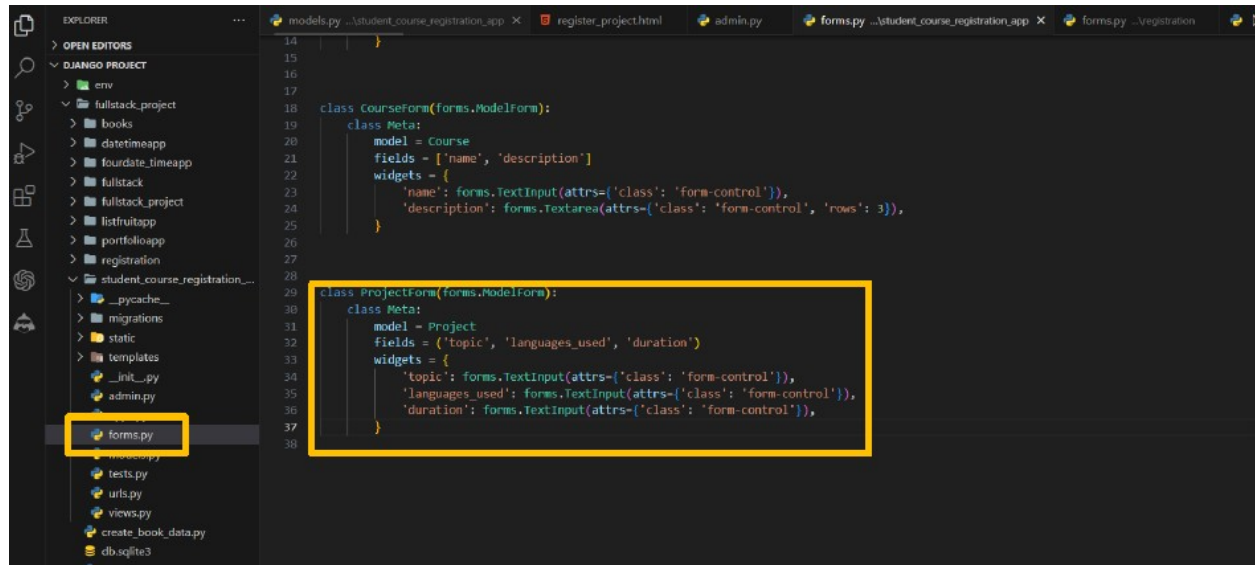
        widgets = {

            'topic': forms.TextInput(attrs={'class': 'form-control'}),

            'languages_used': forms.TextInput(attrs={'class': 'form-control'}),

            'duration': forms.TextInput(attrs={'class': 'form-control'}),

        }
```



Step 5: In views.py view function, import the ProjectForm and handle the form submission.

from .forms import ProjectForm

def register_project(request):

 if request.method == 'POST':

 form = ProjectForm(request.POST)

 if form.is_valid():

 project = form.save()

 # Redirect to a success page or another view

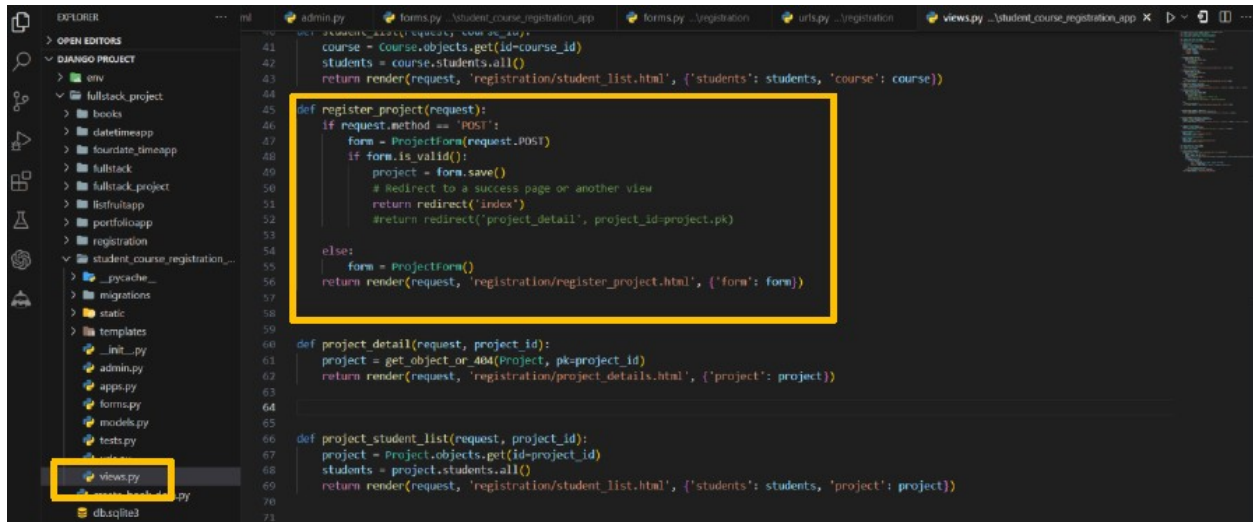
 return redirect('index')

 #return redirect('project_detail', project_id=project.pk)

 else:

 form = ProjectForm()

 return render(request, 'registration/register_project.html', {'form': form})

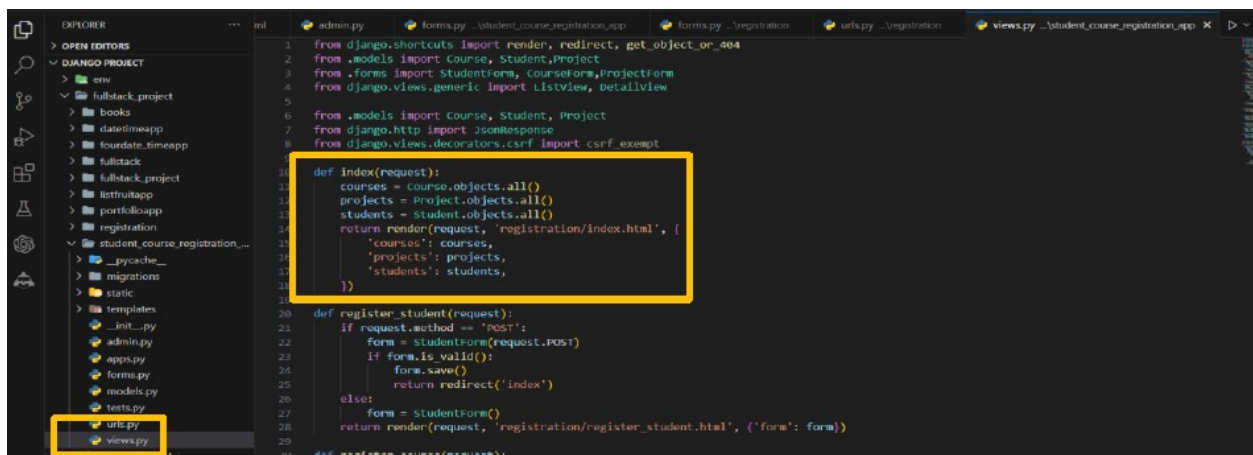


Step 6: In views.py index function, add the projects Objects for Displaying the Register Projects .

```

def index(request):
    courses = Course.objects.all()
    projects = Project.objects.all()
    return render(request, 'registration/index.html', {
        'courses': courses,
        'projects': projects,
    })

```



Step 7: Create an HTML template file register_project.html in the templates/registration/ directory and render the form.

```
<!-- create_project.html -->

{% extends 'base.html' %}

{% block content %}

<div class="container my-3">

    <h1>Create Project</h1>

    <form method="post">

        {% csrf_token %}

        <div class="form-group">

            <label for="id_topic">Topic:</label>

            {{ form.topic }}

        </div>

        <div class="form-group">

            <label for="id_languages_used">Languages Used:</label>

            {{ form.languages_used }}

        </div>

        <div class="form-group">

            <label for="id_duration">Duration:</label>

            {{ form.duration }}

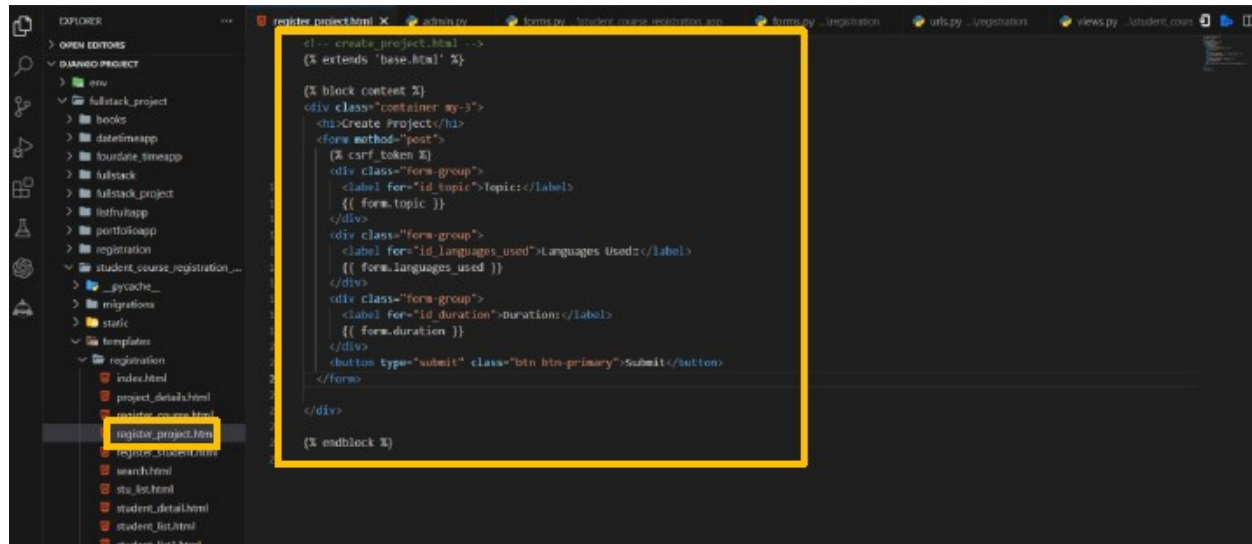
        </div>

        <button type="submit" class="btn btn-primary">Submit</button>

    </form>

</div>

{% endblock %}
```



Step 8: add the below code in index.html to show the registered projects

```
<h1 class="mt-5 my-5">Projects</h1>
```

```
<ul class="list-group mt-3">
```

```
  {% for project in projects %}
```

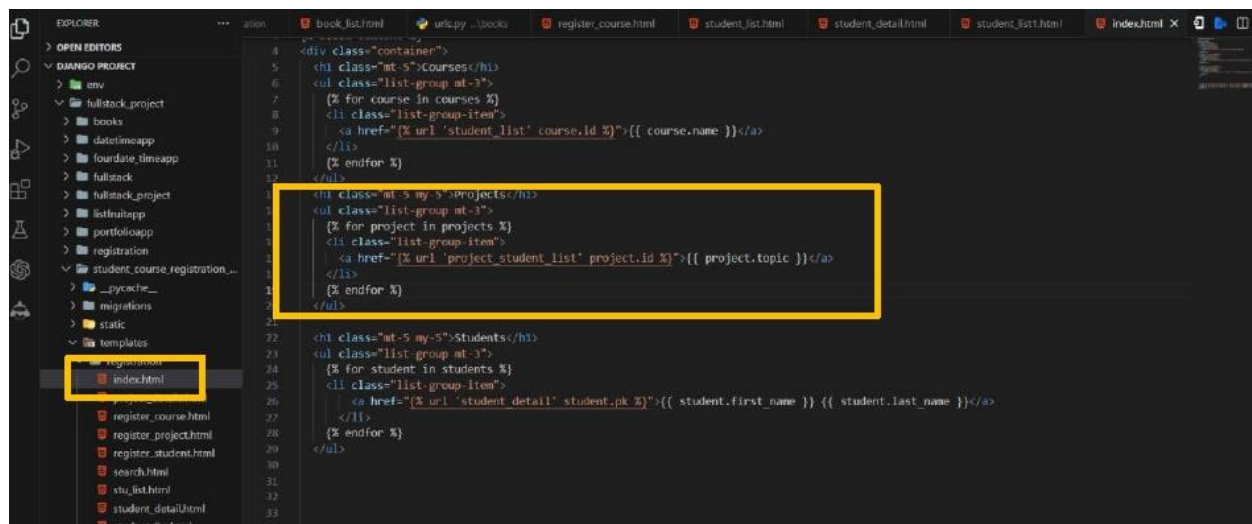
```
    <li class="list-group-item">
```

```
      <a href="{% url 'project_student_list' project.id %}">{{ project.topic }}</a>
```

```
    </li>
```

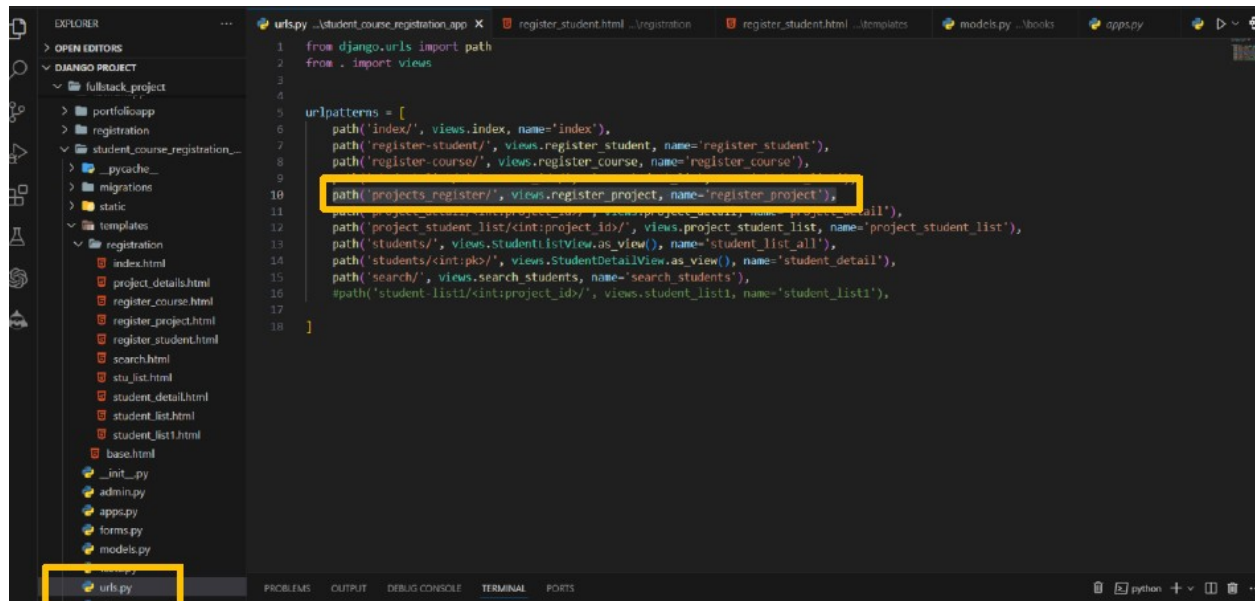
```
  {% endfor %}
```

```
</ul>
```



Step 9: Add a URL pattern for the register_project in your urls.py file

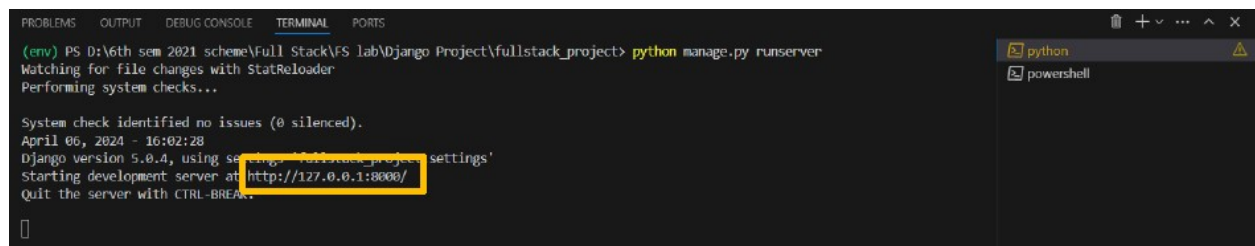
`path('projects_register/', views.register_project, name='register_project'),`

**Step-10: Run the development server**

- In the VS Code terminal, navigate to your Django project's directory (the one containing `manage.py`).
- Run the development server

python manage.py runserver

- Open your web browser and visit **`http://127.0.0.1:8000/`**.



- Type or copy this **`http://127.0.0.1:8000/index/`**

Final Output By Clicking Register Project and Register Student Projects

Covide vaccine
child Vaccine System

Students

Hanumanthu hanu
asfd dsfad
Hanumanthu Hanu
hh df
hanumanthu hanu
hello how
gangu gangu
nagesh nagu

[Register Student](#) [Register Course](#) [Register Project](#)

Fig: Home Page Screen

Create Project

Topic:

Languages Used:

Duration:

[Submit](#)

Fig: Project Creation Screen

Software Engineering
Software Engineering

Projects

Covide vaccine
child Vaccine System
Login App

Fig: Project Successfully Creation Screen

Student Registration

First name:

Demo

Last name:

demo

Email:

demo@gmail.com

Courses:

- DBMS
- CN
- Software Engineering
- Software Engineering

Project:

- Covide vaccine
- child Vaccine System
- Login App

Register

Fig: Student Register the Project

Students Registered for

• Demo demo

Fig: Student Registered Particular Projects

Experiment-10

For students enrolment developed in Module 2, create a generic class view which displays list of students and detail view that displays student details for any selected student in the list.

Step-1: First, let's create models for Student and Course if you haven't already:

Models.py

class Student(models.Model):

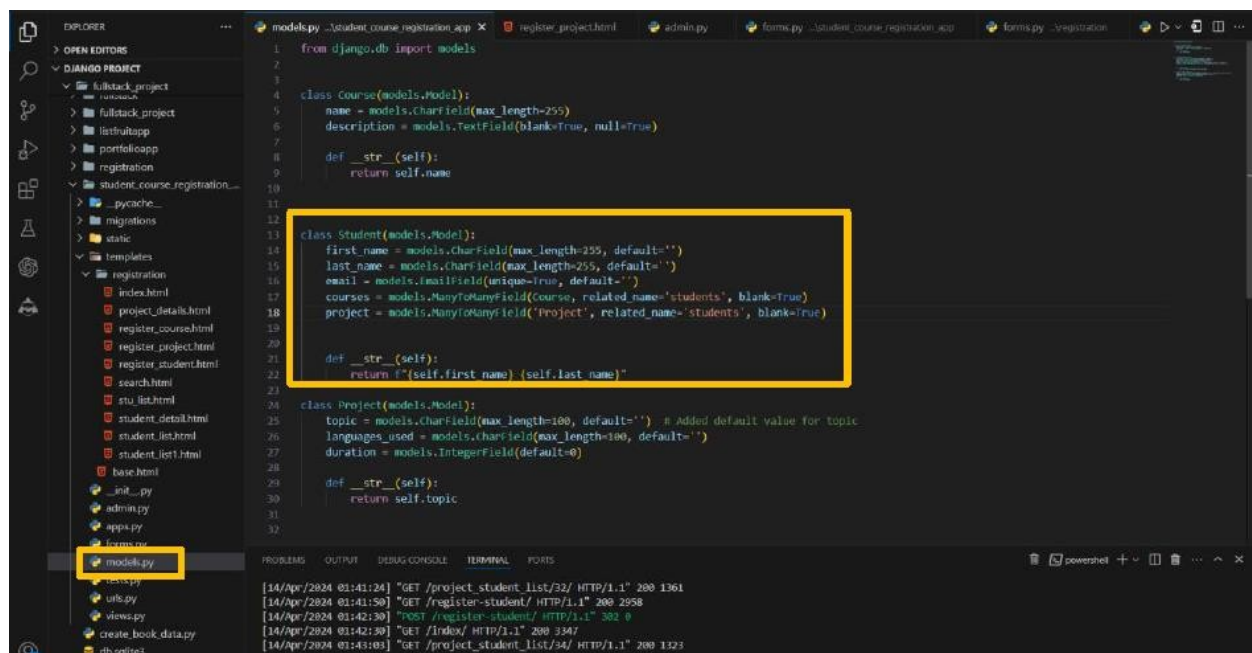
 first_name = models.CharField(max_length=255, default="")

 last_name = models.CharField(max_length=255, default="")

 email = models.EmailField(unique=True, default="")

 courses = models.ManyToManyField(Course, related_name='students', blank=True)

 project = models.ManyToManyField('Project', related_name='students', blank=True)



Step-2: Next, create views for the list and detail views and import necessary packages:

from django.views.generic import ListView, DetailView

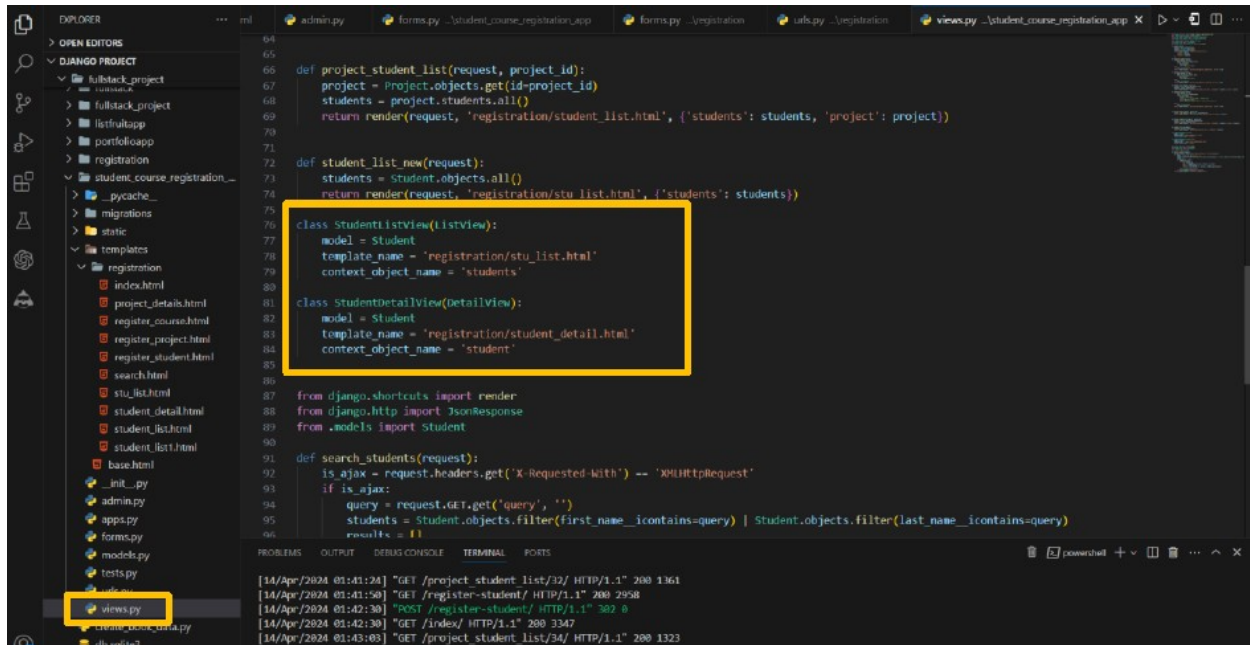
class StudentListView(ListView):

 model = Student

 template_name = 'registration/stu_list.html'

 context_object_name = 'students'

```
class StudentDetailView(DetailView):
    model = Student
    template_name = 'registration/student_detail.html'
    context_object_name = 'student'
```



- In the StudentListView, we've overridden the get_queryset method to filter the students based on the course_id parameter in the URL. If the course_id is will return all students.

Step-3: Now, let's create the templates:

stu_list.html

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<h1>Students</h1>
```

```
<ul>
```

```
{% for student in students %}
```

```
<li>
```

```
<a href="{% url 'student_detail' student.pk %}">{{
student.first_name }} {{ student.last_name }}

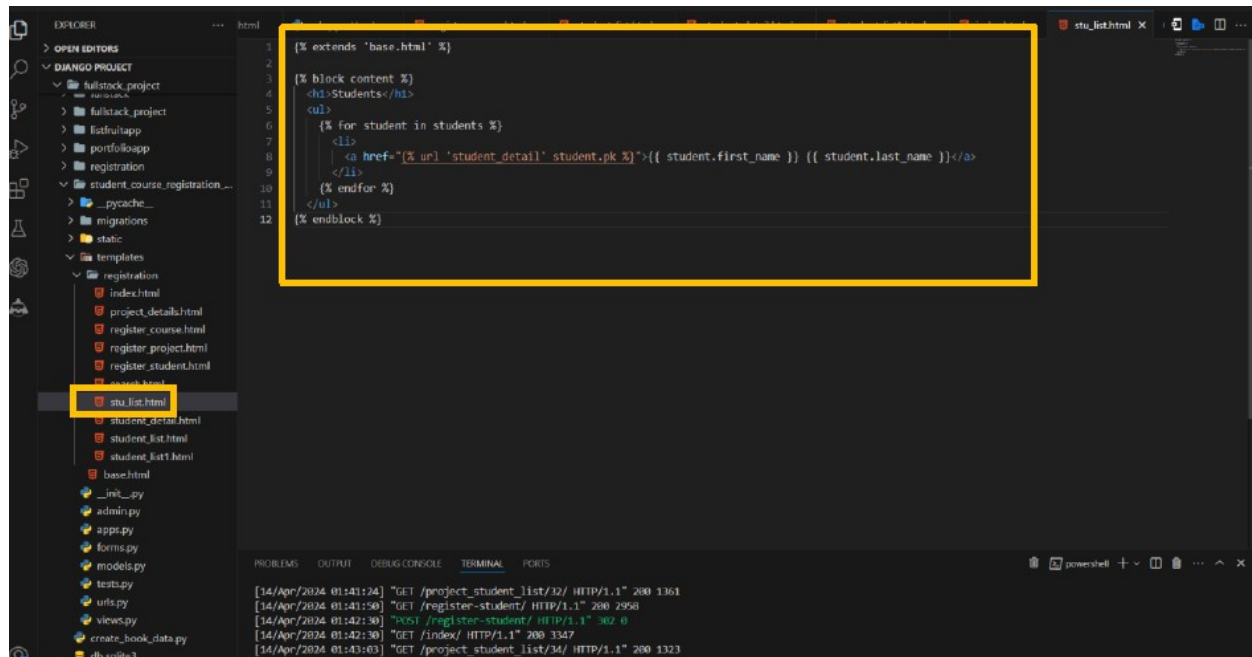
```

```
</li>
```

```
{% endfor %}
```

```
</ul>
```

```
{% endblock %}
```



student_detail.html

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div class="container my-5">
```

```
<div class="row">
```

```
<div class="col-md-6 offset-md-3">
```

```
<div class="card">
```

```
<div class="card-body">
```

```
<h1 class="card-title"> Name: {{ student.first_name }} {{
student.last_name }}

```

```
<h1 class="card-title">Email: {{ student.email }}

```

```
</div>
```

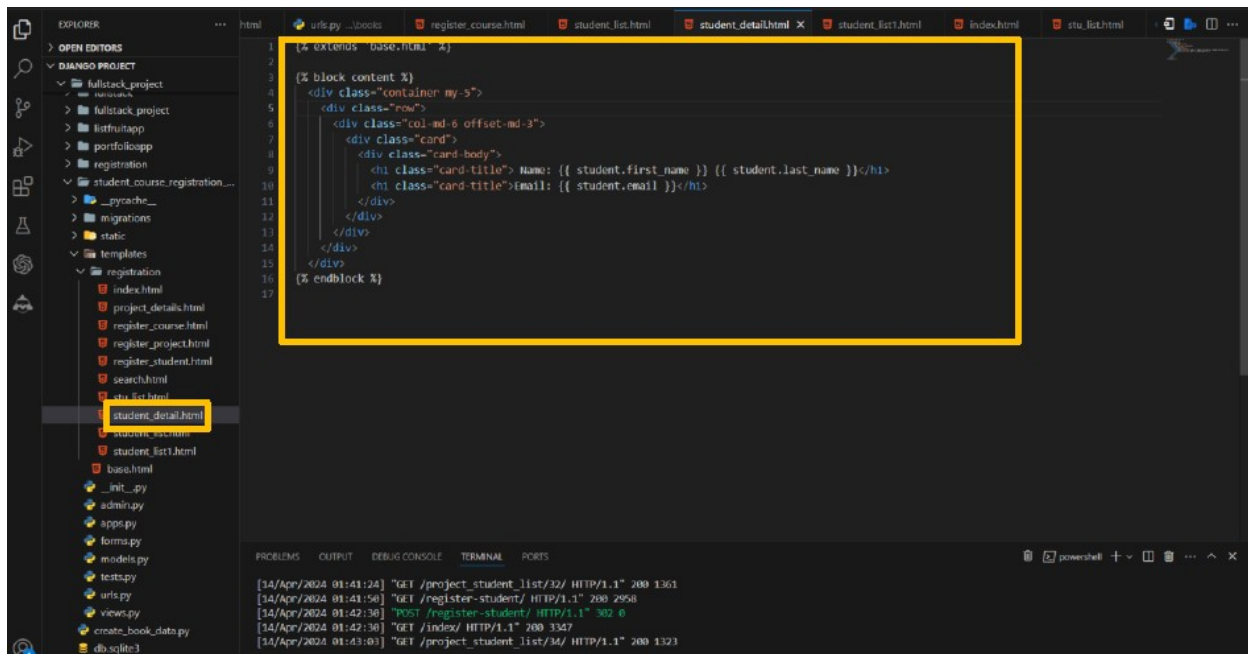
```
</div>
```

```
</div>
```

```

</div>
</div>
{% endblock %}

```

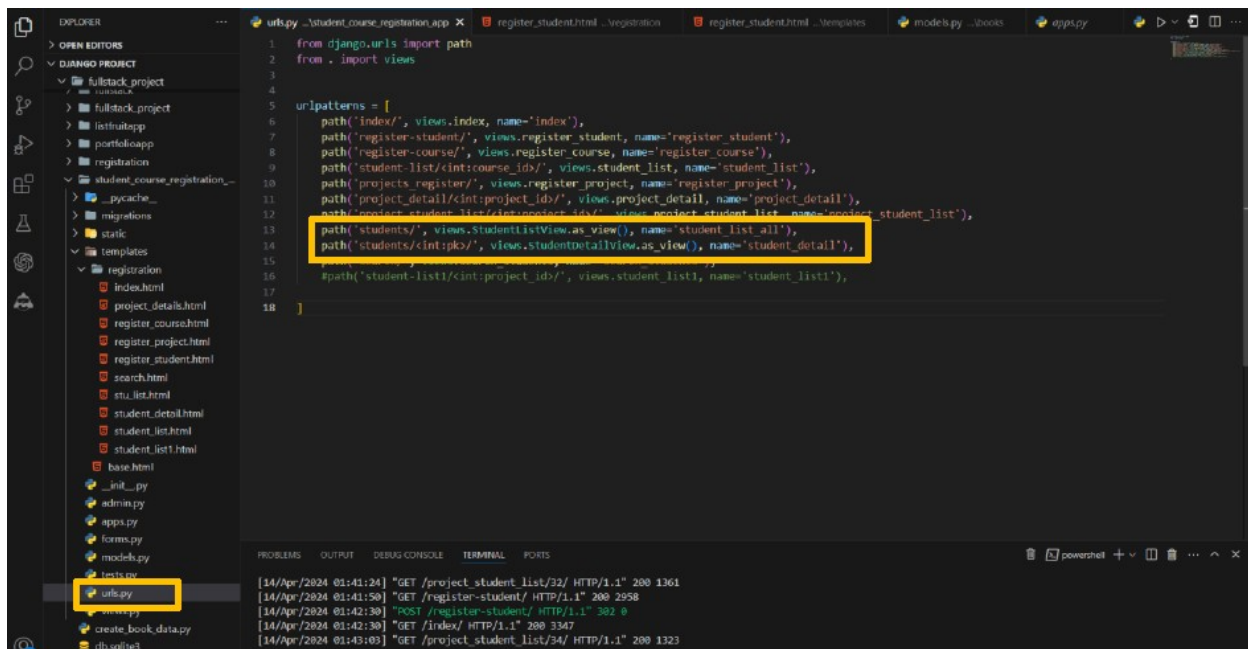


Step-4: add the URL patterns:

```

path('students/', views.StudentListView.as_view(), name='student_list_all'),
path('students/<int:pk>/', views.StudentDetailView.as_view(), name='student_detail'),

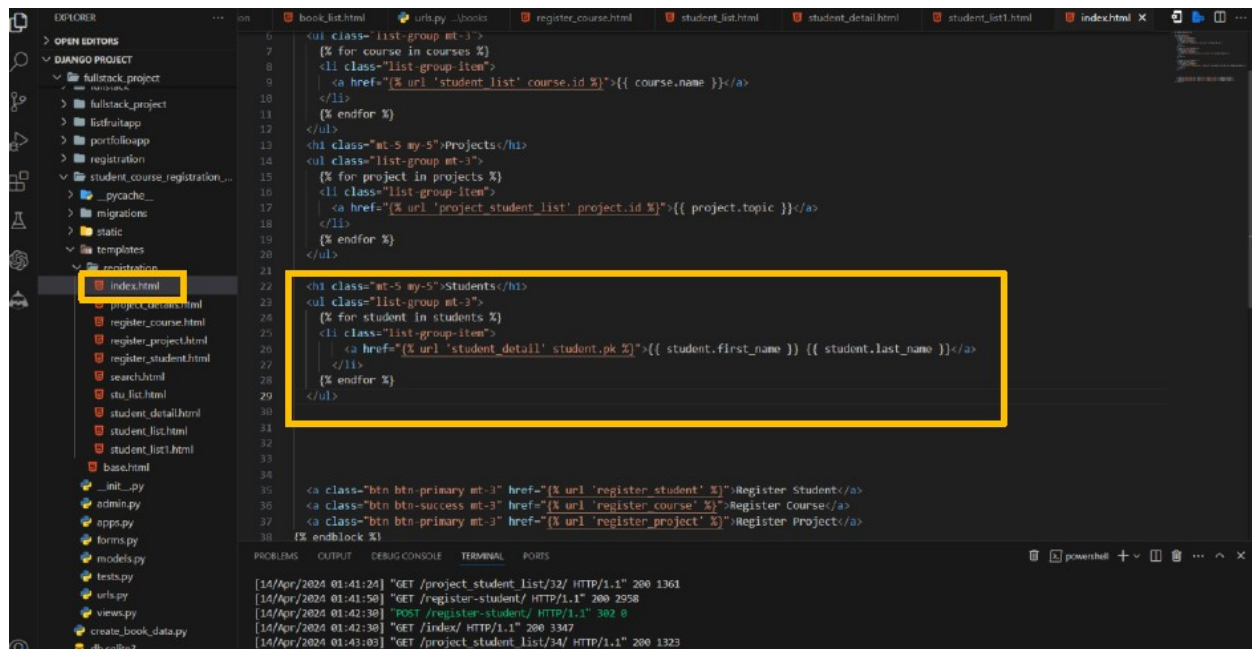
```



Step-5: add below code in Index.html for showing the students in main page:

Index.html

```
<h1 class="mt-5 my-5">Students</h1>
<ul class="list-group mt-3">
  {% for student in students %}
  <li class="list-group-item">
    <a href="{% url 'student_detail' student.pk %}">{{ student.first_name }} {{
      student.last_name }}</a>
  </li>
  {% endfor %}
</ul>
```

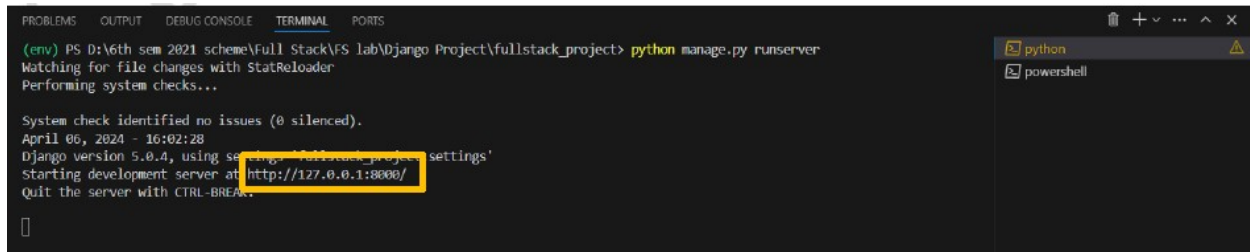


Step-6: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 16:02:28
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[]
```

- Type or copy this **<http://127.0.0.1:8000/index/>**

Final Output By Clicking Register Project and Register Student

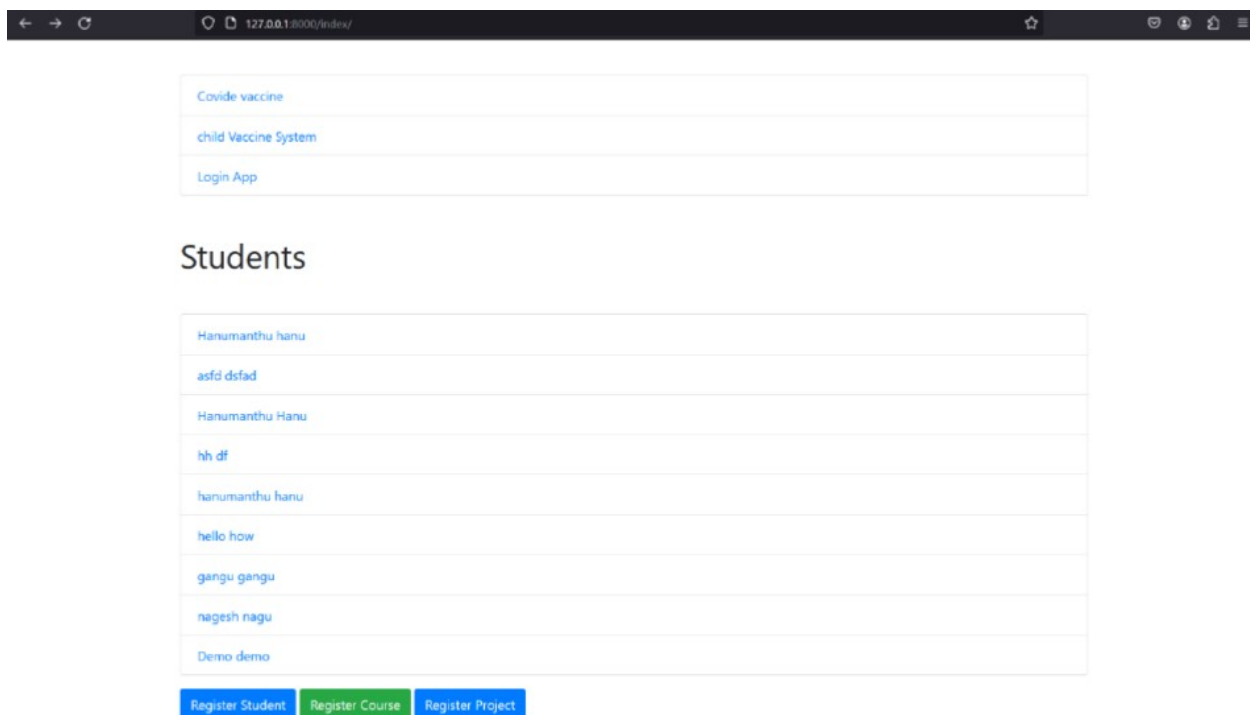


Fig: Main Screen

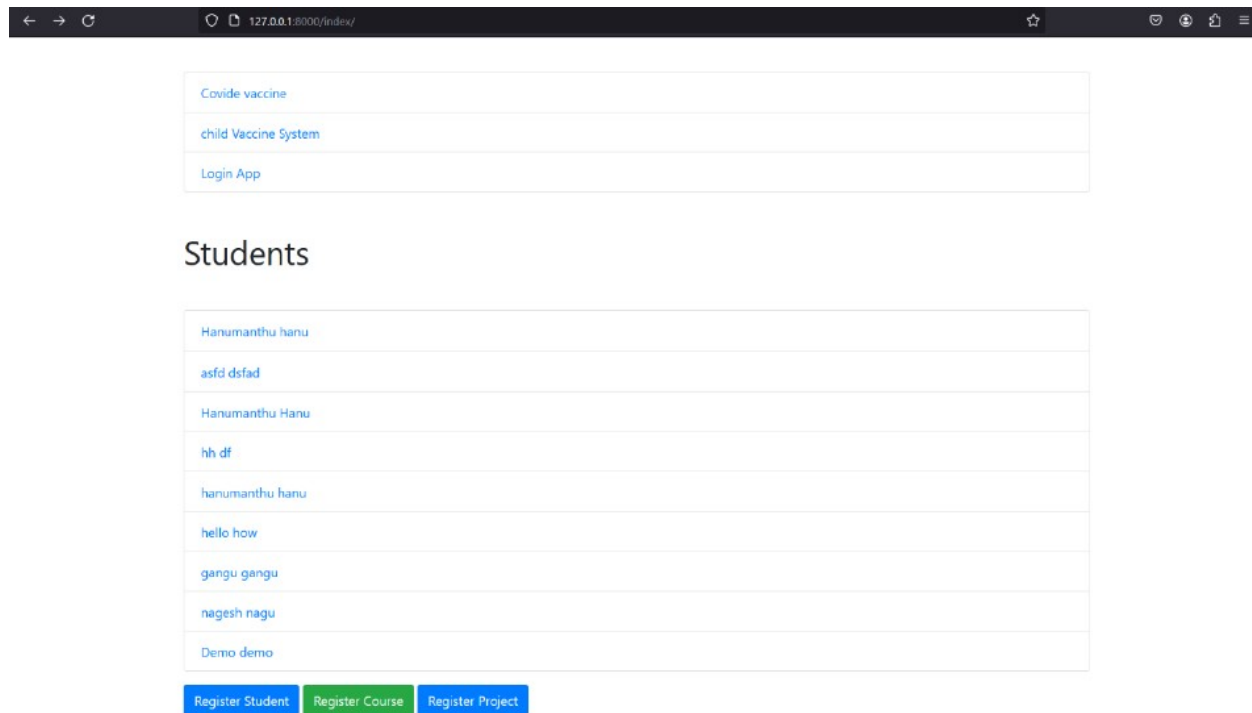


Fig: List View of Students Screen



Fig: Detailed View of Student Screen

Experiment-11

Develop example Django app that performs CSV and PDF generation for any models created in previous laboratory component.

Step-01: This app will be created in the Django project we made earlier.

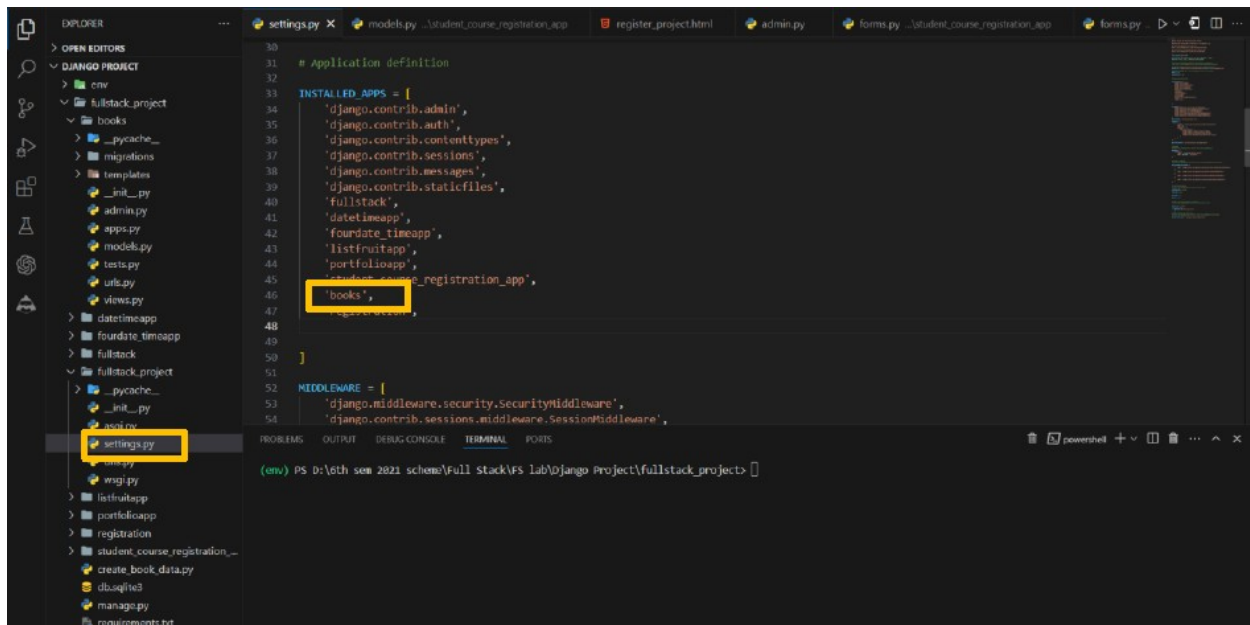


```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS Lab\Django Project\fullstack_project> django-admin startapp books
```

Step-02: Add the app to INSTALLED_APPS

Open the settings.py file in your project's directory (e.g., fullstack_project/settings.py).

Locate the INSTALLED_APPS list and add the name of your new app to the list:



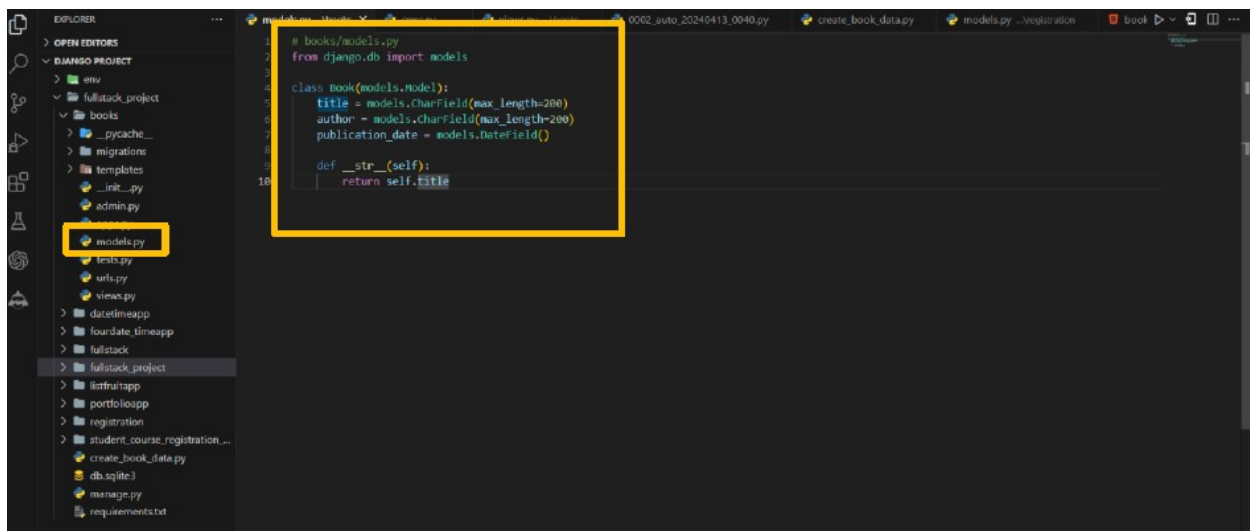
```
30 # Application definition
31
32 INSTALLED_APPS = [
33     'django.contrib.admin',
34     'django.contrib.auth',
35     'django.contrib.contenttypes',
36     'django.contrib.sessions',
37     'django.contrib.messages',
38     'django.contrib.staticfiles',
39     'fullstack',
40     'datetimeapp',
41     'fourdate_timeapp',
42     'listfruitapp',
43     'portfolioapp',
44     'student_course_registration_app',
45     'books',
46 ]
47
48
49
50
51
52 MIDDLEWARE = [
53     'django.middleware.security.SecurityMiddleware',
54     'django.contrib.sessions.middleware.SessionMiddleware',
55 ]
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Step-03: Create models

- Open the models.py file inside the student_course_registration_app and define your models:

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=200)
    publication_date = models.DateField()
    def __str__(self):
        return self.title
```

**Step-04: Create a views.py file in your books app and define the views for generating CSV and PDF files**

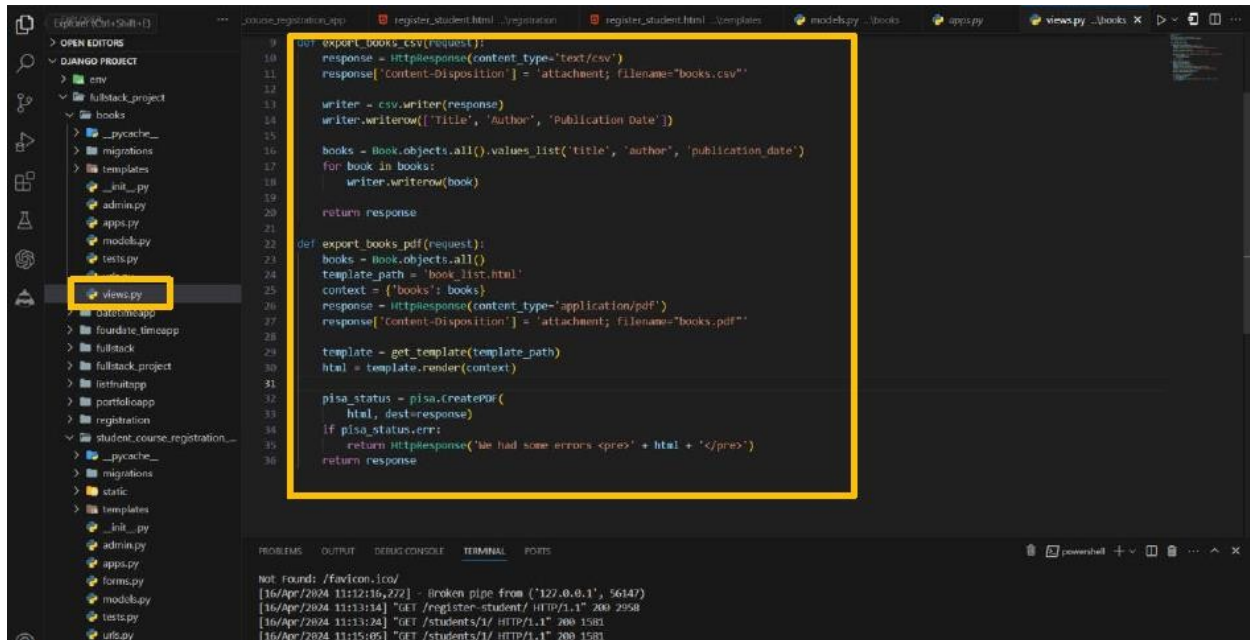
```
import csv

from django.http import HttpResponse
from django.template.loader import get_template
from xhtml2pdf import pisa

from .models import Book
```

```
def export_books_csv(request):  
    response = HttpResponse(content_type='text/csv')  
    response['Content-Disposition'] = 'attachment; filename="books.csv"'  
    writer = csv.writer(response)  
    writer.writerow(['Title', 'Author', 'Publication Date'])  
    books = Book.objects.all().values_list('title', 'author', 'publication_date')  
    for book in books:  
        writer.writerow(book)  
    return response
```

```
def export_books_pdf(request):  
    books = Book.objects.all()  
    template_path = 'book_list.html'  
    context = {'books': books}  
    response = HttpResponse(content_type='application/pdf')  
    response['Content-Disposition'] = 'attachment; filename="books.pdf"'  
    template = get_template(template_path)  
    html = template.render(context)  
    pisa_status = pisa.CreatePDF(html, dest=response)  
    if pisa_status.err:  
        return HttpResponse('We had some errors <pre>' + html + '</pre>')  
    return response
```



Step-04: In your books app, create a templates directory and an html file for rendering the PDF: Path: books/ templates/ books/ book_list.html

book_list.html

<!DOCTYPE html>

<html>

<head>

<title>Book List</title>

<style>

body {

font-family: Arial, sans-serif;

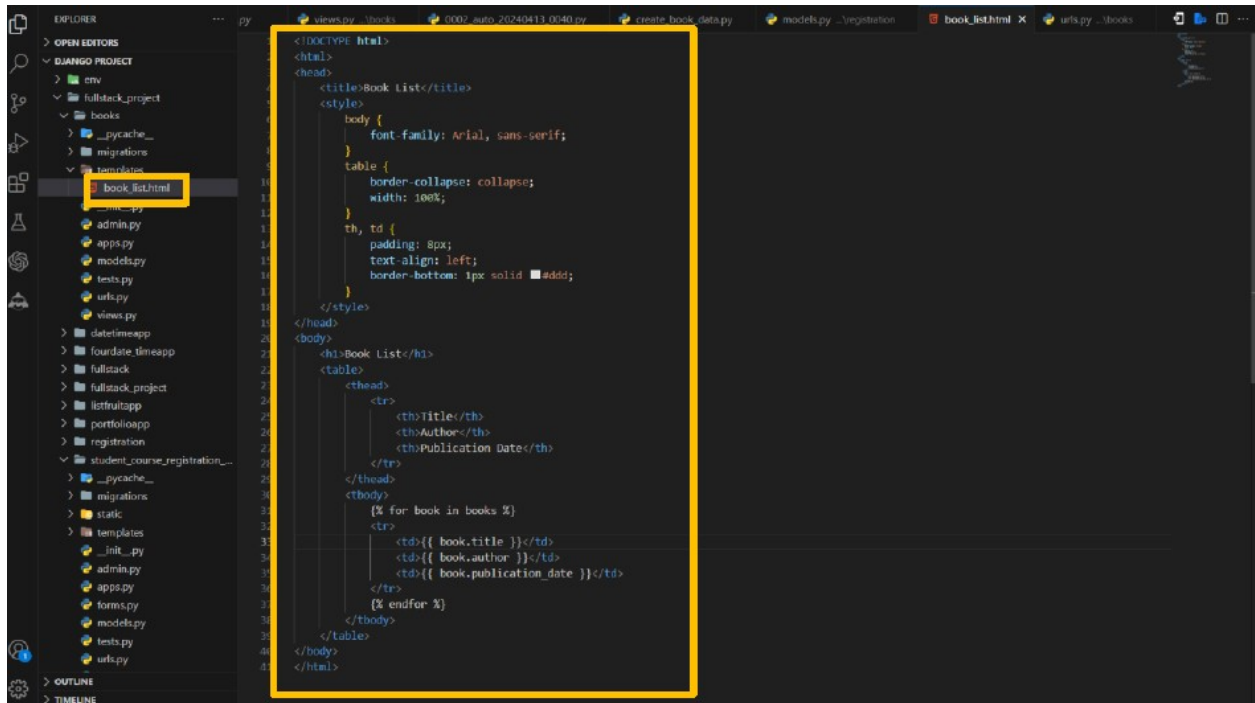
}

table {

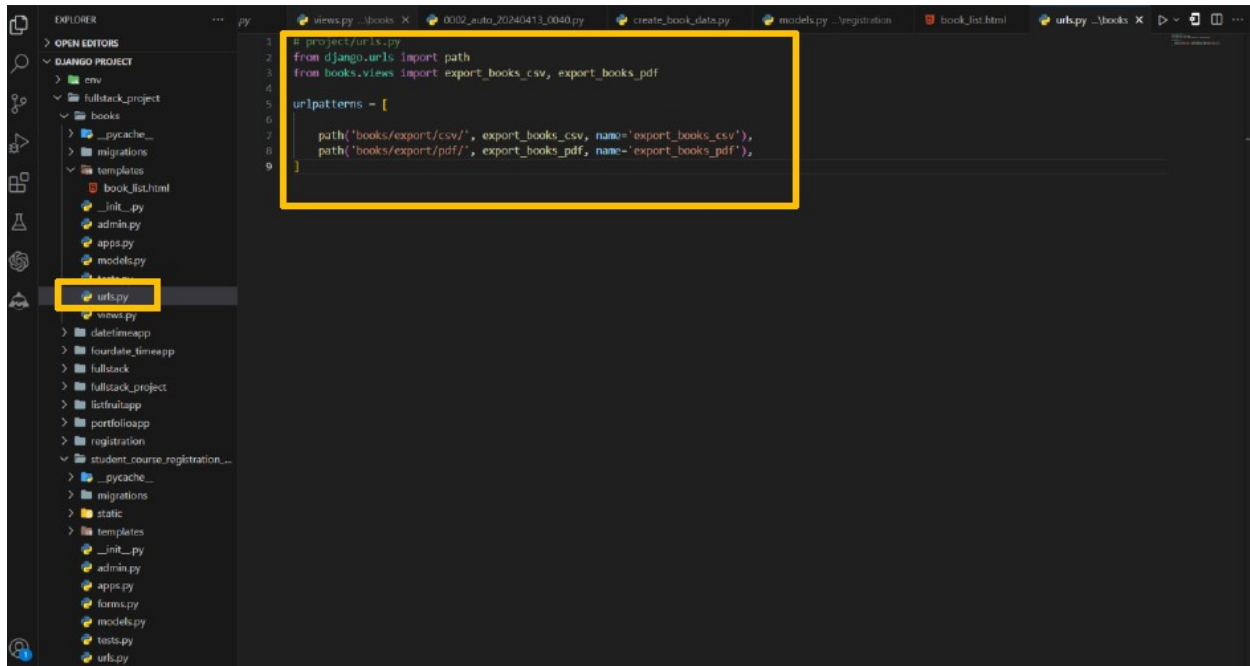
border-collapse: collapse;

width: 100%;

```
    }
    th, td {
        padding: 8px;
        text-align: left;
        border-bottom: 1px solid #ddd;
    }
</style>
</head>
<body>
    <h1>Book List</h1>
    <table>
        <thead>
            <tr>
                <th>Title</th>
                <th>Author</th>
                <th>Publication Date</th>
            </tr>
        </thead>
        <tbody>
            {% for book in books %}
            <tr>
                <td>{{ book.title }}</td>
                <td>{{ book.author }}</td>
                <td>{{ book.publication_date }}</td>
            </tr>
```

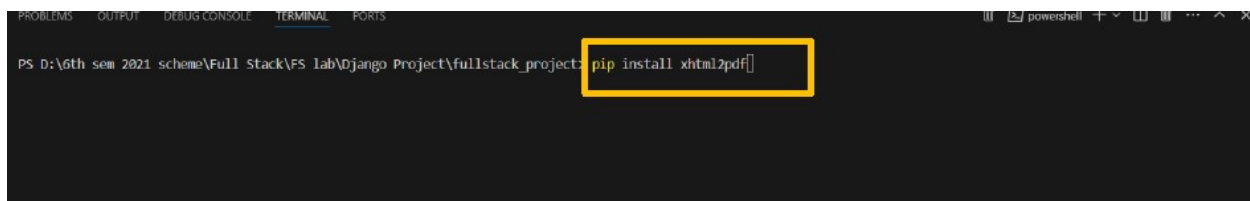


]



Step-06: Install the xhtml2pdf library for generating PDF files:

pip install xhtml2pdf



Step-07: Custom Script

- If you need to create the book data programmatically (e.g., during deployment or as part of a data import process), you can create a custom Python script and run it as needed.
- Create a new Python file (e.g., create_book_data.py) in your project's root directory, and add the code to create the book objects:

```
import os

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'fullstack_project.settings')

import django
django.setup()

from books.models import Book

Book.objects.bulk_create([

    Book(title='To Kill a Mockingbird', author='Harper Lee', publication_date='1960-07-11'),

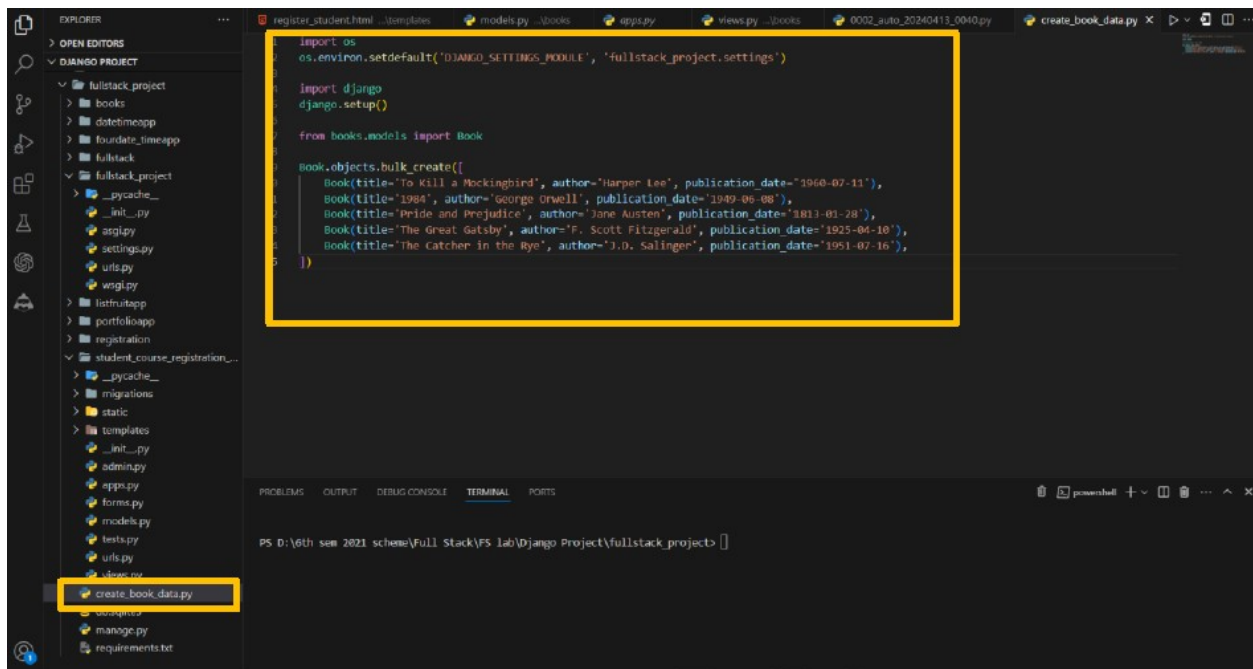
    Book(title='1984', author='George Orwell', publication_date='1949-06-08'),

    Book(title='Pride and Prejudice', author='Jane Austen', publication_date='1813-01-28'),

    Book(title='The Great Gatsby', author='F. Scott Fitzgerald', publication_date='1925-04-10'),

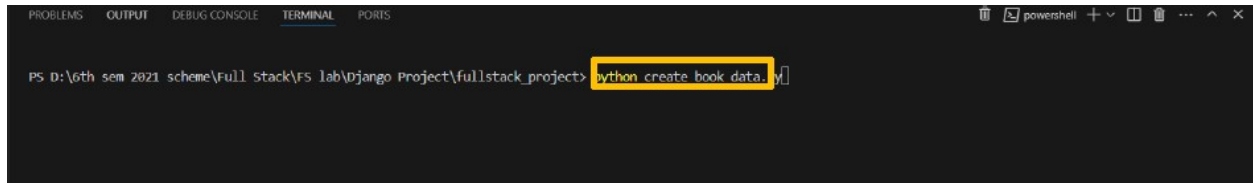
    Book(title='The Catcher in the Rye', author='J.D. Salinger', publication_date='1951-07-16'),

])
```



Run the script using the following command:

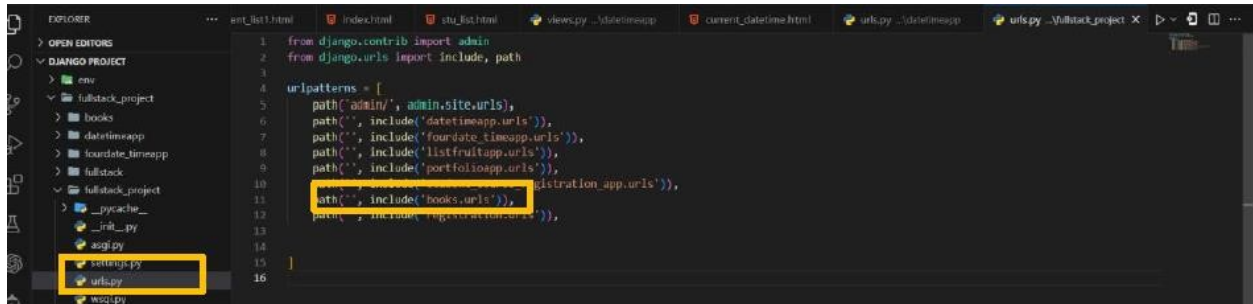
```
python create_book_data.py
```



```
PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python create_book_data.py
```

Step-08: Update project URLs

- Open the urls.py file inside your project and include the URLs from the book app:



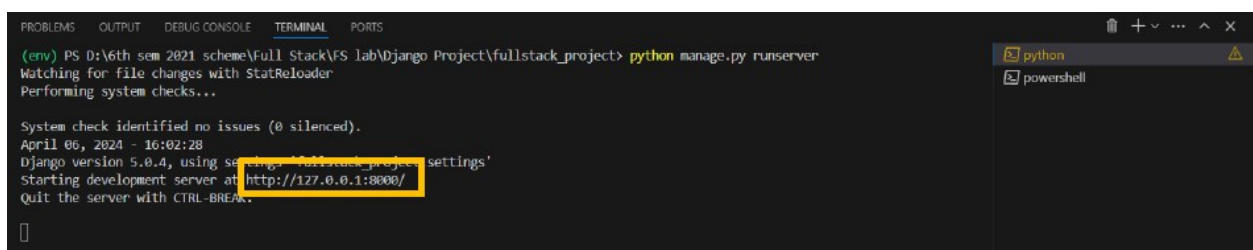
```
1 from django.contrib import admin
2 from django.urls import include, path
3
4 urlpatterns = [
5     path('admin/', admin.site.urls),
6     path('', include('datetimeapp.urls')),
7     path('', include('fourdate_timeapp.urls')),
8     path('', include('listfruitapp.urls')),
9     path('', include('portfolioapp.urls')),
10    path('', include('registration_app.urls')),
11    path('', include('books.urls')),
12    path('', include('export_csv.urls')),
13 ]
```

Step-9: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

`python manage.py runserver`

- Open your web browser and visit `http://127.0.0.1:8000/`.



```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 16:02:28
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[]
```

- Type or copy this `http://127.0.0.1:8000/books/export/csv/`

- Type or copy this <http://127.0.0.1:8000/books/export/csv/>



Fig: CSV File Download Screen

A screenshot of a Microsoft Excel spreadsheet. The spreadsheet has three columns: 'Title', 'Author', and 'Publication Date'. The data is as follows:

Title	Author	Publication Date
To Kill a Mockingbird	Harper Lee	11-07-1960
1984	George Orwell	06-06-1949
Pride and Prejudice	Jane Austen	1813-01-28
The Great Gatsby	F. Scott Fitzgerald	10-04-1925
The Catcher in the Rye	J.D. Salinger	16-07-1951
To Kill a Mockingbird	Harper Lee	11-07-1960
1984	George Orwell	06-06-1949
Pride and Prejudice	Jane Austen	1813-01-28
The Great Gatsby	F. Scott Fitzgerald	10-04-1925
The Catcher in the Rye	J.D. Salinger	16-07-1951

- Type or copy this <http://127.0.0.1:8000/books/export/pdf/>

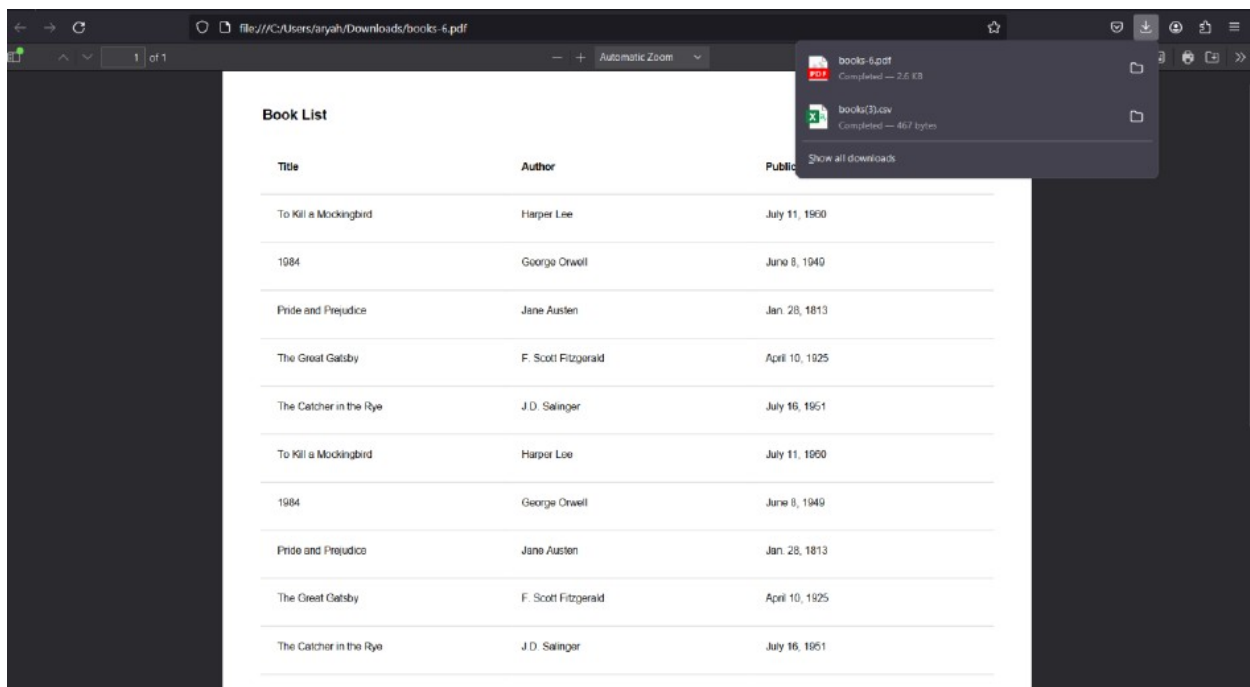
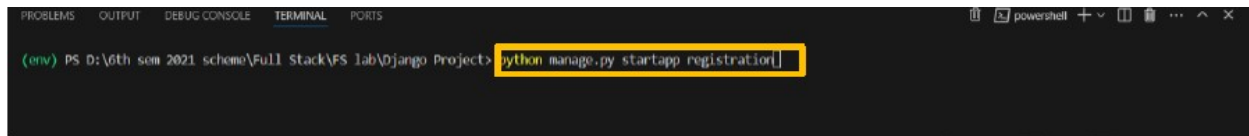


Fig: PDF File Download Screen

Experiment-12

Develop a registration page for student enrolment as done in Module 2 but without page refresh using AJAX.

Step-01: This app (registration) will be created in the Django project we made earlier.

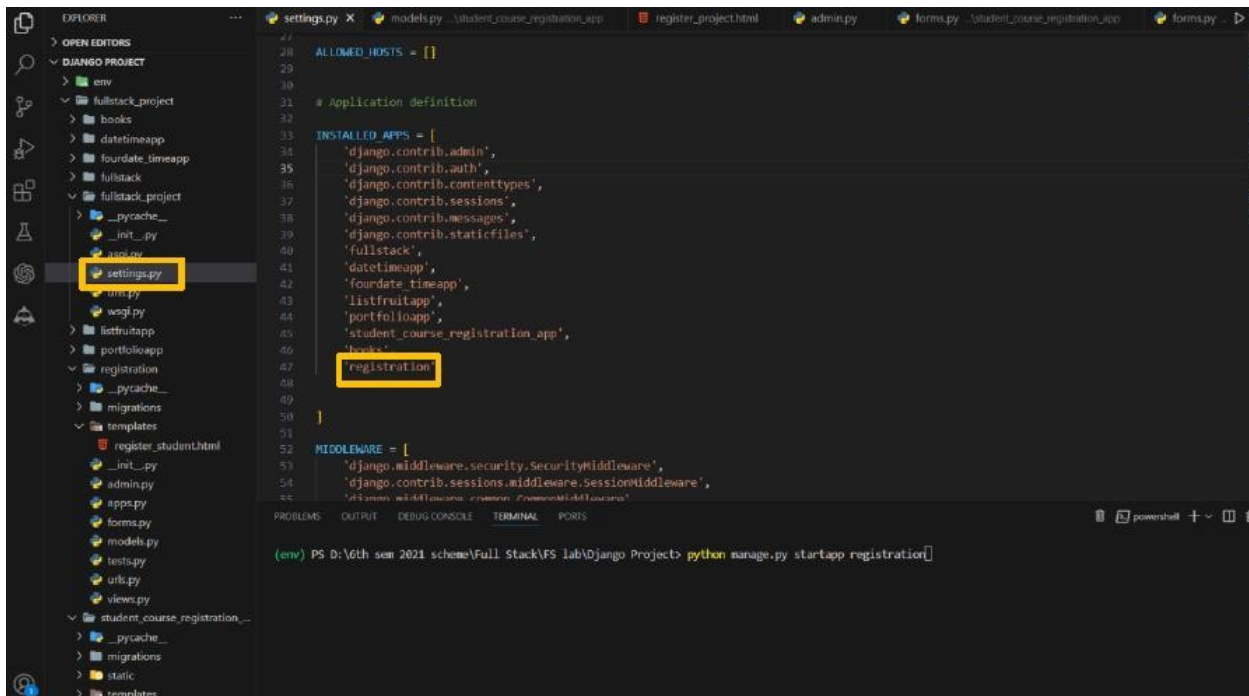


```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project> python manage.py startapp registration
```

Step-02: Add the app to INSTALLED_APPS

Open the settings.py file in your project's directory (e.g., fullstack_project/settings.py).

Locate the INSTALLED_APPS list and add the name of your new app to the list:



```
24 ALLOWED_HOSTS = []
25
26
27 # Application definition
28
29 INSTALLED_APPS = [
30     'django.contrib.admin',
31     'django.contrib.auth',
32     'django.contrib.contenttypes',
33     'django.contrib.sessions',
34     'django.contrib.messages',
35     'django.contrib.staticfiles',
36     'fullstack',
37     'datetimeapp',
38     'fourdate_timeapp',
39     'listfruitapp',
40     'portfolioapp',
41     'student_course_registration_app',
42     'books',
43     'registration'
44 ]
45
46 MIDDLEWARE = [
47     'django.middleware.security.SecurityMiddleware',
48     'django.contrib.sessions.middleware.SessionMiddleware',
49     'django.middleware.common.CommonMiddleware',
50     'django.middleware.csrf.CsrfViewMiddleware',
51 ]
```

Step-03: Create models

- **Open the models.py file inside the registration app and define your models:**

```
from django.db import models
```

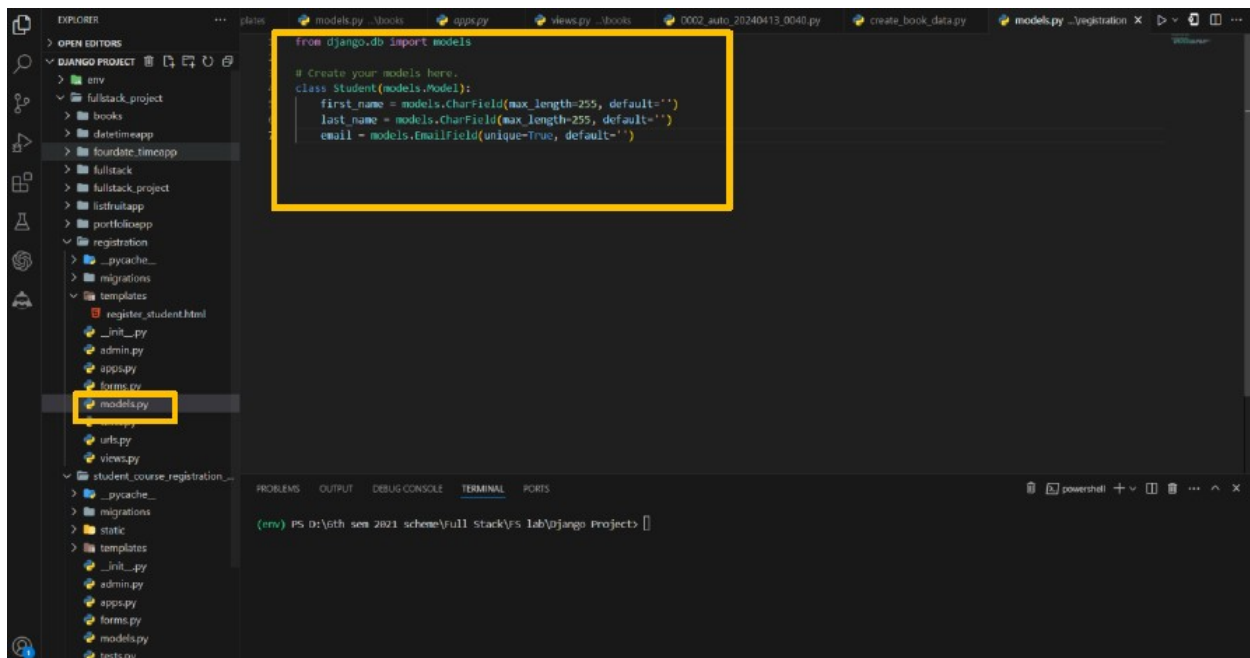
```
# Create your models here.
```

```
class Student(models.Model):
```

```
    first_name = models.CharField(max_length=255, default="")
```

```
    last_name = models.CharField(max_length=255, default="")
```

```
    email = models.EmailField(unique=True, default="")
```

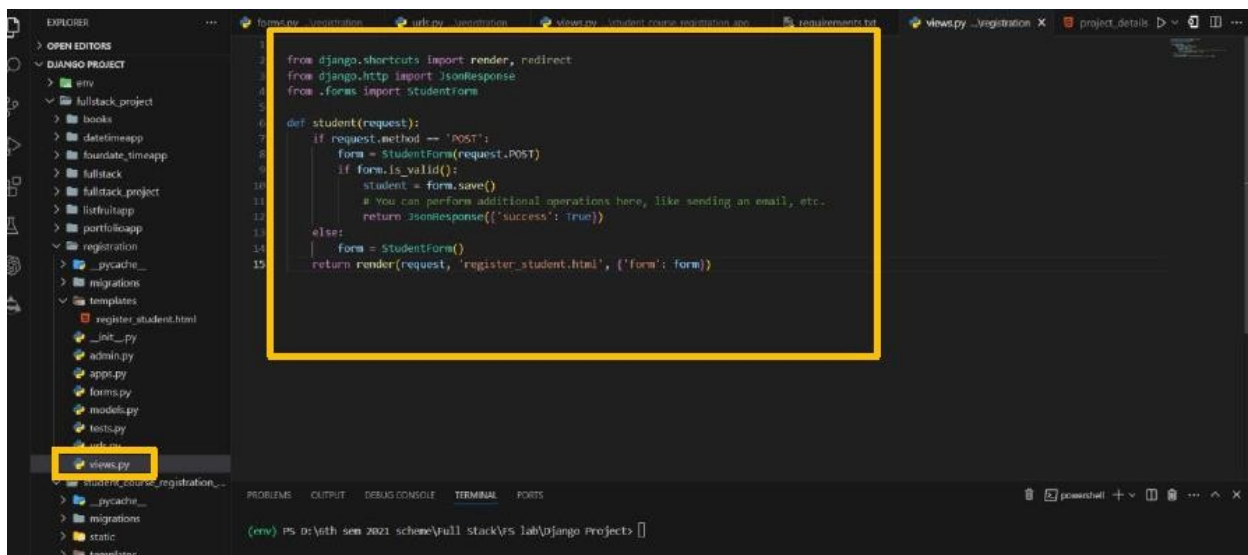
**Step-04: Create a views.py file in your app and define the views**

```
from django.shortcuts import render, redirect
```

```
from django.http import JsonResponse
```

```
from .forms import StudentForm
```

```
def student(request):  
  
    if request.method == 'POST':  
  
        form = StudentForm(request.POST)  
  
        if form.is_valid():  
  
            student = form.save()  
  
            # You can perform additional operations here, like sending an email, etc.  
  
            return JsonResponse({'success': True})  
  
        else:  
  
            form = StudentForm()  
  
        return render(request, 'register_student.html', {'form': form})
```



Step-05: Create a forms.py file in your app and define the forms

```
from django import forms  
  
from .models import Student  
  
class StudentForm(forms.ModelForm):  
  
    class Meta:  
  
        model = Student
```

```

fields = ['first_name', 'last_name', 'email']

widgets = {

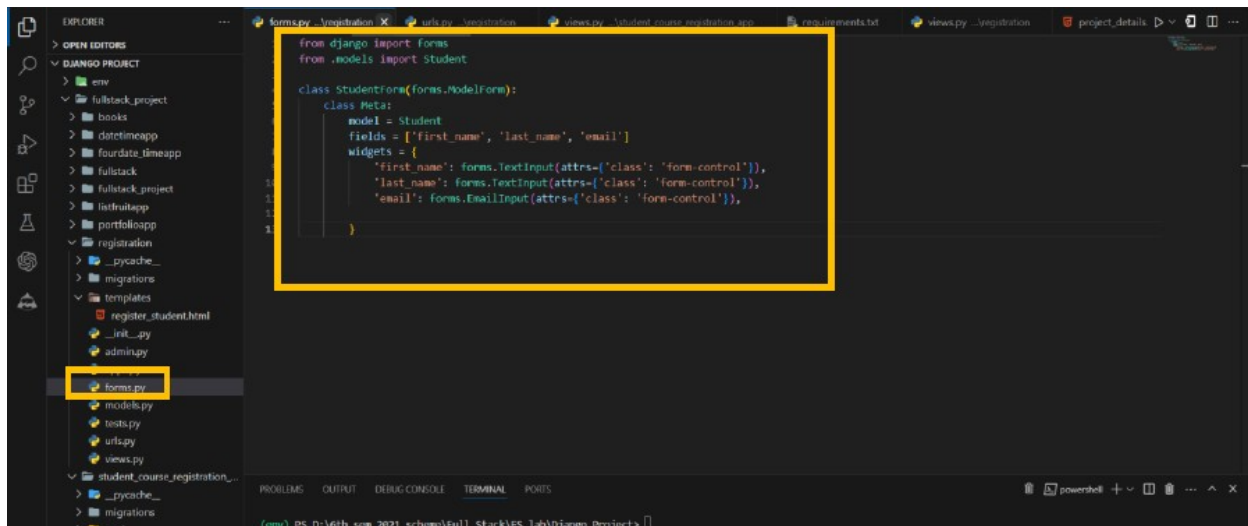
    'first_name': forms.TextInput(attrs={'class': 'form-control'}),

    'last_name': forms.TextInput(attrs={'class': 'form-control'}),

    'email': forms.EmailInput(attrs={'class': 'form-control'}),

}

```



Step-06: In your app, create a templates directory and an html file

register_student.html

```
{% load static %}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Student Registration</title>
```

```
<link
                                rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css">
```

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script> </head>
```

```
<body>
```

```

<div class="container">

    <h1>Student Registration</h1>

    <form id="student-form" method="post">

        {% csrf_token %}

        {% for field in form %}

            <div class="form-group">

                {{ field.label_tag }}

                {{ field }}

                {% if field.help_text %}

                    <small class="form-text text-muted">{{
                        field.help_text }}</small>

                {% endif %}

                {% for error in field.errors %}

                    <div class="alert alert-danger">{{ error }}</div>

                {% endfor %}

            </div>

        {% endfor %}

        <button type="submit" class="btn btn-primary">Submit</button>

    </form>

    <div id="response"></div>

</div>

<script>

$(document).ready(function() {

    alert("Page Loaded");

    $(document).on('submit', '#student-form', function(e) {

        e.preventDefault();

```

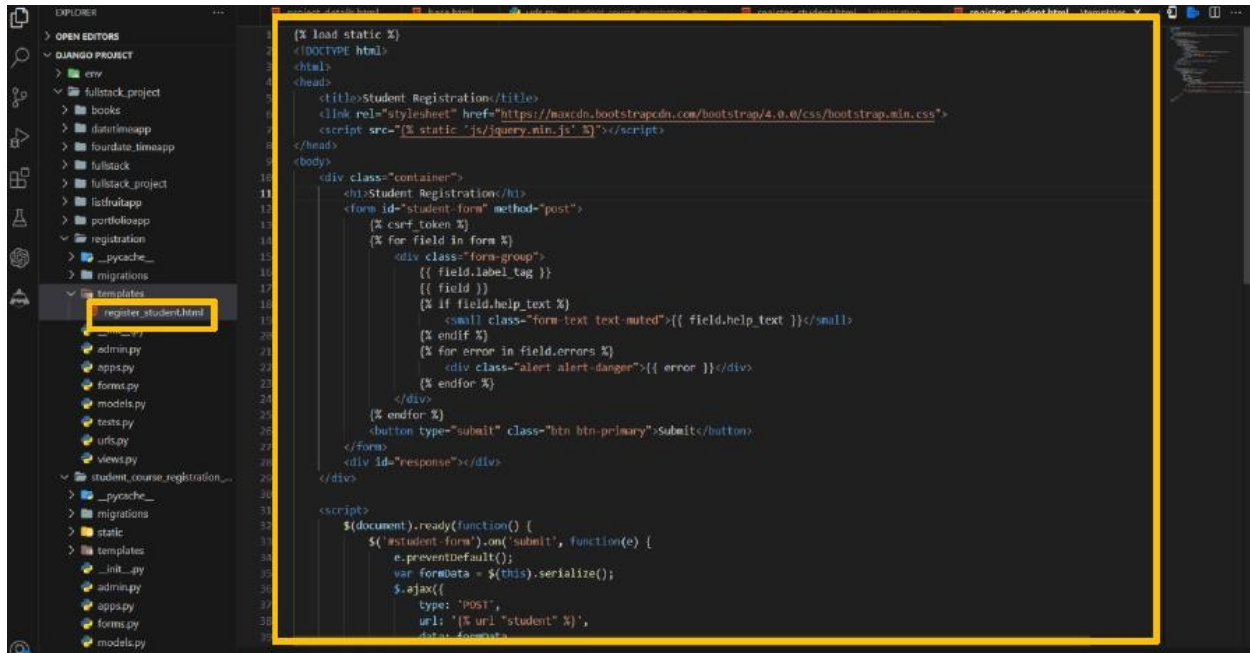
```
        alert("Submit handler running");

        $.ajax({
            type: 'POST',
            url: '/student/',
            data: $(this).serialize(),
            success: function(response) {
                alert("AJAX Success");
                if (response.success) {
                    $('#response').html(
                        '<div class="alert alert-success">Student registered successfully.</div>'
                    );
                } else {
                    $('#response').html(
                        '<div class="alert alert-danger">Failed</div>'
                    );
                }
            }
        });
    });

</script>

</body>

</html>
```



Step-07: Map the view function to a URL

- Open the urls.py file in your Django app's directory (e.g., registration/urls.py).
- Import the view function at the top of the file
- Add a new URL pattern to the urlpatterns list

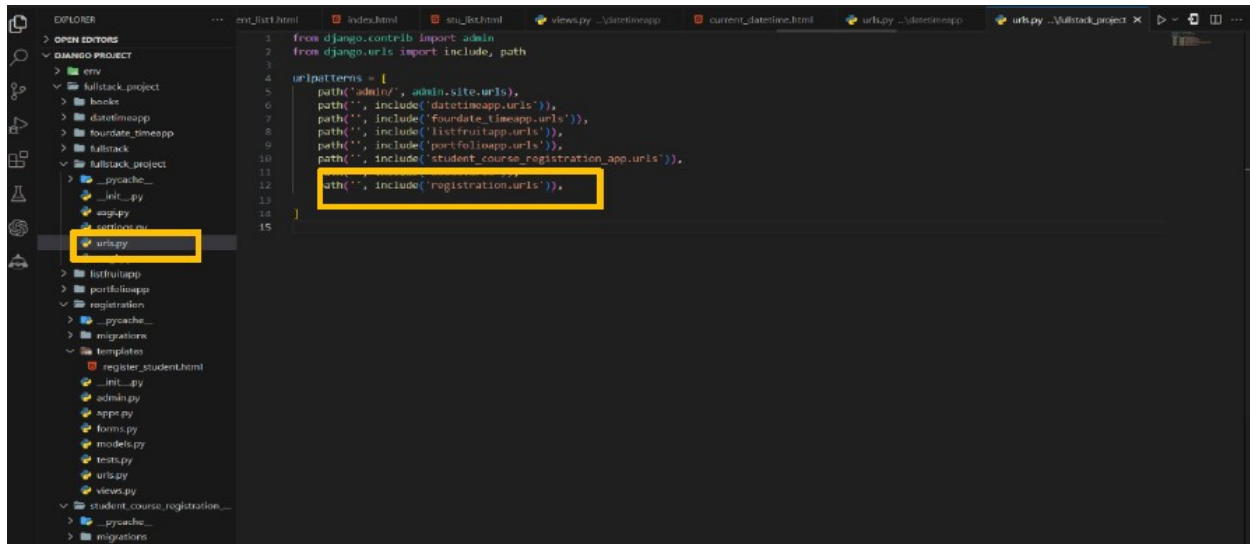
from django.urls import path

from registration.views import student

```
urlpatterns = [
    path('student/', student, name='student'),
]
```

Step-08: Include the app's URLs in the project's URL patterns

- Open the **urls.py** file inside your project and include the URLs from the registration app:

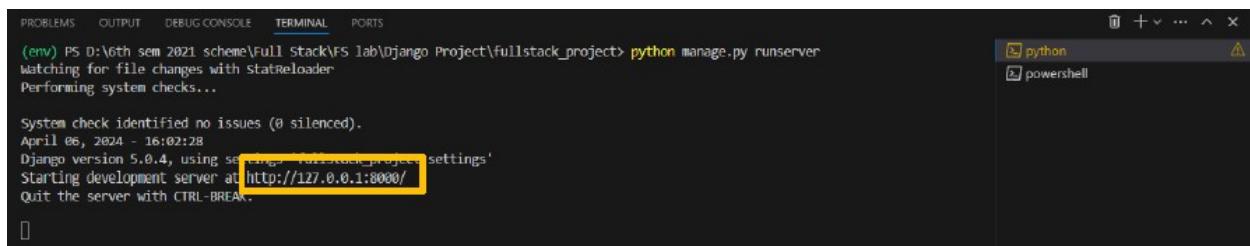


Step-09: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.

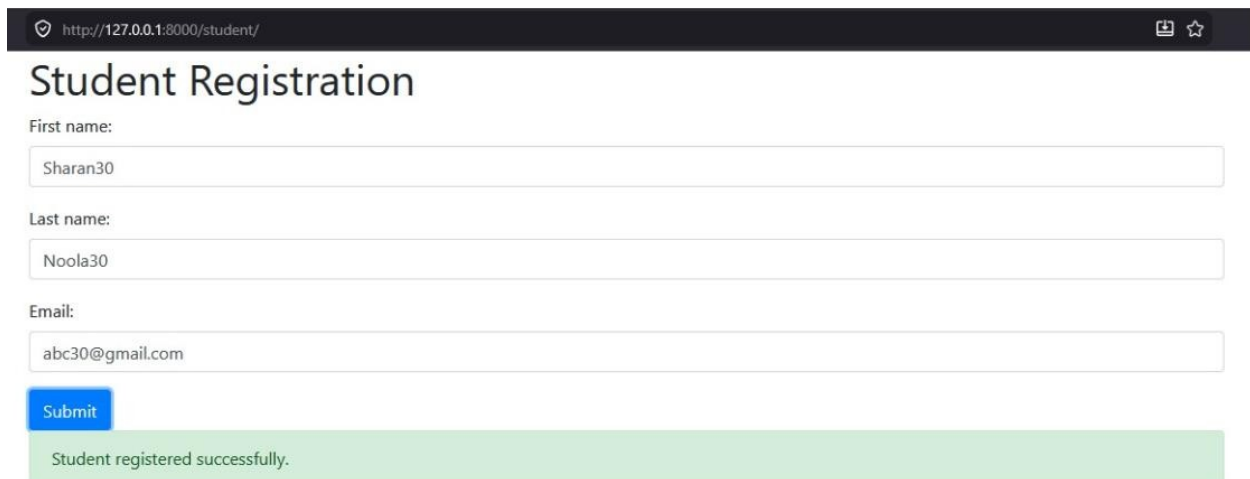


- Type or copy this **http://127.0.0.1:8000/student/**



A screenshot of a web browser showing the 'Student Registration' page. The browser's address bar displays '127.0.0.1:8000/student/'. The page has a title 'Student Registration' and three input fields labeled 'First name:', 'Last name:', and 'Email:'. Each field is empty. Below the fields is a blue 'Submit' button.

Fig: Student Registration Screen



A screenshot of the same 'Student Registration' page, but now the input fields are filled with the text 'Sharan30', 'Noola30', and 'abc30@gmail.com'. The blue 'Submit' button is still present. Below the form, a green message box displays the text 'Student registered successfully.'

Fig: Student Registered successfully